Planeación de acciones para robots de servicio mediante modelos de lenguaje y sistemas basados en reglas

Planeación de acciones para robots de servicio a partir de instrucciones en lenguaje natural mediante Qwen, CLIPS y modelos GPT

Planeación de acciones para robots de servicio a partir de instrucciones en lenguaje natural: comparación de una arquitectura neuro-simbólica y un modelo de lenguaje generativo

Planeación de acciones para robots de servicio mediante modelos de lenguaje y reglas de producción

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción **1**

1.1. Contexto y motivación 2
1.2. Planteamiento del problema 3
1.3. Alcance y delimitaciones 4
1.4. Hipótesis 5
1.5. Objetivos 5
1.5.1. Objetivo general 5
1.5.2. Objetivos específicos 6
1.6. Contribuciones 6
1.7. Estructura del documento 7

2. Antecedentes y estado del arte **8**

2.1. Sistemas basados en reglas 8
2.2. CLIPS como sistema de producción 9
2.3. Modelos de lenguaje para instrucciones robóticas 10
2.3.1. Familias Qwen y GPT 10
2.3.2. Adaptación eficiente: LoRA y QLoRA 11
2.3.3. Salidas estructuradas y validación 11
2.3.4. Limitaciones para la planificación robótica 12
2.4. Arquitecturas neuro-simbólicas 12
2.5. Trabajos relacionados con lenguaje y planificación 12
2.5.1. Razonamiento semántico en ViRBot y Justina 12
2.5.2. SayCan: selección de habilidades disponibles 13
2.5.3. Code as Policies: generación de programas 13
2.5.4. LLM+P: formalización y planificación clásica 14
2.5.5. Comparación de los enfoques 14

2.6. Síntesis y conexión con la propuesta 15
2.6.1. Brecha de investigación 15

3. Formalización del problema y arquitecturas conceptuales 16
3.1. Contexto arquitectónico y alcance del sistema 16
3.1.1. Integración dentro de la arquitectura ViRBot 16
3.2. GPSR como dominio de referencia 17
3.2.1. La prueba General Purpose Service Robot 17
3.2.2. Generación de comandos y conocimiento del dominio 18
3.2.3. Dominio de referencia y alcance lingüístico 18
3.3. Formalización de órdenes, objetivos y planes 19
3.3.1. Información del dominio y condiciones de planificación 20
3.3.2. Orden en lenguaje natural 21
3.3.3. Equivalencia semántica entre formulaciones 21
3.3.4. Entidades y conocimiento del dominio 22
3.3.5. Representación canónica de objetivos 22
3.3.6. Objetivos semánticos y acciones ejecutables 24
3.3.7. Ejemplo de transformación entre representaciones 24
3.3.8. Dependencias y contexto entre objetivos 24
3.4. Arquitecturas conceptuales evaluadas 26
3.4.1. Base común de las arquitecturas evaluadas 26
3.4.2. Arquitectura Qwen–CLIPS 27
3.4.3. Arquitectura de planificación directa con GPT 29
3.4.4. Comparación conceptual de las arquitecturas 31
3.5. Criterios conceptuales de validez 31
3.5.1. Niveles de validez 33
3.5.2. Validez de la interpretación semántica 33
3.5.3. Coherencia del plan y cumplimiento de la tarea 35
3.5.4. Robustez, consistencia y trazabilidad 35
3.5.5. Validez de la respuesta del sistema 36
3.5.6. Condición de control para la etapa simbólica 36
3.5.7. Planificación y ejecución física 37

4. Metodología y diseño de las arquitecturas evaluadas 38
4.1. Diseño metodológico de la investigación 39

4.1.1. Problema experimental 39
4.1.2. Variables y condiciones de comparación 39
4.1.3. Estrategia general de evaluación 40
4.2. Construcción del dominio experimental 41
4.2.1. Fuentes de conocimiento GPSR 41
4.2.2. Catálogos de personas, objetos y ubicaciones 41
4.2.3. Familias de tareas 42
4.2.4. Unidades de organización y análisis 43
4.2.5. Etiquetas semánticas para entrenamiento 43
4.2.6. Referencias para la evaluación comparativa 44
4.3. Construcción del conjunto de datos 44
4.3.1. Adaptación del CommandGenerator 45
4.3.2. Generación de órdenes 47
4.3.3. Generación de objetivos canónicos 48
4.3.4. Variación lingüística 48
4.3.5. Balance y deduplicación 49
4.3.6. Particiones de ajuste y separación de la prueba final 49
4.3.7. Validación previa del conjunto de datos 51
4.4. Diseño de la arquitectura Qwen–CLIPS 52
4.4.1. Función asignada a Qwen 52
4.4.2. Especialización para interpretación semántica 52
4.4.3. Interfaz entre objetivos y planificación simbólica 53
4.4.4. Función asignada a CLIPS 54
4.5. Diseño de la arquitectura GPT directo 55
4.5.1. Funciones asignadas al modelo 55
4.5.2. Conocimiento y restricciones proporcionadas 55
4.5.3. Contrato de salida 55
4.6. Diseño de la comparación 56
4.6.1. Igualdad del dominio de evaluación 56
4.6.2. Igualdad del repertorio de acciones 57
4.6.3. Separación entre interpretación y planificación 57
4.6.4. Registro previo y límites de la comparación 58

5. Implementación **59**
5.1. Plataforma experimental 60
5.1.1. Robot Justina 60
5.1.2. Plataforma de cómputo 60
5.1.3. Entorno de software y ROS 2 61
5.2. Implementación del conjunto de datos 61
5.2.1. Generador de órdenes 61
5.2.2. Catálogos y CompetitionTemplate 62
5.2.3. Validadores 63
5.2.4. Corpus de entrenamiento 64
5.3. Entrenamiento e inferencia Qwen 65
5.3.1. Preparación del modelo 65
5.3.2. Configuración QLoRA 65
5.3.3. Hiperparámetros de entrenamiento 66
5.3.4. Selección y conservación del checkpoint 66
5.3.5. Inferencia 68
5.3.6. Despliegue del modelo 70
5.4. Implementación del planificador CLIPS 71
5.4.1. Conversión de objetivos a hechos gpsr-goal 71
5.4.2. Representación del estado simbólico 73
5.4.3. Reglas de expansión 74
5.4.4. Generación de acciones 76
5.4.5. Finalización y organización del diálogo 76
5.5. Integración Qwen–CLIPS con Justina 77
5.5.1. Componentes ROS 2 77
5.5.2. Comunicación y entrega del plan 79
5.5.3. Extracción de entidades durante la interacción 79
5.5.4. Alcance de las comprobaciones de integración 80
5.6. Ejemplo de funcionamiento Qwen–CLIPS 80
5.6.1. De la instrucción a los hechos simbólicos 81
5.6.2. Aplicación de reglas y evolución del estado 82
5.6.3. Finalización y construcción de la salida 82
5.6.4. Plan resultante 82
5.7. Instrumento de campañas experimentales 84

5.7.1. Preparación y conservación de la configuración 84
5.7.2. Recorrido Qwen–CLIPS y control auxiliar 85
5.7.3. Productos y tratamiento de incidencias 85
5.8. Implementación del planificador GPT 86
5.8.1. Interfaz con la API 86
5.8.2. Instrucciones del sistema 87
5.8.3. JSON Schema y Structured Outputs 88
5.8.4. Generación de objetivos y planes 88
5.8.5. Tratamiento de aclaraciones y rechazos 89
5.8.6. Validación determinista 89
5.9. Reproducibilidad y alcance de los registros 90
5.9.1. Disponibilidad del código 90
5.9.2. Identidad de modelos, datos y recursos 90
5.9.3. Entornos y configuración efectiva 90
5.9.4. Tiempos, recursos y conservación de evidencia 91

6. Diseño experimental y criterios de evaluación 93
6.1. Objetivo y diseño de la comparación 93
6.2. Conjuntos de desarrollo y evaluación final 94
6.2.1. Antecedentes de desarrollo 94
6.2.2. Bloques del conjunto final propuesto 95
6.2.3. Fichas, referencias y auditoría de cobertura 96
6.3. Configuraciones y condiciones experimentales 98
6.3.1. Control auxiliar con objetivos de referencia 98
6.4. Procedimiento de ejecución y revisión 99
6.4.1. Preparación y secuencia de la campaña 99
6.4.2. Fallos, tiempos límite y recuperación 100
6.4.3. Revisión semántica y procedencia de los dictámenes 100
6.5. Criterios y métricas de evaluación 101
6.5.1. Dictámenes y cumplimiento principal 101
6.5.2. Interpretación y coherencia interna del plan 102
6.5.3. Condiciones de escenario y límites de capacidad 103
6.5.4. Robustez y consistencia entre ejecuciones 103
6.5.5. Clasificación y diagnóstico de errores 104
6.5.6. Latencia, recursos y costo observable 104

6.6. Análisis estadístico y presentación de resultados 105
6.7. Controles y alcance de las conclusiones 106

7. Resultados **107**
7.1. Resultados de la especialización de Qwen 108
7.1.1. Comportamiento durante entrenamiento 108
7.1.2. Evaluación del modelo seleccionado 108
7.2. Resultados de interpretación semántica 108
7.2.1. Qwen 108
7.2.2. GPT 108
7.2.3. Comparación 108
7.3. Resultados de planificación 108
7.3.1. Qwen–CLIPS 108
7.3.2. GPT directo 108
7.3.3. Condición de control CLIPS 108
7.3.4. Comparación 108
7.4. Análisis de errores 108
7.4.1. Errores de interpretación 108
7.4.2. Errores de planificación 108
7.4.3. Casos representativos 108
7.5. Resultados de ejecución física 108
7.6. Rendimiento computacional 108
7.6.1. Latencia 108
7.6.2. Recursos computacionales 108
7.6.3. Tokens y costo 108

8. Discusión **109**
8.1. Interpretación de los resultados respecto a la hipótesis 110
8.2. Interpretación semántica 110
8.2.1. Especialización de Qwen 110
8.2.2. Capacidad de GPT 110
8.2.3. Efecto de la variabilidad lingüística 110
8.3. Planificación de acciones 110
8.3.1. Expansión determinista mediante CLIPS 110
8.3.2. Planificación generativa mediante GPT 110

8.3.3. Separación entre errores de interpretación y planificación 110
8.4. Comparación Qwen–CLIPS frente a GPT directo 110
8.4.1. Exactitud 110
8.4.2. Consistencia 110
8.4.3. Latencia 110
8.4.4. Recursos y costo 110
8.4.5. Dependencia de infraestructura externa 110
8.5. Generalización fuera de las plantillas GPSR 110
8.6. Limitaciones y amenazas a la validez 110
8.6.1. Dominio de tareas 110
8.6.2. Dataset 110
8.6.3. Modelo Qwen 110
8.6.4. Modelo GPT 110
8.6.5. Plataforma física 110
8.7. Implicaciones para la planificación en robots de servicio 110

9. Conclusiones y trabajo futuro 111
9.1. Conclusiones principales 111
9.2. Cumplimiento de los objetivos 111
9.3. Evaluación de la hipótesis 111
9.4. Contribuciones de la investigación 111
9.4.1. Representación canónica de órdenes 111
9.4.2. Arquitectura Qwen–CLIPS 111
9.4.3. Comparación con planificación directa mediante GPT 111
9.4.4. Metodología para separar errores de interpretación y planificación 111
9.5. Trabajo futuro 111

A. Procedimientos de operación e integración 112
A.1. Consulta del código y documentación 112
A.2. Comunicación ROS 2 y operación del sistema 112
A.2.1. Recepción del plan y entrega de instrucciones 114
A.3. Inicio del servidor Qwen 115
A.4. Identidad de modelos, datos y recursos 116
A.5. Inventario de software 116

Índice de figuras

1.1. Robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM y utilizado como plataforma de referencia en esta investigación. 2

2.1. Ciclo simplificado de un sistema de producción en CLIPS. Las operaciones sobre los hechos pueden activar o desactivar reglas y actualizar la agenda. 9

3.1. Arquitectura general ViRBot del robot Justina. Tomada de *Semantic reasoning in service robots using expert systems* (1). 17

3.2. Orden ilustrativa, objetivos suministrados, hechos adaptados y plan obtenido. Los campos vacíos de los hechos se omiten para facilitar la lectura; se conservan los identificadores de paso del plan. El estado inicial y las convenciones de retorno y cierre condicionan la expansión. Elaboración propia. 25

3.3. Arquitectura Qwen–CLIPS y productos entre componentes. Las flechas discontinuas indican información utilizada por el procesamiento determinista, sin implicar su envío directo a Qwen. Elaboración propia. . . . 28

3.4. Arquitectura GPT directo: respuesta generada y validación posterior. Los fallos técnicos se registran aparte de los tipos de respuesta. La validación de formato y contrato no demuestra cumplimiento semántico. Elaboración propia. 30

3.5. Distinción entre fidelidad semántica, coherencia interna del plan y cumplimiento simbólico de la tarea. Los dictámenes corresponden al contexto declarado y a las acciones de tarea mostradas de forma esquemática; no representan éxito físico. Elaboración propia. 34

4.1. Construcción del conjunto de datos a partir de catálogos y estructuras de tareas. El balance guía la selección y redistribución de cuotas; la comprobación con CLIPS examina la expansión de las secuencias seleccionadas. Elaboración propia. 45
4.2. Construcción de dos muestras de entrenamiento con campos compartidos y variantes lingüísticas distintas. Las órdenes y sus etiquetas corresponden a dos filas del corpus; se verificaron las plantillas y la función semántica que las construyen. La etiqueta se obtiene del contexto de generación, sin inferencia de Qwen ni expansión a un plan. Elaboración propia. 46
4.3. Función de los datos de ajuste, desarrollo y evaluación final. El corpus de ajuste permite entrenar y seleccionar los adaptadores; las pruebas de desarrollo orientan las correcciones. La prueba final se reserva para comparar configuraciones fijadas sobre órdenes y referencias revisadas. Elaboración propia. 50

5.1. Entrenamiento de Qwen2.5-3B y Qwen2.5-7B mediante QLoRA. (a) Pérdidas durante las 9 000 actualizaciones. (b) Detalle de validación desde el paso 1 000; las estrellas señalan los checkpoints seleccionados: 7 000 para 3B y 5 000 para 7B. Las curvas describen el ajuste y no equivalen al cumplimiento de tareas de planificación. Elaboración propia. 68
5.2. Despliegue e interfaces del recorrido Qwen–CLIPS en Justina. El cliente coordina interpretación y conversión; CLIPS publica el plan para sus consumidores. El gestor se inicia por separado y requiere los remapeos descritos en el texto. Elaboración propia. 78
5.3. Transformaciones de la instrucción del ejemplo: orden, objetivos predichos y campos de los hechos `gpsr-goal`. La predicción de Qwen es ilustrativa. Cada fila de la tabla representa un hecho construido por el adaptador. Elaboración propia. 81
5.4. Expansión de los cuatro objetivos y evolución del estado simbólico. Se muestran las acciones operativas principales; los anuncios y las frases asociadas se conservan en el plan completo. Los cambios de estado representan efectos previstos durante la planificación. Elaboración propia. 83

Capítulo 1

Introducción

Los robots de servicio cubren usos muy variados: desde la asistencia a personas mayores hasta la gestión de tareas en entornos residenciales (2). En estos escenarios, deben interpretar instrucciones formuladas en lenguaje natural y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción (3).

Esta transformación presenta dos problemas relacionados: interpretar las subtareas, entidades y dependencias de la orden, y construir una secuencia de acciones compatibles con las capacidades del robot. Los sistemas basados en reglas permiten representar explícitamente conocimiento operativo dentro de un dominio definido (4), mientras que los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) ofrecen mecanismos para procesar distintas formulaciones lingüísticas. Sin embargo, una respuesta generada por un modelo puede ser coherente en su redacción y contener entidades, acciones o relaciones incompatibles con la tarea (5).

Esta tesis estudia dos arquitecturas para transformar órdenes en planes. La primera, Qwen–CLIPS, utiliza Qwen2.5 como intérprete semántico para producir objetivos canónicos, es decir, una representación estructurada de las subtareas y sus argumentos. Estos objetivos se validan y adaptan a hechos que CLIPS procesa mediante reglas de producción para generar el plan. La segunda arquitectura, GPT directo, genera conjuntamente los objetivos y las acciones a partir de la orden y de una especificación del dominio. Su respuesta se solicita conforme a un JSON Schema, que define la estructura admitida, y se somete posteriormente a validación determinista.

Ambas se implementaron como arquitecturas independientes. La comparación utiliza el mismo conjunto de órdenes, contexto y criterios de evaluación.

1.1. Contexto y motivación

El robot de servicio Justina (Figura 1.1), desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra capacidades de percepción, navegación, manipulación e interacción mediante ROS 2 (*Robot Operating System 2*). La planificación conecta las instrucciones recibidas con las operaciones ofrecidas por esos componentes. Una orden de entrega, por ejemplo, puede requerir desplazarse a una habitación, localizar un objeto, adquirirlo y entregarlo a una persona, conservando la identidad del objeto y del destinatario entre subtareas. En este contexto, planificar acciones de manera autónoma y compatible con las capacidades del robot constituye un requisito fundamental.

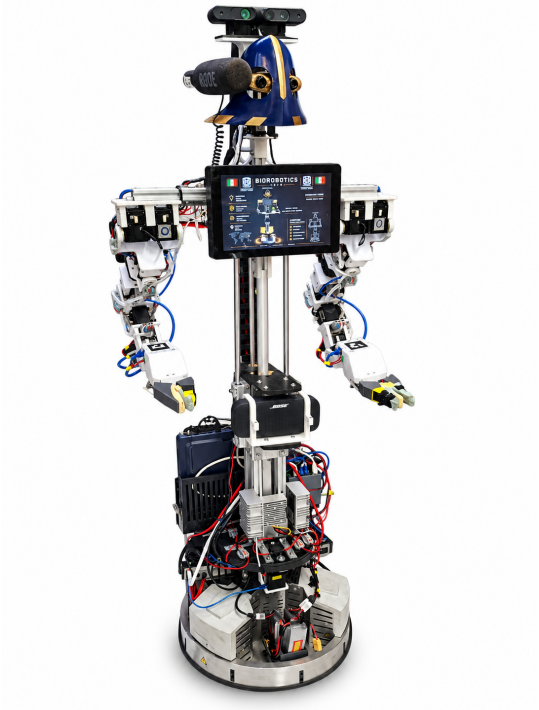


Figura 1.1: Robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM y utilizado como plataforma de referencia en esta investigación.

El dominio de referencia se apoya en General Purpose Service Robot (GPSR), una prueba de RoboCup@Home que incluye órdenes de servicio compuestas (6). En este trabajo, las estructuras del generador de comandos, sus extensiones locales y las capacidades implementadas en Justina delimitan las tareas consideradas. La variación lingüística permite estudiar diferentes maneras de expresar esas tareas, además de las formulaciones utilizadas para construir los datos de entrenamiento.

La motivación de la arquitectura neuro-simbólica es separar la interpretación lingüística de la expansión operativa de los objetivos. Esta separación permite inspeccionar las representaciones intermedias y las reglas utilizadas para producir las acciones. No obstante, su utilidad depende tanto de la fidelidad de la interpretación como de la cobertura y corrección del componente simbólico. El contraste con GPT directo permite examinar qué ventajas y limitaciones presenta cada organización del sistema y qué costos acompañan su despliegue.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se trata de identificar qué errores, costos y condiciones de uso caracterizan a cada arquitectura cuando se evalúan con las mismas órdenes, el mismo dominio y criterios comunes de cumplimiento.

1.2. Planteamiento del problema

Una instrucción puede expresar varias subtareas, omitir operaciones implícitas y utilizar pronombres o descripciones que dependen del contexto. Su interpretación debe conservar las entidades, los atributos solicitados y las relaciones de precedencia. La generación del plan debe traducir esa interpretación a acciones con argumentos y efectos compatibles con el ejecutor.

Una solución cuya interpretación depende de reglas lingüísticas predefinidas requiere ampliar su cobertura ante nuevas formulaciones. Un modelo de lenguaje puede procesar variaciones de expresión, pero también omitir restricciones o proponer operaciones no disponibles. Por su parte, una expansión simbólica puede generar una secuencia internamente coherente a partir de objetivos que no corresponden a la orden original. Por ello, la corrección del formato, la fidelidad de los objetivos y el cumplimiento de la tarea deben examinarse por separado.

El problema de investigación consiste en determinar cómo se comportan las arquitecturas propuestas bajo condiciones comparables: si sus planes satisfacen las subtareas y restricciones solicitadas, cómo responden a reformulaciones y límites de cobertura, y qué errores y costos introduce cada configuración. La pregunta principal se formula de la siguiente manera:

> **¿Qué diferencias presentan las configuraciones Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo en el cumplimiento simbólico de órdenes del dominio implementado, y qué resultados muestran**

en fidelidad semántica, robustez lingüística, consistencia, latencia y costo observable?

El cumplimiento simbólico se entiende como la satisfacción de las subtareas, restricciones y dependencias de la orden mediante acciones compatibles con el contrato del ejecutor, bajo las condiciones del escenario. La robustez lingüística se refiere a conservar la corrección ante formulaciones equivalentes; la consistencia entre ejecuciones describe la estabilidad de las respuestas al repetir una misma entrada. Una respuesta estable no implica necesariamente una respuesta correcta.

1.3. Alcance y delimitaciones

Se comparan dos arquitecturas independientes y tres configuraciones: Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo. Se utilizan las variantes base Qwen2.5-3B y Qwen2.5-7B, especializadas mediante QLoRA, un método de adaptación de bajo rango con cuantización (*Quantized Low-Rank Adaptation*), siguiendo un procedimiento común de ajuste (7). Ambas configuraciones comparten el procesamiento de normalización y adaptación, las reglas y el estado simbólico inicial. Las tres configuraciones reciben las mismas órdenes en inglés y disponen de información equivalente sobre el dominio y las condiciones del escenario.

La unidad comparada es el sistema configurado, que comprende el modelo, su adaptación, las instrucciones de generación, la validación y el mecanismo de planificación. En consecuencia, las diferencias entre Qwen–CLIPS y GPT directo no pueden atribuirse exclusivamente a CLIPS, al tamaño del modelo o al ajuste fino. Las conclusiones se limitan a las configuraciones y condiciones evaluadas.

La evaluación principal considera tareas resolubles dentro de la cobertura implementada. Se analizan por separado composiciones fuera de las estructuras del generador, solicitudes que exceden las capacidades disponibles y entradas ambiguas o contradictorias. Según los criterios de cada caso, una aclaración o un rechazo puede ser una respuesta adecuada. Se distinguen estos comportamientos de los fallos técnicos y del rechazo de salidas inválidas, sin suponer que ambas arquitecturas disponen del mismo mecanismo conversacional.

La comparación es computacional y examina objetivos y planes sin ejecutarlos físicamente en Justina. La compatibilidad con el contrato de acciones y el cumplimiento

simbólico no garantizan el éxito de la percepción, navegación, manipulación o interacción. La integración de Qwen–CLIPS en Justina y su utilización durante la RoboCup 2026 se documentan como una experiencia de aplicación separada.

Para apoyar el diagnóstico, se incorpora un control que proporciona objetivos de referencia aprobados por el mismo adaptador y a las mismas reglas CLIPS. Este control permite examinar el procesamiento posterior a la interpretación y no constituye una cuarta configuración principal. Los recursos de inferencia local y el costo del acceso a GPT mediante una interfaz de programación de aplicaciones (API) se reportan según su disponibilidad; no se dispone de mediciones internas del cómputo del proveedor.

1.4. Hipótesis

La hipótesis principal plantea que, en las tareas resolubles del dominio implementado, cada configuración Qwen–CLIPS alcanza una mayor proporción de planes con cumplimiento simbólico completo que GPT directo. Su fundamento es la separación entre interpretación y expansión operativa, aunque la comparación de sistemas completos no permite atribuir una diferencia exclusivamente a esa separación.

La hipótesis complementaria plantea que GPT directo presenta mayor robustez ante reformulaciones léxicas y sintácticas independientes de las plantillas de entrenamiento. El criterio es conservar el cumplimiento de la tarea ante expresiones equivalentes.

El contraste utiliza las mismas órdenes y agrupa las reformulaciones y repeticiones por escenario. La consistencia, la latencia y el costo se analizan como dimensiones comparativas, sin anticipar una configuración ganadora ni superioridad de 7B sobre 3B. El procedimiento se describe en el Capítulo 6.

1.5. Objetivos

1.5.1. Objetivo general

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con GPT directo en cumplimiento simbólico, fidelidad semántica, robustez lingüística, consistencia, latencia y costo observable.

1.5.2. Objetivos específicos

1. Definir el dominio, la representación canónica de objetivos y el contrato de acciones del ejecutor, y construir los datos supervisados para la interpretación.
2. Especializar Qwen2.5-3B y Qwen2.5-7B mediante QLoRA bajo un procedimiento común.
3. Implementar la validación y adaptación de objetivos, su expansión mediante CLIPS y la integración con ROS 2 en Justina.
4. Implementar GPT directo con la misma especificación operativa, salida estructurada y validación determinista.
5. Establecer una evaluación independiente del desarrollo para comparar cumplimiento, fidelidad, robustez, consistencia, latencia y costo, y diagnosticar errores mediante productos intermedios y el control con objetivos de referencia.
6. Documentar la experiencia de Qwen–CLIPS en Justina durante RoboCup 2026, separada de la comparación computacional.

1.6. Contribuciones

El trabajo se apoyó en ViRBot, en el motor CLIPS y en el generador GPSR. Sobre esta base se desarrollaron:

- La adaptación del generador para producir pares orden–objetivos, con metadatos, controles de distribución y validación.
- Los intérpretes Qwen2.5-3B y Qwen2.5-7B especializados mediante QLoRA, y la interfaz de adaptación que los conecta con la expansión por reglas e integración ROS 2 de Justina.
- El planificador GPT directo, condicionado por el mismo dominio operativo y acompañado de validación determinista.
- El instrumento de campañas y el protocolo que distinguen fidelidad de interpretación, cumplimiento del plan y errores entre componentes, junto con registros de configuración y costo observable.

La aportación experimental consiste en caracterizar estas configuraciones bajo requisitos comunes y analizar dónde se producen sus errores. Los resultados y la experiencia de aplicación en RoboCup se presentan en los capítulos correspondientes.

1.7. Estructura del documento

El capítulo 2 revisa los antecedentes y sitúa la propuesta. El capítulo 3 define las representaciones, las arquitecturas y los criterios de corrección. El capítulo 4 explica la construcción de datos y el diseño metodológico; el capítulo 5 describe la implementación y sus comprobaciones. El capítulo 6 establece el protocolo experimental, y el capítulo 7 presenta los resultados y la experiencia con Justina. Los capítulos 8 y 9 desarrollan la discusión, las conclusiones y el trabajo futuro. El Apéndice A reúne los procedimientos de operación e integración.

Capítulo 2

Antecedentes y estado del arte

Este capítulo revisa los enfoques relacionados con la transformación de instrucciones en lenguaje natural a planes para robots de servicio. Se consideran los sistemas basados en reglas, CLIPS, los modelos de lenguaje y las arquitecturas que combinan interpretación neuronal con razonamiento simbólico. La revisión distingue las herramientas utilizadas de los trabajos que muestran diferentes formas de conectar el lenguaje con las capacidades de un robot.

2.1. Sistemas basados en reglas

Los sistemas basados en reglas (*Rule-Based Systems*, RBS) representan conocimiento mediante hechos y reglas que relacionan condiciones con conclusiones u operaciones. Su desarrollo está ligado a los sistemas expertos, en los que el conocimiento de un dominio se expresa de forma explícita (3, 4).

En robótica, esta representación permite organizar decisiones y secuencias de operaciones a partir de información simbólica sobre el entorno y la tarea. El interés reside en que puede identificarse qué condiciones provocaron una decisión y qué conocimiento se utilizó. Sin embargo, la corrección de las decisiones depende de la base de reglas, de los hechos disponibles y de los supuestos del dominio.

La cobertura de estos sistemas requiere definir los casos y relaciones que pueden procesarse. Si también se utilizan reglas para interpretar lenguaje, las variaciones lingüísticas deben quedar contempladas por ese mecanismo. En cambio, un sistema simbólico que recibe objetivos estructurados puede concentrarse en transformarlos en acciones, dejando la interpretación de la orden a otro componente. Esta distinción permite com-

binar reglas con modelos de lenguaje sin suponer que el componente neuronal amplía automáticamente las capacidades operativas del robot.

2.2. CLIPS como sistema de producción

CLIPS (*C Language Integrated Production System*) es un entorno desarrollado originalmente por la NASA para construir sistemas basados en conocimiento. Proporciona hechos, plantillas, reglas de producción, funciones y un motor de inferencia (4, 8).

CLIPS utiliza encadenamiento hacia adelante, cuyo ciclo se resume en la Figura 2.1. La prioridad de las reglas y la estrategia de resolución de conflictos ordenan las activaciones de la agenda; la ejecución continúa hasta que no quedan activaciones disponibles o se alcanza una condición de parada (8).

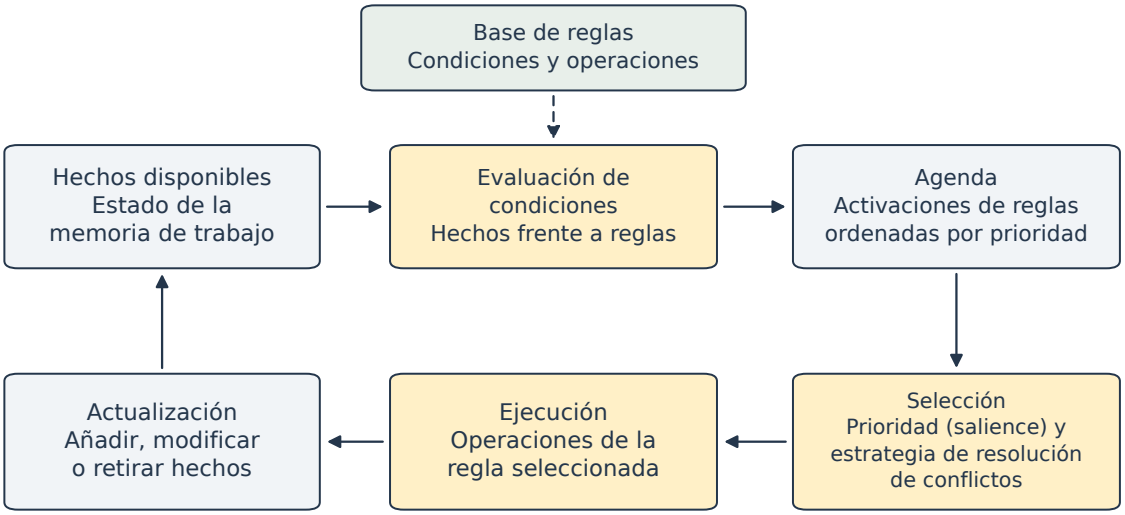


Figura 2.1: Ciclo simplificado de un sistema de producción en CLIPS. Las operaciones sobre los hechos pueden activar o desactivar reglas y actualizar la agenda.

Para la planificación basada en reglas resultan relevantes las siguientes propiedades:

- **Conocimiento explícito:** las condiciones y operaciones pueden expresarse mediante reglas y estructuras simbólicas inspeccionables.
- **Trazabilidad:** es posible registrar las reglas activadas y los cambios en la base de hechos para analizar cómo se obtuvo una salida.

- **Integración:** su implementación en C permite incorporarlo a otros sistemas de software; también existen interfaces para utilizarlo desde otros lenguajes.

La reproducibilidad del comportamiento exige conservar las reglas, los hechos iniciales, la estrategia de resolución de conflictos y las operaciones externas que intervienen. La presencia de reglas explícitas facilita el análisis, pero no constituye por sí misma una verificación formal ni una garantía de seguridad física.

CLIPS proporciona un motor de reglas, no una definición completa del problema de planificación. La representación de objetivos, el conocimiento para expandirlos y las condiciones de terminación deben diseñarse para cada aplicación. Por ello, conviene distinguir el funcionamiento del motor de la corrección de las reglas construidas sobre él.

2.3. Modelos de lenguaje para instrucciones robóticas

La arquitectura Transformer introdujo mecanismos de atención para representar relaciones entre elementos de una secuencia (9). Sobre esta base se desarrollaron modelos de lenguaje capaces de generar texto condicionado por instrucciones y contexto. En robótica, pueden utilizarse para interpretar órdenes, producir representaciones intermedias o proponer operaciones de alto nivel (5).

Estas funciones deben distinguirse. La interpretación recupera las subtareas, entidades y relaciones de la orden. La generación de planes propone las acciones necesarias para atenderlas. Un modelo puede realizar una de estas funciones o ambas, dependiendo de su entrenamiento, de las instrucciones que recibe y de los componentes con los que se integra. La formalización adoptada en esta tesis se presenta en el Capítulo 3.

2.3.1. Familias Qwen y GPT

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. Qwen2.5 incluye variantes de distintos tamaños y modelos base y ajustados para seguimiento de instrucciones (10). La disponibilidad de sus pesos permite adaptar y desplegar las variantes seleccionadas en una infraestructura bajo el control del proyecto. Las denominaciones 3B y 7B corresponden a tamaños nominales de aproximadamente tres y siete mil millones de parámetros.

Los modelos GPT constituyen otra familia de modelos generativos. En esta investigación se accede al modelo seleccionado mediante una API externa. Para reproducir una evaluación es necesario identificar el modelo utilizado y su configuración.

La diferencia de despliegue también influye en la comparación: con un modelo alojado en infraestructura propia pueden observarse recursos locales, mientras que un servicio externo introduce comunicación, disponibilidad y facturación por uso. Estas condiciones deben documentarse junto con la calidad de las respuestas. Los motivos de selección y las configuraciones concretas corresponden a la metodología y la implementación.

2.3.2. Adaptación eficiente: LoRA y QLoRA

El ajuste completo de un modelo requiere actualizar todos sus parámetros y mantener información adicional durante el entrenamiento. Los métodos de adaptación eficiente en parámetros reducen el número de pesos entrenables y mantienen congelado gran parte del modelo original (11).

LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas, sin actualizar los pesos base (12). QLoRA combina estos adaptadores con la cuantización del modelo base para reducir la memoria necesaria durante el ajuste (7). Estas técnicas permiten especializar un modelo en una tarea, aunque su uso no demuestra por sí mismo que la interpretación resultante sea correcta ni que generalice fuera de los datos utilizados.

2.3.3. Salidas estructuradas y validación

Una representación legible por máquina facilita conectar la salida de un modelo con otros programas. JSON (*JavaScript Object Notation*) organiza la información en campos y valores; JSON Schema permite especificar tipos, campos obligatorios y restricciones de estructura (13).

El cumplimiento de ese formato no demuestra que la respuesta resuelva la tarea. Una salida puede ajustarse al esquema y omitir un destino, confundir al destinatario o utilizar un argumento incompatible con una acción. Por ello, las comprobaciones de sintaxis y esquema deben distinguirse de las comprobaciones sobre entidades y acciones, y de la evaluación del significado de la orden. Los criterios conceptuales utilizados en esta tesis se desarrollan en la Sección 3.5.

2.3.4. Limitaciones para la planificación robótica

La generación puede introducir entidades inexistentes, omitir subtareas o alterar dependencias. Además, la respuesta puede variar entre ejecuciones o ante cambios en las instrucciones de generación. Una orden ambigua también puede producir una interpretación plausible que no corresponda a lo que el usuario pretendía (5).

La conexión con capacidades reales constituye una dificultad adicional: describir una operación no implica que el robot pueda realizarla. Las interfaces, los argumentos y las condiciones de uso deben corresponder a los módulos disponibles. De este problema surgen enfoques que restringen la selección de habilidades, generan programas sobre interfaces conocidas o delegan la búsqueda de planes a un componente simbólico.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas combinan aprendizaje neuronal con representaciones y mecanismos de razonamiento explícitos (14, 15). Una forma de integración consiste en transformar una entrada no estructurada en símbolos y utilizar después un componente separado para razonar sobre ellos. Otra utiliza conocimiento simbólico para restringir o comprobar los resultados del modelo.

Esta combinación no requiere que el aprendizaje y el razonamiento ocurran dentro de un único modelo. Su utilidad depende de la interfaz entre componentes: una interpretación incorrecta puede propagarse al razonamiento posterior, mientras que una representación correcta puede exceder la cobertura del sistema simbólico. En consecuencia, la separación facilita el diagnóstico, pero no elimina la necesidad de evaluar el resultado completo.

2.5. Trabajos relacionados con lenguaje y planificación

2.5.1. Razonamiento semántico en ViRBot y Justina

En *Semantic reasoning in service robots using expert systems*, Savage y colaboradores (1) presentan el módulo de razonamiento semántico de la arquitectura ViRBot y su implementación en el robot de servicio Justina. El sistema utiliza representaciones de

dependencia conceptual para expresar el significado de las instrucciones y un sistema experto basado en CLIPS para relacionarlas con el conocimiento y las acciones del robot. Este trabajo constituye un antecedente directo de la presente investigación, al abordar la conexión entre comprensión del lenguaje, representación simbólica y planificación en la misma plataforma robótica.

Sobre este antecedente, la presente tesis estudia el uso de Qwen como intérprete de órdenes hacia objetivos canónicos y su expansión mediante CLIPS. Además, compara esta arquitectura con un planificador GPT directo, para examinar las diferencias de cumplimiento, robustez y costo bajo condiciones comunes.

2.5.2. SayCan: selección de habilidades disponibles

SayCan combina el conocimiento de un modelo de lenguaje con habilidades robóticas preentrenadas. El modelo aporta información sobre la pertinencia de las habilidades para la instrucción, mientras que las funciones de valor asociadas aportan una estimación de su viabilidad en el entorno. El trabajo presenta tareas realizadas con un manipulador móvil (16).

Este antecedente muestra la importancia de conectar las propuestas lingüísticas con las capacidades disponibles. La relación con esta tesis está en esa restricción operativa; el mecanismo es diferente, pues aquí se estudian expansión mediante reglas y generación directa bajo un contrato de acciones, sin incorporar funciones de valor aprendidas para seleccionar habilidades.

2.5.3. Code as Policies: generación de programas

Code as Policies utiliza modelos de lenguaje para generar programas que combinan llamadas a interfaces de percepción y control. Los ejemplos proporcionados relacionan instrucciones con código, y el programa generado puede incluir funciones y bucles que procesan observaciones (17).

Su representación de salida puede expresar comportamiento reactivo y no se limita a una lista fija de acciones. Para esta tesis, su interés es mostrar que la conexión entre lenguaje y robot depende también de las interfaces disponibles. La evaluación de una secuencia simbólica de acciones tiene un alcance distinto de la ejecución de una política expresada como programa.

2.5.4. LLM+P: formalización y planificación clásica

LLM+P transforma una descripción en lenguaje natural a un problema expresado en PDDL (*Planning Domain Definition Language*), utiliza un planificador clásico y convierte la solución a lenguaje natural (18). PDDL permite describir acciones con parámetros, precondiciones y efectos, además de los objetos, el estado inicial y los objetivos de un problema.

Este enfoque separa la traducción lingüística de la búsqueda del plan. Su evaluación utiliza problemas de planificación; sus resultados no deben interpretarse automáticamente como éxito físico de un robot. La relación con Qwen–CLIPS reside en delegar el procesamiento posterior a un componente simbólico. La diferencia es que CLIPS expande objetivos mediante reglas diseñadas para el dominio, mientras que LLM+P emplea un planificador clásico sobre una representación PDDL.

2.5.5. Comparación de los enfoques

La Tabla 2.1 resume las diferencias relevantes para esta investigación. Compara la organización de los sistemas y sus representaciones; los trabajos utilizan tareas y condiciones distintas, por lo que no se establece una clasificación común de desempeño.

Tabla 2.1: Enfoques relacionados para conectar lenguaje y capacidades operativas.

Enfoque	**Función del modelo**	**Conexión operativa**	**Aspecto relevante**
SayCan (16)	Valorar la pertinencia de habilidades.	Habilidades preentrenadas y funciones de valor.	Diferenciar pertinencia lingüística y viabilidad.
Code as Policies (17)	Generar código de políticas.	Interfaces de percepción y control.	Expresar comportamiento mediante programas.
LLM+P (18)	Traducir el problema a PDDL y verbalizar la solución.	Planificador clásico.	Separar formalización y búsqueda del plan.

Los antecedentes muestran distintas formas de distribuir la interpretación, la selección de operaciones y la planificación. De ellos se desprende la necesidad de comprobar tanto la correspondencia con la orden como la compatibilidad con las capacidades del sistema. Una representación formal facilita el procesamiento, pero su utilidad depende de conservar la tarea original.

2.6. Síntesis y conexión con la propuesta

2.6.1. Brecha de investigación

ViRBot aporta el antecedente de razonamiento con CLIPS en Justina; los otros enfoques revisados ilustran alternativas para conectar lenguaje y capacidades operativas. La pregunta de esta tesis es cómo se comportan la interpretación especializada con expansión por reglas y la generación conjunta de objetivos y acciones bajo un mismo contrato.

La revisión identifica una oportunidad de evaluación aplicada: los trabajos examinados emplean tareas y condiciones diferentes y no proporcionan una comparación directamente trasladable al dominio de Justina. Esta observación se limita a los antecedentes analizados; no establece la ausencia de comparaciones similares en toda la literatura.

Esta tesis estudia esa pregunta mediante la comparación de Qwen–CLIPS y GPT directo en el dominio implementado para Justina. La comparación caracteriza los sistemas completos y no permite atribuir las diferencias exclusivamente al uso de reglas, al tamaño del modelo o al ajuste fino. El aporte es evidencia empírica sobre el comportamiento de cada organización bajo condiciones comunes, junto con un análisis de las discrepancias entre objetivos y planes y, en Qwen–CLIPS, de los errores de interpretación, adaptación y expansión simbólica.

El capítulo siguiente define el dominio, las representaciones de órdenes, objetivos y planes, y los criterios conceptuales de corrección. La metodología desarrolla después la construcción de datos y las decisiones de diseño de las arquitecturas.

Capítulo 3

Formalización del problema y arquitecturas conceptuales

Este capítulo define las representaciones de órdenes, objetivos y planes utilizadas en Qwen–CLIPS y GPT directo. El dominio toma como referencia GPSR y las capacidades implementadas en Justina. A partir de estas definiciones se establecen los criterios de interpretación y cumplimiento que sustentan la comparación.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot (1) (véase la Figura 3.1). Esta arquitectura integra las diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano–robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

El sistema desarrollado transforma instrucciones en lenguaje natural en objetivos semánticos y, a partir de ellos, en planes para los módulos de percepción, navegación, manipulación e interacción de Justina. La comparación experimental se limita a la interpretación y la planificación, sin ejecutar físicamente los planes. El uso de Qwen–CLIPS integrado en Justina durante la RoboCup 2026 se analiza por separado.

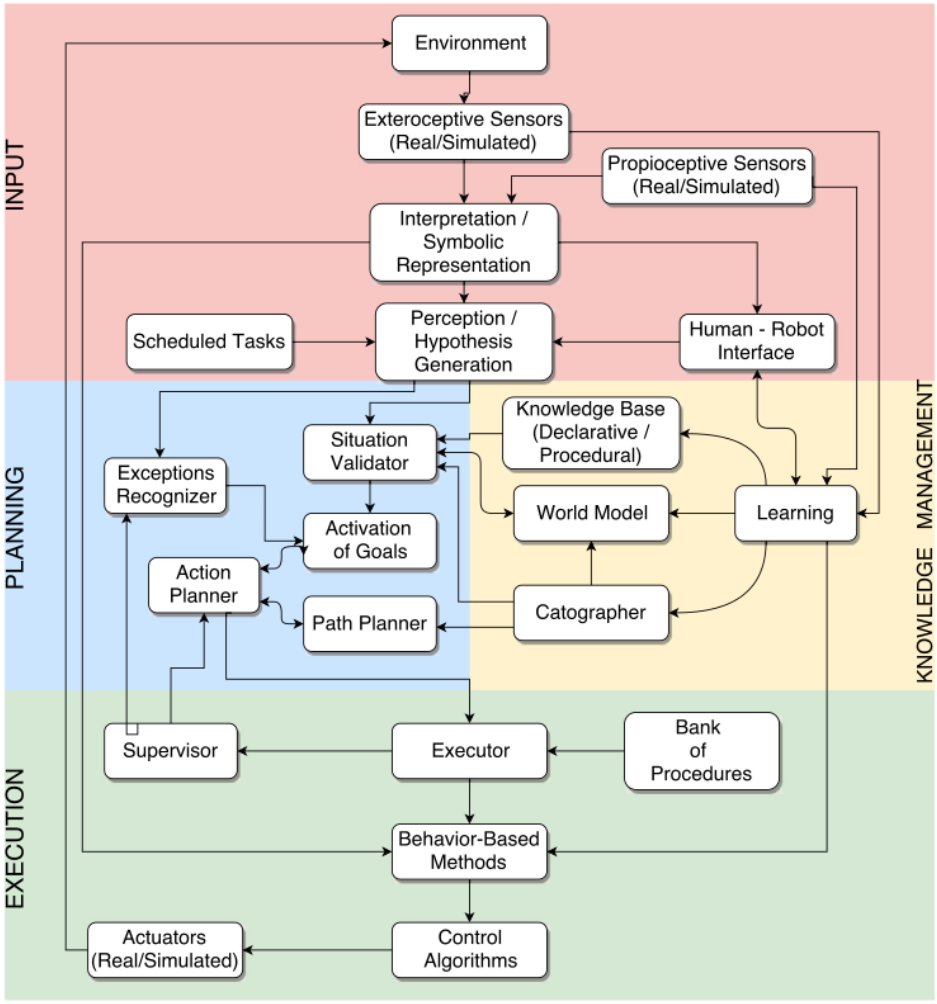


Figura 3.1: Arquitectura general ViRBot del robot Justina. Tomada de *Semantic reasoning in service robots using expert systems* (1).

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para comprender instrucciones expresadas en lenguaje natural y ejecutar tareas que pueden requerir la combinación de distintas habilidades. Entre ellas se encuentran la navegación, la búsqueda y reconocimiento de personas u objetos, la manipulación y la interacción humano–robot(6).

Una característica importante de GPSR es que una instrucción puede representar una tarea compuesta. Por ejemplo, una orden puede requerir que el robot se desplace

hacia una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. Su representación debe conservar esas subtareas y sus relaciones.

El reglamento 2026.2 contempla tres comandos y permite recibirlos juntos o uno por uno. En esta segunda modalidad, el robot vuelve al operador después de cada tarea; al finalizar los comandos debe regresar al *Instruction Point*, el punto de recepción de instrucciones (6). La evaluación de esta tesis procesa cada orden de manera independiente: no estudia la planificación intercalada de los tres comandos.

El reglamento también permite reformulaciones progresivamente más sencillas. Esto motiva distinguir la intención de una tarea de la forma lingüística utilizada para comunicarla, distinción que se desarrolla en la Sección 3.2.3. La comparación computacional utiliza texto como entrada y no evalúa reconocimiento de voz, intervención de operadores ni puntuación física de la competencia.

3.2.2. Generación de comandos y conocimiento del dominio

El *CommandGenerator* oficial de RoboCup@Home combina familias de tareas, estructuras lingüísticas y entidades para producir órdenes simples y compuestas (19). Incluye navegación, búsqueda, conteo, interacción, seguimiento, guía y transporte de objetos. Sus continuaciones permiten reutilizar entidades introducidas en subtareas anteriores.

El generador separa las estructuras de los catálogos de personas, objetos, categorías, ubicaciones y habitaciones. Estos se organizan en *CompetitionTemplate* y pueden sustituirse sin redefinir las familias (20). La adaptación realizada para producir órdenes y etiquetas semánticas se describe en el Capítulo 4.

3.2.3. Dominio de referencia y alcance lingüístico

GPSR proporciona un dominio de tareas y entidades, pero una misma intención puede expresarse mediante formulaciones distintas de las producidas literalmente por el generador oficial.

De manera adicional, el reglamento de GPSR 2026 incorpora un mecanismo de variación sobre las órdenes generadas. Una vez producido un comando mediante el generador oficial, el reglamento establece que éste se proporciona a un modelo de lenguaje para obtener una formulación similar. Además, una instrucción puede ser reformulada hasta tres veces, utilizando expresiones progresivamente más simples cuando sea necesario

(6).

Por ejemplo, el reglamento presenta una transformación equivalente para pasar de una instrucción como:

"Get me a coke from the kitchen."

a una formulación más explícita, como:

"Go to the kitchen, find a coke, and bring it to me."

Bajo las mismas condiciones y la convención canónica adoptada, ambas formulaciones deben tener referencias semánticas equivalentes, como se formaliza en la Sección 3.3.3. La variación lingüística permanece acotada por las intenciones representables y las capacidades operativas disponibles: una instrucción comprensible para una persona puede quedar fuera del alcance del sistema.

El dominio experimental de esta investigación considera instrucciones en inglés, en correspondencia con las estructuras utilizadas por el generador y con el escenario de evaluación. Por ello, los resultados relacionados con variabilidad lingüística deben interpretarse como capacidad para procesar diferentes formulaciones de las tareas soportadas dentro de dicho idioma, y no como evidencia de independencia respecto al idioma o de comprensión irrestricta del lenguaje natural.

3.3. Formalización de órdenes, objetivos y planes

Se utiliza x para la instrucción, G para la secuencia de objetivos semánticos y P para el plan de acciones:

$$x \longrightarrow G \longrightarrow P. \tag{3.1}$$

La interpretación recupera las subtareas y sus argumentos; la planificación determina su realización operativa. Ambas dependen del dominio, el contexto y el contrato del ejecutor. En GPT directo, G y P se generan conjuntamente: el esquema describe productos observables y no etapas internas del modelo.

3.3.1. Información del dominio y condiciones de planificación

Para construir un plan no basta con conocer la orden. También se necesita saber qué entidades existen en el dominio, cuáles son las condiciones de la tarea y qué acciones puede realizar el robot. En esta tesis se distinguen estos elementos:

- **Conocimiento del dominio (K):** los catálogos de personas, objetos y lugares definidos en la Sección 3.3.4. Indican qué entidades se conocen, pero no dónde se encuentran en cada escenario.
- **Contexto (C):** las condiciones y los supuestos de la tarea, como dónde comienza el robot, quién da la instrucción o qué objeto sostiene inicialmente.
- **Contrato del ejecutor ($\mathcal{D}$):** la especificación de las acciones disponibles, sus argumentos, las condiciones necesarias para realizarlas y sus efectos esperados. Por ejemplo, para entregar un objeto se debe indicar cuál es y a quién se entrega, y el robot debe disponer de él. El contrato también establece las convenciones de interacción y cierre del plan.

Estas diferencias importan porque una misma orden puede requerir planes distintos. Si se solicita entregar una manzana y el robot ya la sostiene, puede ser innecesario buscarla y tomarla de nuevo; si no la sostiene, el plan debe incluir su adquisición.

El **estado simbólico** es la descripción de condiciones como la ubicación del robot o el objeto que sostiene. Se utiliza S_0 para indicar el estado inicial considerado al generar el plan. A partir de él se razona sobre los cambios esperados: después de una acción para tomar la manzana, por ejemplo, las acciones posteriores pueden considerar que el robot dispone de ella.

Ese estado describe lo que se espera durante la planificación; no confirma lo ocurrido físicamente. Además, la implementación puede representar solo parte de los efectos del contrato. Por ello, generar una acción no basta para demostrar que se cumplen todos los requisitos de la tarea.

Para comparar las arquitecturas, ambas deben disponer de información equivalente sobre el dominio, el contexto y el contrato. Esa información puede incorporarse mediante entrenamiento, reglas o instrucciones al modelo; la forma de proporcionarla se documenta para cada configuración.

3.3.2. Orden en lenguaje natural

Una instrucción puede representar una tarea simple o una composición de varias subtareas relacionadas. Por ejemplo:

"Go to the kitchen, find an apple and bring it to me."

incluye navegación, búsqueda, manipulación y entrega de un objeto. Por tanto, una instrucción lingüística no corresponde necesariamente con una única acción del robot.

Incluso una expresión breve puede implicar varias operaciones. Considérese:

"Bring me an apple from the kitchen."

Dentro del dominio utilizado en esta investigación, esta orden puede representarse mediante la siguiente secuencia semántica:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

La interpretación no consiste únicamente en identificar palabras clave. También debe reconocer las subtareas explícitas o implícitas, las entidades involucradas y las relaciones necesarias entre ellas.

La estructura de los objetivos obtenidos se describe en la Sección 3.3.5.

3.3.3. Equivalencia semántica entre formulaciones

Dos órdenes son equivalentes cuando solicitan las mismas subtareas, entidades, restricciones y dependencias bajo el mismo contexto. Por ejemplo, *"Bring me an apple from the kitchen"* y *"Go to the kitchen, find an apple, take it and bring it to me"* pueden compartir objetivos de referencia según la convención adoptada.

La equivalencia se establece antes de observar las predicciones. Repetir una interpretación equivocada no acredita robustez, y varios planes pueden cumplir una misma tarea sin coincidir literalmente.

3.3.4. Entidades y conocimiento del dominio

Los catálogos del dominio, denotados conjuntamente por K, contienen personas, objetos, categorías, ubicaciones y habitaciones. Las referencias pueden ser nominales, como `Robin`; genéricas, como `person`; o descritas mediante atributos, como una persona sentada o el objeto de mayor tamaño. La interpretación debe conservar su identidad, tipo y restricciones. La pertenencia de un nombre al catálogo no determina su ubicación en un escenario.

3.3.5. Representación canónica de objetivos

El resultado de la interpretación se expresa mediante una secuencia ordenada de objetivos:

$$G = (g_1, g_2, \ldots, g_n).$$

Cada objetivo g_i es una instancia formada por un tipo de intención y sus argumentos. El vocabulario $\mathcal{G}$ enumera los tipos admitidos; la secuencia G contiene instancias con entidades y relaciones concretas. Algunos ejemplos de objetivos individuales son:

```
go(kitchen)
find(Robin, kind=person)
deliver(apple, to=Robin)
```

La Tabla 3.1 resume los tipos de objetivos y sus argumentos principales.

Los objetivos `find` y `count` requieren distinguir explícitamente si la entidad considerada corresponde a una persona o a un objeto. Para ello se utiliza el argumento `kind`. Por ejemplo:

```
find(Robin, kind=person)
find(apple, kind=object)
count(person, kind=person)
count(drinks, kind=object)
```

Otros objetivos tienen un tipo de entidad implícito. Por ejemplo, `take`, `deliver` y `place` operan sobre objetos, mientras que `guide`, `follow` y `greet` operan sobre personas.

Tabla 3.1: Vocabulario de objetivos semánticos utilizado en la representación canónica.

Objetivo	**Propósito**	**Argumentos principales**
`go`	Desplazarse hacia una ubicación	ubicación
`find`	Localizar una persona u objeto	objetivo, `kind`, gesto, pose, vestimenta o propiedad
`take`	Tomar un objeto	objeto
`deliver`	Entregar un objeto	objeto, destinatario
`place`	Colocar un objeto en un destino	objeto, destino y relación espacial
`guide`	Guiar a una persona	persona, destino
`follow`	Seguir a una persona	persona, destino opcional
`count`	Contar personas u objetos	objetivo, `kind` y atributos opcionales
`tell`	Comunicar información	información o propiedad
`save`	Conservar información obtenida durante una interacción	nombre, pose o gesto
`answer_question`	Atender una pregunta	—
`greet`	Saludar a una persona	persona

Los objetivos pueden incluir además calificadores relevantes presentes en la instrucción. Entre ellos se encuentran gestos, poses, vestimenta, características de objetos y relaciones espaciales. Por ejemplo:

```
find(person, kind=person, gesture='waving person')
find(person, kind=person, pose='sitting person')
find(person, kind=person, wearing='red shirt')
find(drink, kind=object, property=largest)
guide(Robin, to=kitchen)
place(apple, on=table)
```

La representación canónica busca conservar la información necesaria para describir la intención de la orden, eliminando al mismo tiempo las diferencias lingüísticas que no modifican su significado dentro del dominio.

La forma concreta utilizada para serializar y transmitir esta secuencia entre componentes pertenece a la implementación y se describe posteriormente.

Fidelidad semántica. Una secuencia de objetivos admitidos puede omitir subtareas o introducir intenciones ajenas. La fidelidad se juzga frente a la orden, según los criterios de la Sección 3.5.2.

3.3.6. Objetivos semánticos y acciones ejecutables

Los objetivos expresan subtareas canónicas; las acciones especifican operaciones del ejecutor. Por ejemplo, `deliver(apple, to=Robin)` puede requerir localizar al destinatario, aproximarse y ejecutar `give`. La colocación puede necesitar `find_space` y `drop`.

El repertorio de acciones se denota por $\mathcal{A}$ y el plan por $P = (a_1, \ldots, a_m)$, donde cada acción incluye los argumentos del contrato. Un objetivo puede expandirse en varias acciones. A su vez, Qwen ya descompone operaciones implícitas en subtareas: su función excede la extracción de verbos, aunque no produce directamente el plan.

3.3.7. Ejemplo de transformación entre representaciones

La Figura 3.2 ilustra las representaciones con una orden de navegación y búsqueda. Los objetivos se suministraron directamente al adaptador y a las reglas disponibles, sin consultar Qwen. La comprobación produjo un plan y no dejó objetivos pendientes; esto acredita la expansión mostrada, no una evaluación de interpretación ni una ejecución física.

La orden contiene dos subtareas, pero el plan incluye además anuncios, retorno al punto de instrucciones y cierre. Estas acciones provienen de las convenciones de expansión del sistema. El identificador `apple_1` pertenece a la representación de la búsqueda utilizada por el ejecutor; no acredita que se haya detectado físicamente una manzana.

3.3.8. Dependencias y contexto entre objetivos

G es una secuencia ordenada según la convención canónica. Ese orden debe conservar las dependencias de la tarea, aunque las subtareas independientes pueden admitir otras secuencias válidas.

El orden semántico puede diferir del orden de los verbos: *"Bring me an apple"* implica adquisición antes de entrega. Asimismo, repetir `person` en varios objetivos abrevia una referencia compartida, sin garantizar que el ejecutor conserve su identidad. Los nombres, identificadores y mecanismos de interacción implementados se describen

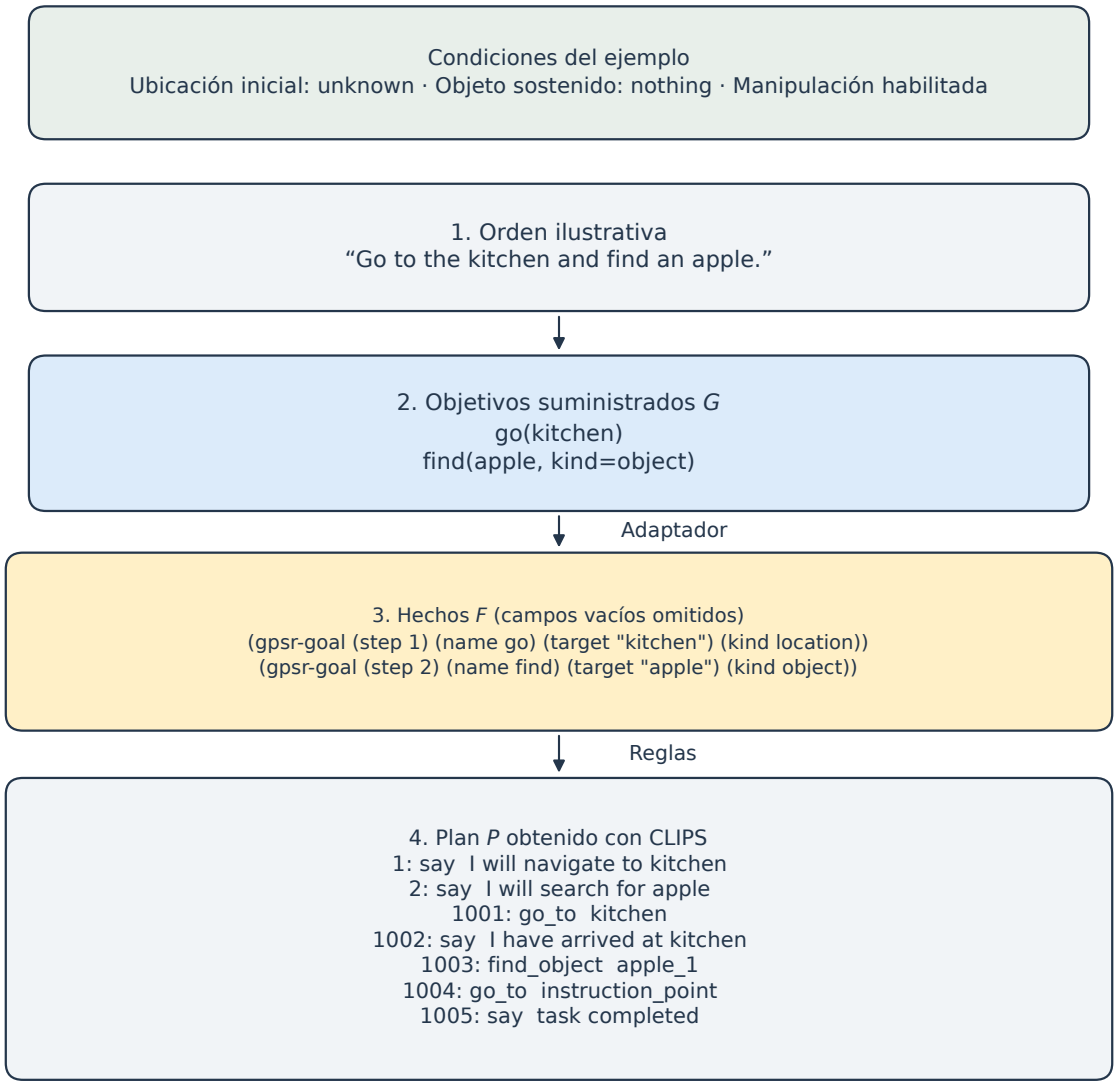


Figura 3.2: Orden ilustrativa, objetivos suministrados, hechos adaptados y plan obtenido. Los campos vacíos de los hechos se omiten para facilitar la lectura; se conservan los identificadores de paso del plan. El estado inicial y las convenciones de retorno y cierre condicionan la expansión. Elaboración propia.

Tabla 3.2: Dependencias que deben conservar objetivos y planes.

Dependencia	**Ejemplo**	**Requisito**
Precedencia	Tomar y entregar una manzana.	Adquirirla antes de entregarla si el robot no la sostiene inicialmente.
Referencia	Buscar a una persona que saluda y guiarla.	Guiar a la misma persona identificada.
Espacio y estado	Ir a la cocina y buscar un objeto.	Realizar la búsqueda en el contexto espacial establecido.

en el capítulo 5; sus límites deben comprobarse cuando intervienen varios referentes. La referencia `me` identifica al emisor bajo el contexto declarado y no a cualquier persona encontrada.

3.4. Arquitecturas conceptuales evaluadas

Las dos arquitecturas resuelven el problema formalizado en la Sección 3.3, distribuyendo de manera distinta la interpretación y la planificación: Qwen–CLIPS las asigna a componentes separados y GPT directo genera ambos productos en una misma respuesta. Dentro de Qwen–CLIPS se comparan las variantes de 3B y 7B, manteniendo común el procesamiento simbólico posterior.

3.4.1. Base común de las arquitecturas evaluadas

Las arquitecturas se comparan sobre las mismas órdenes y requisitos de tarea, con un vocabulario de objetivos $\mathcal{G}$, catálogos K, contexto C y contrato $\mathcal{D}$ comunes. Las referencias utilizadas para evaluar son externas a las predicciones; no se proporcionan al modelo para resolver la orden.

El conocimiento se distribuye de manera diferente. Qwen produce la descomposición semántica y las reglas CLIPS contienen el conocimiento para expandirla. GPT directo recibe una especificación que permite producir objetivos y acciones conjuntamente. Se comparan sistemas completos, por lo que las diferencias no aíslan el efecto causal de las reglas, el ajuste fino o el tamaño del modelo.

La Tabla 3.3 recupera la distinción entre configuraciones principales y control auxiliar. Los detalles del ajuste y de los programas independientes corresponden a metodo-

logía e implementación.

Tabla 3.3: Configuraciones evaluadas y control auxiliar.

Configuración	**Origen de objetivos**	**Generación del plan**
Qwen2.5-3B–CLIPS	Qwen2.5-3B ajustado con QLoRA	Adaptación y reglas CLIPS
Qwen2.5-7B–CLIPS	Qwen2.5-7B ajustado con QLoRA	Adaptación y reglas CLIPS
GPT directo	GPT, junto con el plan	GPT condicionado por la especificación
Control auxiliar	Objetivos de referencia aprobados	Mismo adaptador y reglas CLIPS

3.4.2. Arquitectura Qwen–CLIPS

Esta arquitectura se instancia con las dos variantes indicadas en la Tabla 3.3. Ambas comparten el procesamiento posterior a la interpretación y siguen un procedimiento común de ajuste, descrito en la metodología.

Qwen recibe la orden normalizada y produce objetivos canónicos. El adaptador valida la salida, normaliza identificadores, aplica heurísticas y reparaciones, y construye los hechos entregados a CLIPS. Se conservan la predicción original, la representación adaptada y los hechos para examinar cambios de significado. El capítulo 5 describe estas operaciones.

La Figura 3.3 resume el flujo conceptual de esta arquitectura.

CLIPS expande los objetivos mediante reglas de producción y estado simbólico. Las reglas procesan la secuencia recibida; no reinterpretan la orden ni realizan una búsqueda libre de planes.

Estado esperado durante la planificación

Durante la expansión, CLIPS utiliza ubicación, posesión y contexto de interacción para considerar algunos resultados esperados de las subtareas anteriores. Este seguimiento es parcial y no constituye un modelo exhaustivo del ejecutor.

Las comprobaciones de precondiciones se distribuyen entre la validación de objetivos y las reglas de expansión, con alcances diferentes. La evolución simbólica no incorpora

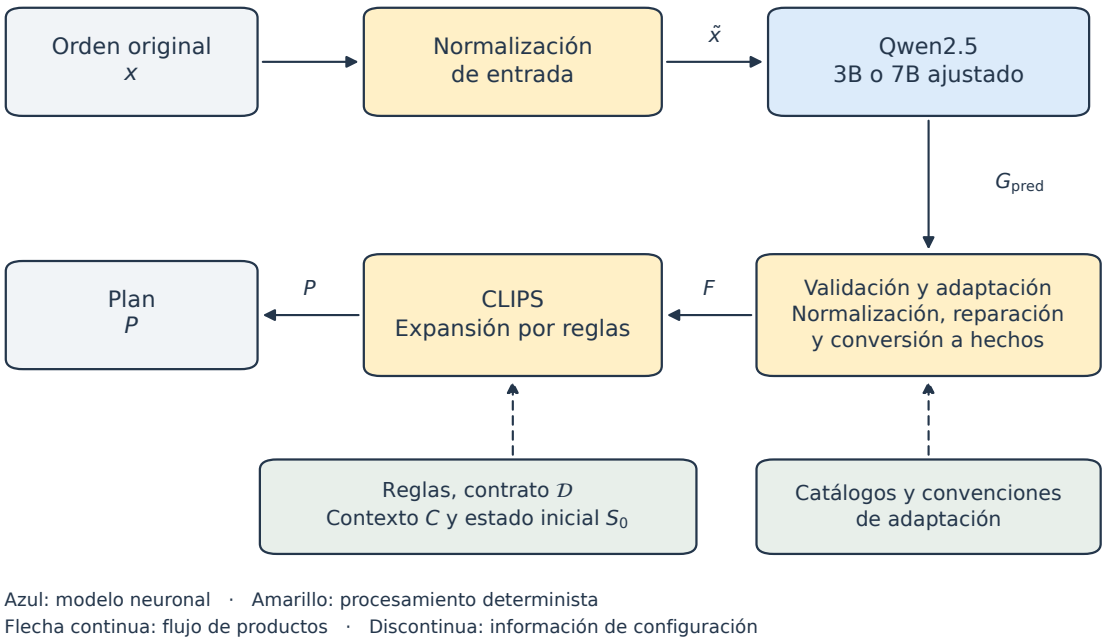


Figura 3.3: Arquitectura Qwen–CLIPS y productos entre componentes. Las flechas discontinuas indican información utilizada por el procesamiento determinista, sin implicar su envío directo a Qwen. Elaboración propia.

necesariamente todos los efectos del contrato. Por ello, el cumplimiento debe contrastarse con los requisitos de la tarea; la terminación de CLIPS no demuestra por sí misma la conservación de referentes, la satisfacción de precondiciones ni el cumplimiento completo del plan.

Separación de errores

Se distinguen errores en la interpretación, en la adaptación a hechos y en la expansión simbólica. Un plan puede ser coherente con objetivos equivocados, por lo que deben examinarse tanto los productos intermedios como su relación con la orden. La Sección 3.5.6 define el control auxiliar que apoya este diagnóstico.

3.4.3. Arquitectura de planificación directa con GPT

GPT directo produce los objetivos G y el plan P en una misma respuesta, condicionada por la orden y la especificación operativa.

Ambos productos se solicitan en una respuesta estructurada conforme a un JSON Schema y se someten a validación determinista. El esquema establece la forma admitida; las comprobaciones de contenido contrastan los valores con los catálogos y el contrato en la medida implementada. Ninguna de estas operaciones sustituye la evaluación del cumplimiento de la orden. El contrato de respuesta y las comprobaciones concretas se documentan en metodología e implementación.

Los objetivos declarados describen la interpretación de la orden y el plan propone su realización. La Sección 3.4.3 explica cómo se examina la correspondencia entre ambos productos.

La Figura 3.4 presenta el flujo conceptual de esta arquitectura.

La planificación directa genera acciones sin delegar su expansión a CLIPS. GPT recibe la especificación del dominio y del contrato descrita en la Sección 3.4.1, y debe conservar la coherencia con el estado esperado durante la generación. Sus objetivos declarados no se envían a CLIPS ni se utilizan para corregir su plan.

Tipos de respuesta

La arquitectura no obliga al modelo a producir un plan para cualquier instrucción recibida. Dependiendo de la información disponible y de la compatibilidad de la orden con el dominio, se contemplan tres tipos de respuesta:

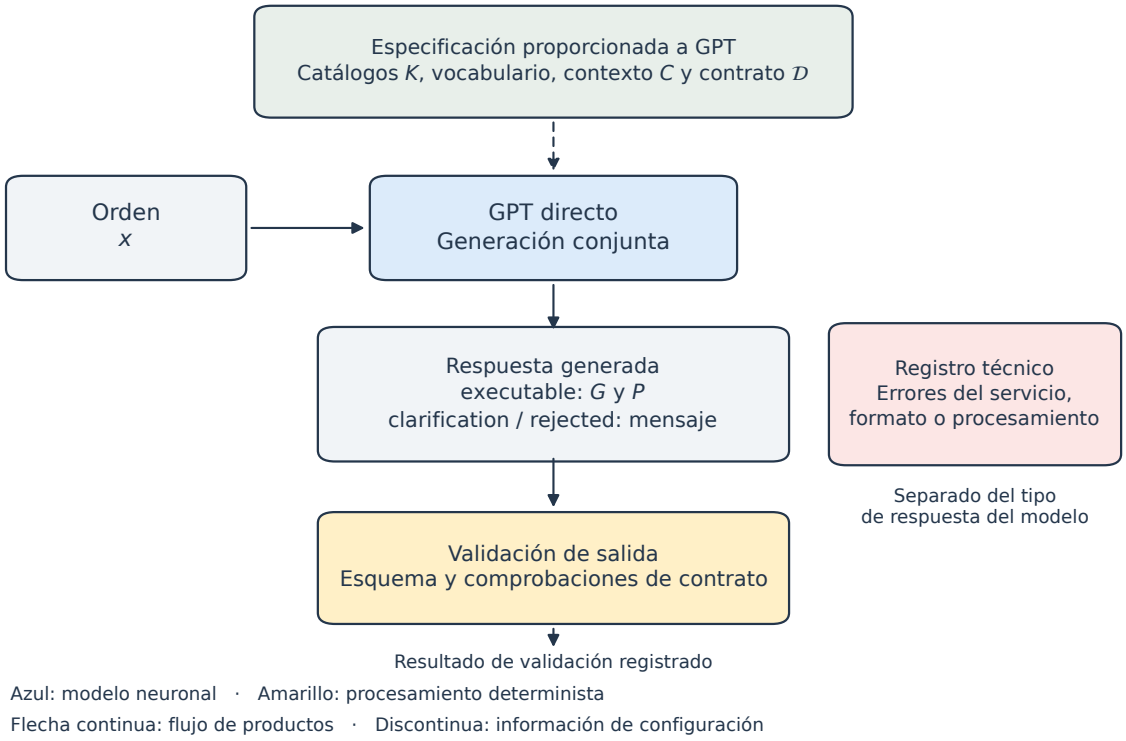


Figura 3.4: Arquitectura GPT directo: respuesta generada y validación posterior. Los fallos técnicos se registran aparte de los tipos de respuesta. La validación de formato y contrato no demuestra cumplimiento semántico. Elaboración propia.

- `executable:` la instrucción contiene información suficiente y puede representarse mediante objetivos y acciones soportadas. En este caso se generan G y P.
- `clarification:` existe información faltante o una ambigüedad que puede resolverse mediante una pregunta breve al usuario. En este caso no se genera un plan hasta resolver dicha ambigüedad.
- `rejected:` la instrucción se encuentra fuera del dominio, requiere una capacidad no disponible o contradice las restricciones establecidas. En este caso tampoco se genera un plan.

Esta clasificación permite solicitar respuestas distintas de un plan. Que el modelo seleccione la categoría adecuada debe comprobarse frente a los criterios del caso.

Separación posterior de interpretación y planificación

Los objetivos y el plan declarados por GPT se evalúan por separado. Puede observarse una discrepancia semántica en G, una omisión en P o una falta de correspondencia entre ambos. Sin embargo, como se generan conjuntamente, estos resultados no identifican con certeza una etapa causal interna del modelo. Un objetivo omitido en G podría estar realizado en P, y un objetivo correcto no asegura una expansión adecuada.

3.4.4. Comparación conceptual de las arquitecturas

La Tabla 3.4 resume las diferencias de organización entre Qwen–CLIPS y GPT directo y sus consecuencias para la representación, el análisis de errores y la variabilidad.

Los criterios de la Sección 3.5 permiten examinar los productos de ambas arquitecturas sin equiparar conformidad de formato, corrección semántica y ejecución física.

La comparación también considera la latencia y el costo observable, cuyo análisis complementa la evaluación de corrección. Los procedimientos de medición se presentan en el Capítulo 6.

3.5. Criterios conceptuales de validez

La evaluación distingue estructura, interpretación y cumplimiento del plan. El resultado físico constituye un nivel separado y queda fuera de la comparación computacional.

Tabla 3.4: Comparación conceptual entre las arquitecturas evaluadas.

Característica	**Qwen–CLIPS (3B y 7B)**	**GPT directo**
Interpretación semántica	Qwen	GPT
Representación explícita de G	Sí	Sí
Generación del plan P	CLIPS mediante reglas de producción	GPT mediante generación condicionada
Conocimiento para expandir los objetivos	Representado explícitamente mediante reglas	Proporcionado mediante contexto y restricciones
Separación entre interpretación y planificación	Explícita entre componentes	Integrada en una misma inferencia
Estado durante la planificación	Estado simbólico esperado explícito	Coherencia con el estado esperado solicitada mediante el contexto
Dependencia de un servicio externo de terceros	No	Sí
Identificación de errores	Inspección de etapas y control auxiliar	Análisis de productos G y P
Variabilidad del resultado	Qwen puede variar entre inferencias; la expansión es determinista con los mismos objetivos adaptados, reglas y estado inicial	Puede presentar variación entre inferencias

3.5.1. Niveles de validez

Para analizar las salidas producidas por las arquitecturas evaluadas se consideran los siguientes niveles:

1. **Validez sintáctica:** la salida puede ser procesada por el sistema y respeta la sintaxis esperada.
2. **Validez estructural:** los objetivos, acciones, argumentos y tipos utilizados pertenecen a las estructuras admitidas por el sistema.
3. **Validez semántica:** la representación G conserva la intención expresada en la instrucción original.
4. **Coherencia interna del plan:** P realiza los objetivos declarados en G y respeta el contrato y el estado esperado.
5. **Cumplimiento simbólico de la tarea:** P satisface las subtareas, restricciones y dependencias de la orden de referencia bajo el contexto declarado, aunque los objetivos predichos sean incorrectos.
6. **Éxito de ejecución:** el robot consigue realizar físicamente las acciones planificadas y alcanzar el resultado esperado.

La Figura 3.5 contrasta tres casos para una misma orden: un plan coherente con objetivos incompletos, objetivos correctos con un plan incompleto y ambos productos correctos. La evaluación mantiene el mismo contexto y supone satisfechas las convenciones operativas omitidas en el esquema. Son casos conceptuales, no resultados de las configuraciones evaluadas.

3.5.2. Validez de la interpretación semántica

La fidelidad semántica se evalúa mediante los siguientes criterios:

1. **Cobertura de las subtareas:** deben conservarse todos los objetivos necesarios para representar la tarea, incluyendo aquellas operaciones implícitas que resulten necesarias dentro del dominio.
2. **Ausencia de intenciones no solicitadas:** la representación no debe introducir objetivos adicionales que modifiquen la intención original de la orden.
3. **Exactitud de entidades y tipos:** las personas, objetos, categorías y ubicaciones utilizadas deben corresponder con las referencias presentes en la instrucción y ser compatibles con el conocimiento K.
4. **Conservación de relaciones y dependencias:** deben mantenerse las relaciones



Figura 3.5: Distinción entre fidelidad semántica, coherencia interna del plan y cumplimiento simbólico de la tarea. Los dictámenes corresponden al contexto declarado y a las acciones de tarea mostradas de forma esquemática; no representan éxito físico. Elaboración propia.

de precedencia, referencia y contexto necesarias entre los objetivos.

Cuando se dispone de una representación de referencia G_{ref}, la interpretación producida por el sistema puede compararse con ella. La coincidencia exacta con los objetivos de referencia es un indicador estricto. El análisis también distingue omisiones, adiciones y errores de entidades o relaciones, y admite alternativas semánticas conforme a criterios fijados para cada caso.

3.5.3. Coherencia del plan y cumplimiento de la tarea

Una vez obtenida la representación semántica G, debe determinarse si el plan P constituye una realización coherente de dichos objetivos.

La coherencia interna del plan se examina mediante los siguientes criterios:

1. **Acciones soportadas:** todas las acciones utilizadas pertenecen al repertorio $\mathcal{A}$ disponible para el sistema de ejecución.
2. **Parámetros válidos:** cada acción utiliza argumentos compatibles con su función y con las entidades del dominio.
3. **Respeto de dependencias:** la secuencia conserva el orden y las relaciones necesarias para realizar la tarea.
4. **Cobertura de los objetivos:** las acciones generadas permiten atender los objetivos contenidos en G sin omitir partes necesarias de la tarea.
5. **Coherencia con el estado esperado:** las acciones posteriores son compatibles con los resultados esperados de las acciones previamente incluidas en el plan.

El cumplimiento se contrasta directamente con los requisitos de la orden, como ilustra la Figura 3.5. No se exige coincidencia con la expansión de CLIPS: se admiten secuencias alternativas que satisfagan la tarea bajo el mismo contrato y contexto.

3.5.4. Robustez, consistencia y trazabilidad

La **robustez lingüística** es la conservación de la corrección ante cambios de formulación que mantienen la tarea, según la equivalencia semántica definida en la Sección 3.3.3. Se examina tanto la interpretación como el cumplimiento del plan ante esas reformulaciones. La igualdad de objetivos ante paráfrasis es un indicador parcial: repetir una respuesta equivocada no demuestra robustez.

La **consistencia entre ejecuciones** describe la estabilidad al repetir una misma entrada y contexto. Puede analizarse en objetivos, acciones, tipo de respuesta y dicta-

men. La coincidencia literal es distinta de la equivalencia operativa; una diferencia de redacción no implica necesariamente un cambio de comportamiento. Los criterios de comparación deben fijarse antes del análisis.

La **trazabilidad** consiste en poder relacionar una salida con su entrada, configuración y transformaciones registradas. En Qwen–CLIPS incluye los objetivos originales, su adaptación a hechos y las reglas activadas cuando se registren. En GPT directo permite examinar entrada y productos declarados, pero no reconstruir con certeza el razonamiento interno. Disponer de esa evidencia facilita analizar errores y no demuestra por sí mismo que la tarea se haya resuelto.

3.5.5. Validez de la respuesta del sistema

La evaluación contrasta el tipo de respuesta de GPT con la información disponible y los requisitos del caso, según las categorías definidas en la Sección 3.4.3. Una respuesta bien estructurada puede ser inadecuada si propone un plan sin resolver una ambigüedad o rechaza una tarea que sí puede atenderse.

Por ejemplo:

> *"Bring it to her."*

puede requerir una aclaración si no existe suficiente contexto para determinar qué representan *it* y *her*.

Por tanto, generar un plan no constituye siempre la respuesta correcta. Ante una orden ambigua o no soportada, solicitar una aclaración o rechazar la solicitud puede representar el comportamiento esperado del sistema.

Estos criterios también sirven para analizar los límites de Qwen–CLIPS, sin atribuirle por ello las mismas clases de respuesta conversacional. Una validación fallida, un objetivo sin soporte y un error de servicio deben distinguirse de una aclaración o un rechazo adecuados. El bloque principal evalúa tareas que exigen plan; los casos ambiguos o fuera de cobertura requieren criterios propios.

3.5.6. Condición de control para la etapa simbólica

Los objetivos de referencia aprobados se procesan mediante el mismo adaptador, reglas y estado inicial de Qwen–CLIPS. El control examina el procesamiento posterior a la interpretación y no constituye una cuarta configuración principal.

Solo procede cuando existe una referencia aprobada y aplicable. El plan del control se juzga contra los mismos requisitos de tarea, sin asumir que es automáticamente correcto. La ausencia de una referencia utilizable se registra como control no realizado y se distingue de un fallo del adaptador o de CLIPS.

Si el control genera un plan correcto pero una configuración Qwen–CLIPS falla para la misma instrucción, deben examinarse los objetivos predichos y su transformación en hechos para localizar el origen de la discrepancia.

Si el control también produce un plan incorrecto, deben revisarse la adaptación de los objetivos, las reglas de expansión y las condiciones iniciales utilizadas.

El análisis distingue los siguientes tipos de error:

- **Error de interpretación:** la secuencia G no representa correctamente la instrucción.
- **Error de adaptación:** la transformación de los objetivos en hechos no conserva correctamente la información contenida en G.
- **Error de planificación simbólica:** los objetivos son correctos y se encuentran adecuadamente representados mediante hechos, pero el plan P no constituye una expansión coherente con ellos y con las condiciones iniciales.

La condición de control apoya esta clasificación, pero la atribución de un fallo requiere examinar las representaciones intermedias y el plan generado. Esta separación se utiliza para analizar los resultados de la evaluación.

3.5.7. Planificación y ejecución física

Un plan puede fallar físicamente por percepción, navegación, manipulación o interacción. La comparación examina el cumplimiento simbólico bajo supuestos declarados; la experiencia en RoboCup 2026 se documenta separadamente y no equivale a una prueba física comparativa de ambas arquitecturas.

Los criterios de esta sección definen conceptualmente una interpretación correcta y un plan válido. Las métricas y procedimientos empleados para evaluarlos se presentan en el capítulo de validación y experimentación.

Capítulo 4

Metodología y diseño de las arquitecturas evaluadas

En esta investigación se desarrolló una arquitectura para transformar instrucciones en lenguaje natural en planes de acciones para el robot Justina, mediante la especialización de modelos Qwen como intérpretes semánticos y la expansión de objetivos mediante reglas CLIPS. Esta arquitectura emplea los modelos base Qwen2.5-3B y Qwen2.5-7B, especializados mediante QLoRA. Como arquitectura de comparación se utiliza GPT directo, que genera los objetivos y el plan durante una misma inferencia. De este modo, el diseño contempla dos arquitecturas y tres configuraciones experimentales.

Durante el desarrollo se adaptó el generador de comandos GPSR, se construyó un conjunto de datos con etiquetas semánticas, se implementó la interfaz entre interpretación y planificación y se incorporó el conocimiento operativo en ambas arquitecturas. Estas decisiones se apoyan en las representaciones y los criterios definidos en el Capítulo 3.

Este capítulo explica cómo se construyó el dominio de trabajo, qué modificaciones se realizaron para obtener el conjunto de datos y cómo se diseñó la comparación entre las arquitecturas. Los detalles de código, entrenamiento y despliegue se presentan en el capítulo 5. El protocolo de evaluación, las métricas y las condiciones concretas de los experimentos se desarrollan en el capítulo 6.

4.1. Diseño metodológico de la investigación

4.1.1. Problema experimental

El diseño compara el cumplimiento simbólico de las tres configuraciones y utiliza fidelidad semántica, robustez, consistencia, latencia y costo como dimensiones complementarias. Cada caso contiene una orden, su contexto y requisitos de tarea; cuando corresponde, incluye objetivos de referencia aprobados. Los casos ambiguos o fuera de capacidad cuentan con criterios de respuesta propios.

Se utiliza G_{pred} para los objetivos producidos por cada configuración y G_{ref} para su referencia. En Qwen–CLIPS se observan además la adaptación y el plan resultante; en GPT directo se analizan los objetivos y las acciones de una misma respuesta. La configuración de origen identifica cada registro, sin añadir notación específica para cada tamaño de modelo.

4.1.2. Variables y condiciones de comparación

El factor principal de comparación es la arquitectura completa. En Qwen–CLIPS, la interpretación se obtiene mediante un modelo especializado y la expansión se realiza con reglas explícitas. En GPT directo, ambas funciones se asignan a un modelo condicionado por una especificación del dominio y del comportamiento operativo esperado.

Dentro de Qwen–CLIPS se compara la variante del intérprete semántico, Qwen2.5-3B o Qwen2.5-7B. Los ajustes utilizan un corpus común y el mismo procedimiento de partición. La comparación mantiene fijos los procedimientos de normalización y adaptación, las reglas y las condiciones iniciales de planificación. Las condiciones comunes de especialización se describen en la Sección 4.4.2; sus valores técnicos se reúnen en la Tabla 5.1.

Las variables de respuesta comprenden el cumplimiento simbólico, la fidelidad de los objetivos, la conformidad con el contrato, la robustez lingüística, la consistencia entre ejecuciones y el rendimiento y costo observables. La familia de tarea, la longitud de la secuencia de objetivos, los tipos de entidades y las dependencias referenciales se utilizan para organizar el análisis. Las variantes que expresan una misma intención se mantienen identificadas como tales para interpretar adecuadamente los resultados por caso y por grupo de reformulaciones.

El rendimiento computacional se examina mediante la latencia observada desde los

programas experimentales para las tres configuraciones. Adicionalmente, se compara el consumo de recursos de ambas variantes Qwen sobre la misma plataforma y bajo condiciones comunes de inferencia. Para GPT directo mediante API no se incluyen mediciones internas de memoria o procesamiento del modelo. Las métricas y los procedimientos de medición se especifican en el capítulo 6.

La comparación se delimita a instrucciones en inglés, un vocabulario semántico común y un repertorio compartido de acciones. El diseño exige igualdad de catálogos, configuración de manipulación y convenciones de inicio y finalización. Estas condiciones permiten plantear la misma tarea a ambos sistemas, aunque cada uno utilice mecanismos distintos para representar y aplicar el conocimiento.

Dado que entre las arquitecturas cambian también el modelo de lenguaje y su adaptación, las diferencias observadas no pueden atribuirse exclusivamente a la presencia de reglas. Las conclusiones se refieren al comportamiento de las configuraciones evaluadas dentro del dominio establecido.

4.1.3. Estrategia general de evaluación

La unidad de observación es el procesamiento de una orden por una configuración en una repetición. En el bloque principal, el dictamen es binario: cumplimiento completo cuando el plan satisface todos los requisitos, e incumplimiento si omite alguno o no se obtiene un plan evaluable. La cobertura parcial se utiliza para diagnosticar errores.

La comparación empareja las mismas órdenes y considera la dependencia entre reformulaciones y repeticiones de un escenario. El Capítulo 6 establece denominadores, agregación e intervalos de confianza.

El control auxiliar procesa referencias aprobadas mediante el mismo adaptador, reglas y estado inicial de Qwen–CLIPS. Se distinguen referencias aplicables, controles realizados, terminados y correctos. La ausencia de referencia no se cuenta como fallo ni como acierto del control.

Se separaron los datos utilizados para ajustar o corregir componentes de los destinados a la comparación. La revisión de referencias precede a la observación de respuestas. La experiencia física en RoboCup se documenta aparte, conforme al alcance del capítulo 1.

4.2. Construcción del dominio experimental

4.2.1. Fuentes de conocimiento GPSR

Se tomó como punto de partida el CommandGenerator oficial de RoboCup@Home y la organización del conocimiento de CompetitionTemplate, descritos en la Sección 3.2.2 (19, 20). El primero aporta estructuras de comandos y familias de tareas; el segundo proporciona las entidades con las que se construyen las instrucciones.

Sobre esta base se desarrolló una adaptación destinada a producir pares de órdenes y objetivos canónicos. Se incorporaron mecanismos para conservar el contexto entre subtareas, generar las etiquetas semánticas, introducir variantes lingüísticas y controlar la distribución y calidad de las muestras. De esta forma, la herramienta de generación se integró en un procedimiento de construcción de datos para la especialización del intérprete.

La separación entre estructuras de tareas y catálogos se conservó en la adaptación. Esto permite modificar las entidades de una instancia sin alterar necesariamente la composición de sus objetivos. El alcance lingüístico se mantiene conforme a la Sección 3.2.3: las plantillas sirven para construir datos controlados, mientras que el sistema busca interpretar otras formulaciones que expresen intenciones pertenecientes al mismo dominio.

4.2.2. Catálogos de personas, objetos y ubicaciones

El conocimiento K, definido en la Sección 3.3.4, se concretó mediante catálogos de nombres de personas, objetos, categorías, habitaciones y ubicaciones. Se incluyeron formas singulares y plurales de categorías, así como las descripciones admitidas para búsquedas de personas y selección de objetos.

Se implementó una validación de los catálogos para detectar condiciones que comprometen la generación de etiquetas, como un catálogo esencial vacío o una entidad clasificada simultáneamente como persona y objeto. También se incorporó una comparación de nombres con el banco de conocimiento de Justina para identificar diferencias entre la generación de datos y el sistema integrado.

En la generación de etiquetas, las entidades se clasifican mediante los catálogos y el contexto semántico, con tipos explícitos en `find` y `count`. La capitalización no determina por sí sola esos tipos: un objeto puede tener un nombre con mayúscula

inicial. Este criterio de generación debe distinguirse de las heurísticas del adaptador posterior, que todavía utiliza nombres, capitalización y contexto para inferir tipos y puede alterar el declarado en la etiqueta o la predicción.

También se distinguió una ubicación de navegación de un soporte de colocación: la primera indica dónde debe desarrollarse una subtarea; el segundo especifica una superficie o recipiente relacionado con la manipulación.

La configuración de evaluación identifica la versión de los catálogos utilizados en la generación y en las instrucciones de GPT. La alineación del adaptador se comprueba mediante sus transformaciones y hechos de salida, pues no consulta directamente esos catálogos.

Los catálogos concretan K, pero el contexto C y el contrato $\mathcal{D}$ se documentan por separado, conforme a la Sección 3.3.1. Para cada caso deben fijarse el estado inicial S_0, los supuestos de identidad y ubicación y las condiciones necesarias para juzgar los efectos de las acciones. Compartir nombres de entidades no demuestra por sí solo que ambas arquitecturas reciban condiciones equivalentes.

En la comparación computacional, S_0 se fija con ubicación desconocida y ausencia de objeto sostenido. Este valor inicial corresponde a la representación del planificador. En Justina, las herramientas de navegación y manipulación manejan la información de estado necesaria para resolver las operaciones solicitadas; la relación con el estado previsto por CLIPS se describe en la Sección 5.4.2.

4.2.3. Familias de tareas

El generador adaptado contempla 29 familias de comandos, organizadas en dos categorías principales: tareas de personas y tareas de objetos. Estas categorías sirven para distribuir las muestras según la familia generadora. Una tarea de objetos puede incluir un destinatario humano, por lo que la clasificación no exige que todas las entidades de la instrucción pertenezcan a un mismo tipo.

Se incluyeron tareas de navegación, búsqueda, saludo, seguimiento, guía, conteo, adquisición y comunicación de información, y transporte de objetos. Durante la adaptación se añadieron familias simples para buscar una persona, buscar un objeto y saludar a una persona en una habitación. Su incorporación permite representar estas intenciones como órdenes completas, además de su aparición dentro de instrucciones compuestas.

Para distinguir las composiciones alcanzables dentro de una familia se utilizó la firma de objetivos: una secuencia que conserva los nombres de los objetivos y omite

sus argumentos concretos. Por ejemplo, [go, find] identifica una búsqueda precedida de navegación. Las firmas permiten distribuir muestras según la estructura de la tarea y analizar diferencias que la clasificación por familia, por sí sola, puede ocultar.

4.2.4. Unidades de organización y análisis

Para describir la cobertura se distinguen las siguientes unidades:

- **Familia:** ruta o estructura generadora de tareas. Puede producir varias composiciones y entidades.
- **Firma:** secuencia de tipos de objetivos sin sus argumentos. Dos tareas con la misma firma pueden tener restricciones o dependencias diferentes.
- **Intención:** contenido solicitado, incluidas las subtareas y sus relaciones; no se identifica únicamente mediante la firma.
- **Escenario:** tarea concreta con entidades, restricciones y condiciones iniciales fijadas. Puede comunicarse mediante varias formulaciones equivalentes.
- **Orden:** formulación lingüística utilizada como entrada en un caso.
- **Ejecución experimental:** procesamiento de una orden por una configuración en una repetición determinada; no significa ejecución física del plan.

Estos identificadores deben permitir agrupar formulaciones y repeticiones sin contarlas como tareas independientes. Cambiar un nombre u objeto puede producir una nueva instancia sin crear una familia o una estructura nueva. Los criterios de agrupación y de análisis estadístico se fijan en el protocolo experimental.

4.2.5. Etiquetas semánticas para entrenamiento

Las etiquetas del conjunto de entrenamiento utilizan el vocabulario canónico de la Tabla 3.1 y conservan el orden de los objetivos. Se denotan mediante G_{etq} para distinguir su procedencia de las referencias experimentales G_{ref}. Para construirlas se desarrollaron funciones semánticas asociadas con las familias de comandos, que transforman las entidades y relaciones del contexto de generación en objetivos.

La construcción es programática: la etiqueta se obtiene del contexto con el que se produce la instrucción, sin solicitar una interpretación al modelo sometido al ajuste. En las tareas compuestas, las funciones propagan el objeto o la persona de referencia hacia las subtareas posteriores. Los objetivos `find` y `count` incluyen el tipo mediante `kind`, mientras que los argumentos conservan los destinos, destinatarios y atributos

relevantes.

La Figura 4.2 muestra dos formulaciones del corpus que comparten etiqueta, junto con los campos de generación y metadatos que las vinculan.

La revisión manual inicial de pares orden–objetivos fue parcial; se complementó con comprobaciones automatizadas de etiquetas y propiedades de generación, sin constituir una anotación independiente y exhaustiva del corpus.

Los controles de la Sección 4.3.7 detectan errores de contrato y de composición, pero no sustituyen una comparación semántica independiente entre cada instrucción y su etiqueta. Un error compartido por la función generadora y sus controles puede propagarse a varias muestras.

Las intenciones de comunicación de información se representan mediante `tell` y las de colocación mediante `place`, conforme al vocabulario de la Tabla 3.1. Se mantiene la distinción entre objetivos y acciones ejecutables: la expansión de `place` puede incluir una acción `drop` del ejecutor.

4.2.6. Referencias para la evaluación comparativa

Las referencias finales comparten el formato canónico de las etiquetas, pero requieren una revisión de la orden y de sus condiciones de escenario antes de observar las respuestas de los sistemas. Una etiqueta generada puede servir como propuesta; su origen automático no acredita que sea una referencia aprobada.

Cada caso debe establecer las subtareas, restricciones, dependencias y clases de respuesta admisibles. Cuando proceda un plan, se revisan los objetivos de referencia y los efectos requeridos, admitiendo alternativas que cumplan la tarea. Cuando proceda aclaración o rechazo, se fijan los criterios para juzgar su adecuación. Las referencias y decisiones de revisión deben conservar su procedencia y versión.

La aprobación no se condiciona a que CLIPS resuelva el caso. De este modo, el conjunto final puede mostrar errores de adaptación o falta de cobertura simbólica, en lugar de excluirlos de antemano. El control auxiliar utiliza solo referencias aprobadas y aplicables y se evalúa contra los requisitos de la tarea.

4.3. Construcción del conjunto de datos

La Figura 4.1 resume el procedimiento de generación y aceptación de muestras. La comprobación adicional con CLIPS examina la expansión de un subconjunto de

secuencias, según el recorrido descrito en la Sección 4.3.7.

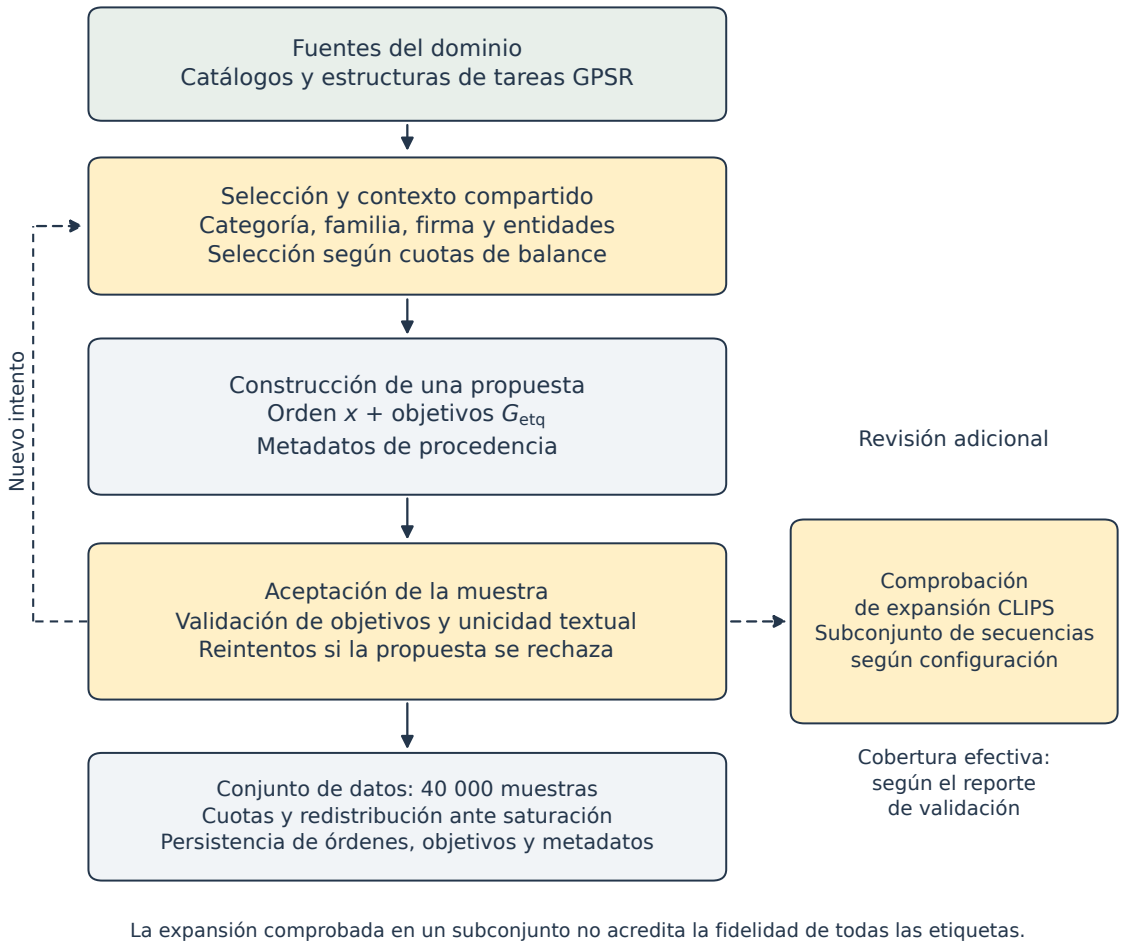


Figura 4.1: Construcción del conjunto de datos a partir de catálogos y estructuras de tareas. El balance guía la selección y redistribución de cuotas; la comprobación con CLIPS examina la expansión de las secuencias seleccionadas. Elaboración propia.

4.3.1. Adaptación del CommandGenerator

La Figura 4.2 ilustra la separación entre las plantillas lingüísticas y la función semántica de una familia. Ambas utilizan los mismos campos de contenido para construir cada par de entrenamiento. Añadir una formulación para una intención existente no exige redefinir sus objetivos; una tarea nueva sí requiere especificar su composición.

La Tabla 4.1 recoge las modificaciones realizadas y sus propósitos. Incluye la solicitud de captura de información mediante `save` antes de `tell`, el destino `at` para el transporte

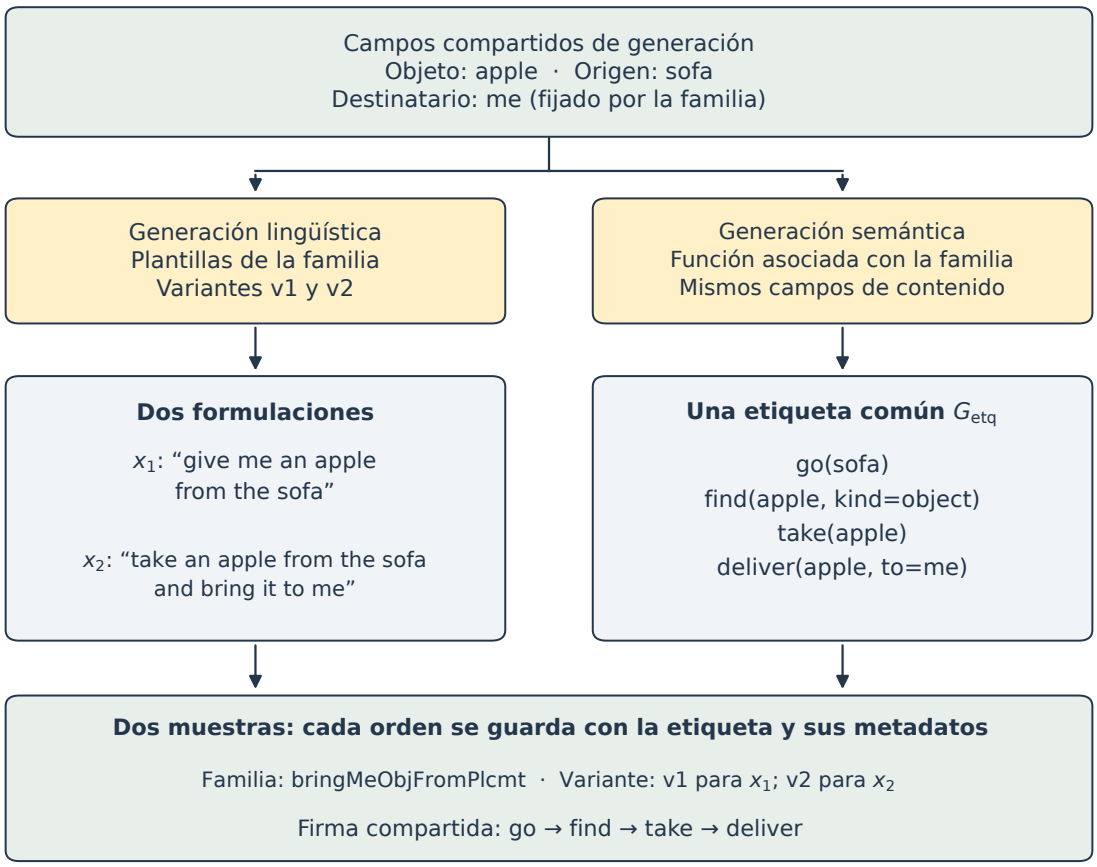


Figura 4.2: Construcción de dos muestras de entrenamiento con campos compartidos y variantes lingüísticas distintas. Las órdenes y sus etiquetas corresponden a dos filas del corpus; se verificaron las plantillas y la función semántica que las construyen. La etiqueta se obtiene del contexto de generación, sin inferencia de Qwen ni expansión a un plan. Elaboración propia.

entre ubicaciones sin imponer una relación de apoyo sobre una superficie, y los cambios de cobertura y distribución. También se reclasificó una familia de adquisición de objetos para asignarla a la categoría correspondiente. La estructura de archivos y funciones se presenta en el capítulo 5.

Tabla 4.1: Modificaciones incorporadas en la adaptación del generador.

Aspecto	**Modificación**	**Propósito**
Construcción de etiquetas	Funciones semánticas asociadas con familias y subtareas.	Obtener objetivos a partir del contexto utilizado para construir la orden.
Contexto compartido	Propagación del objeto o la persona entre subtareas relacionadas.	Mantener la identidad de las entidades durante la composición de la tarea.
Variación lingüística	Separación de las plantillas superficiales y la generación semántica.	Variar la formulación conservando la intención y sus argumentos.
Cobertura de tareas	Incorporación de familias simples de búsqueda y saludo.	Representar esas intenciones como órdenes completas, además de subtareas.
Composición semántica	Captura de información con `save` y representación del destino de transporte mediante `at`.	Conservar operaciones y relaciones necesarias antes de informar o colocar.
Distribución y control	Balance por categoría, familia y firma; deduplicación y validación.	Controlar la cobertura, evitar repeticiones y detectar inconsistencias.

4.3.2. Generación de órdenes

Cada muestra contiene la instrucción, los objetivos y metadatos que identifican su procedencia. Se registran la familia, la categoría, la variante superficial, la firma de objetivos, los tipos de entidades y los campos utilizados. Esta información permite analizar la cobertura del conjunto y agrupar los errores por características de la tarea.

Se incorporaron una semilla de generación y un límite de reintentos para controlar el procedimiento. La muestra se acepta después de comprobar su etiqueta y su condición de unicidad. Cuando una combinación no cumple los criterios, se realiza un nuevo intento dentro de los límites establecidos. La reproducción del conjunto depende de conservar, además de la semilla, las mismas versiones del generador y de sus catálogos.

4.3.3. Generación de objetivos canónicos

Cada propuesta de G_{etq}, construida mediante las funciones de la Sección 4.2.5, se somete al validador de objetivos antes de aceptarse. Este comprueba argumentos y relaciones admitidas, además de ciertas dependencias de la secuencia.

El validador parte de una colección vacía de objetos adquiridos y exige un `take` previo del objeto que se entrega o coloca. Es un supuesto de validación del dataset, no una regla universal: si un escenario declara que el robot ya sostiene el objeto en S_0, la adquisición previa puede ser innecesaria. Ese caso requiere un tratamiento explícito, pues el validador no recibe dicho estado inicial.

Para comparar representaciones se distingue la escritura de las entidades de su contenido semántico. La normalización de mayúsculas puede utilizarse cuando preserve la identidad establecida por el catálogo. En todos los casos se mantienen el tipo, los valores de los argumentos, las relaciones y el orden de los objetivos, de modo que una normalización superficial no oculte una diferencia de significado.

4.3.4. Variación lingüística

El corpus se construyó con tres plantillas superficiales por cada una de las 29 familias. Estas se instancian con distintas entidades y combinaciones, como muestra la Figura 4.2. A partir de ellas se generaron 40 000 muestras.

Se implementó una comprobación de los campos de las variantes al iniciar la generación. Su finalidad es detectar omisiones o incorporaciones de información incompatibles con la familia. La conservación de esos campos no equivale a una demostración de equivalencia lingüística: el orden, el alcance de las expresiones y las relaciones entre subtareas también determinan el significado de la instrucción.

La diversidad introducida mediante plantillas se considera variación controlada dentro del dominio. Para estudiar formulaciones que no procedan de esas plantillas, el diseño de evaluación contempla un grupo separado de reformulaciones elaboradas de manera independiente del entrenamiento. Esta distinción permite analizar por separado la interpretación de estructuras conocidas y la respuesta ante nuevas formas de expresar las mismas tareas.

4.3.5. Balance y deduplicación

El conjunto de datos se construyó con 40 000 muestras. Para distribuir la generación se asignó la mitad a familias de personas y la mitad a familias de objetos. Dentro de cada categoría se establecieron cuotas por familia y por firma de objetivos, con el propósito de reducir la concentración de ejemplos en las estructuras con mayor probabilidad de ser seleccionadas.

La deduplicación se realizó sobre la instrucción convertida a minúsculas y con espacios normalizados. Cuando una misma entrada aparecía asociada con objetivos diferentes, el procedimiento la identificaba como un conflicto de etiqueta y la rechazaba. Este criterio evita repeticiones textuales normalizadas y etiquetas contradictorias para una misma entrada, aunque no agrupa todas las expresiones semánticamente equivalentes.

Se incorporó una redistribución de cuotas para las familias cuyo número de combinaciones únicas resulta limitado. Al alcanzar el límite de reintentos, las muestras faltantes se reasignan dentro de la misma categoría, dando prioridad a las familias y firmas menos representadas. Así se conserva la distribución entre tareas de personas y objetos sin introducir duplicados para completar una cuota.

El balance resultante debe interpretarse en estos dos niveles: la proporción entre categorías se mantiene, mientras que la distribución entre familias puede variar debido a la saturación. El capítulo 5 presenta el tamaño del corpus y el número de familias, firmas y variantes superficiales identificadas en la auditoría.

4.3.6. Particiones de ajuste y separación de la prueba final

La Figura 4.3 distingue la partición del corpus de ajuste, las pruebas de desarrollo y la prueba final. Ambos entrenamientos utilizaron el mismo archivo de 40 000 muestras. El procedimiento de partición divide aleatoriamente las filas con semilla 42 en 36 000 ejemplos de entrenamiento y 4 000 de validación. Esta última se utiliza para seleccionar el checkpoint y forma parte del desarrollo de los intérpretes.

La eliminación de duplicados exactos no impide que ambas particiones compartan plantillas o contengan formulaciones de una misma intención. Por ello, los resultados sobre esta división describen el comportamiento de los modelos bajo la distribución generada y no bastan para establecer su capacidad de generalización a formulaciones independientes.

La auditoría de la prueba final debe distinguir coincidencias literales, normalizadas

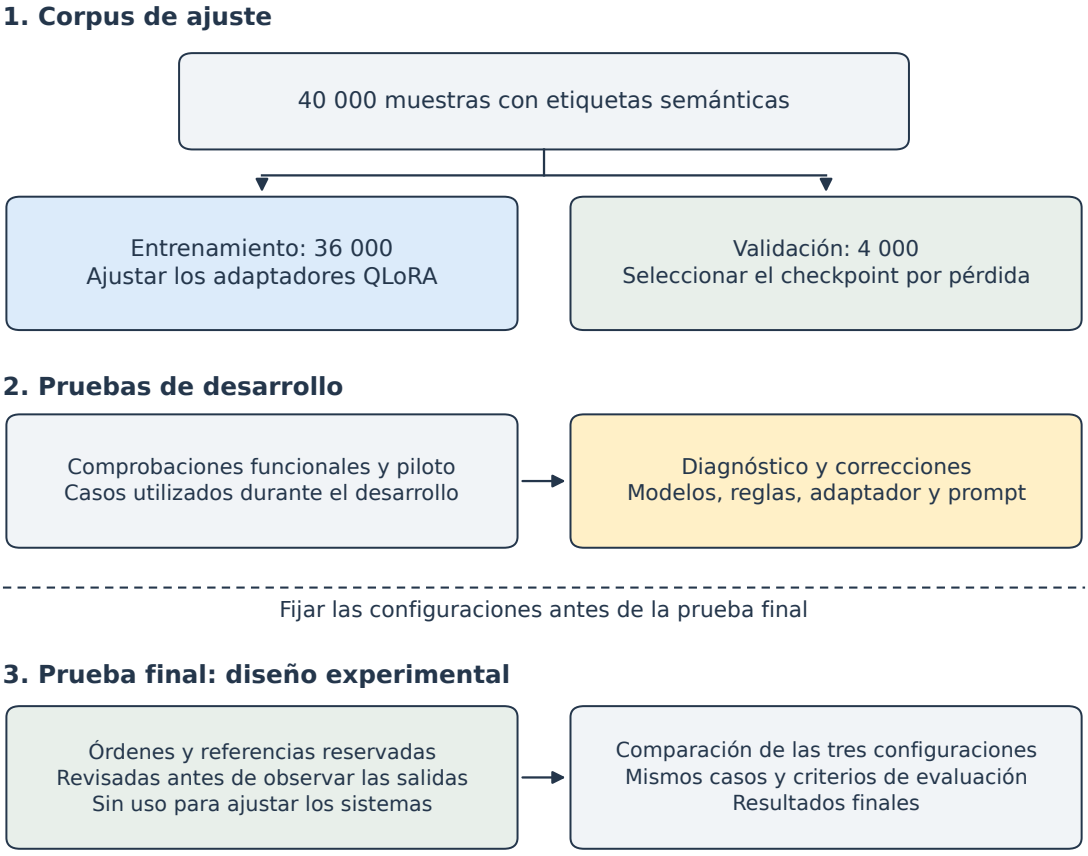


Figura 4.3: Función de los datos de ajuste, desarrollo y evaluación final. El corpus de ajuste permite entrenar y seleccionar los adaptadores; las pruebas de desarrollo orientan las correcciones. La prueba final se reserva para comparar configuraciones fijadas sobre órdenes y referencias revisadas. Elaboración propia.

y estructurales con el corpus de ajuste y con los datos de desarrollo. La ausencia de duplicados textuales no acredita familias inéditas. Si se comparten familias o firmas entrenadas, las conclusiones deben referirse a nuevas instancias o formulaciones de esas estructuras, según la evidencia disponible. No se conoce el posible solapamiento con los datos de entrenamiento del proveedor GPT.

La prueba final se reserva para la comparación conforme al esquema de la Figura 4.3. Las reformulaciones de una misma intención se identifican como grupo y el procedimiento audita su relación con los datos de desarrollo. El procedimiento y las condiciones de independencia se especifican en el Capítulo 6.

Durante el desarrollo se utilizaron además 48 comandos para comprobar el funcionamiento del sistema. Estos casos constituyen una prueba funcional y se mantienen separados de la evaluación experimental. Su descripción corresponde a la verificación de la implementación.

El protocolo final recoge además un piloto de 60 órdenes utilizado para desarrollar la comparación. Los 48 comandos funcionales y las 60 órdenes tienen propósitos diferentes y no se suman para describir la diversidad de tareas. Los ejemplos usados para corregir los sistemas, incluidos los del piloto, se mantienen fuera de la prueba final.

4.3.7. Validación previa del conjunto de datos

Los controles automatizados complementan la revisión descrita en la Sección 4.2.5. Se distinguen la validación de catálogos, los controles sobre las secuencias de objetivos y la comprobación de expansión con CLIPS; cada nivel examina una propiedad específica del conjunto.

El validador de objetivos comprueba la sintaxis, los nombres y argumentos admitidos, la presencia de destinos obligatorios y el tipo explícito en `find` y `count`. También detecta calificadores incompatibles con el tipo declarado y algunos conflictos de tipo entre subtareas. El control de adquisición previa sigue el supuesto del dataset descrito en la Sección 4.3.3. También se detectan ciertas navegaciones o destinos redundantes. Estos controles operan sobre los objetivos; no interpretan de nuevo la instrucción para contrastar su significado.

Se incorporaron además pruebas sobre propiedades de generación, como la cardinalidad solicitada, la ausencia de entradas textuales duplicadas, la proporción entre categorías y la presencia de metadatos. Algunas pruebas contienen expectativas específicas: las familias de búsqueda simple deben producir la firma [go, find], mientras

que la de saludo simple debe producir [go, find, greet]. Otras comprueban la presencia de `save` y del retorno al punto de instrucción en las tareas de información, o el uso de `at` en el transporte entre ubicaciones. Estas expectativas están codificadas como propiedades y firmas.

La comprobación de expansión con CLIPS se aplica a un subconjunto de secuencias. Para cada una, el validador reinicia el entorno, convierte los objetivos en hechos y ejecuta las reglas. Se comprueba que no queden objetivos pendientes, que no se emita la respuesta correspondiente a un objetivo no soportado y que se produzca la señal de finalización `GPSR_DONE`. La aceptación indica que la secuencia satisface estas condiciones de expansión, sin acreditar por sí sola el cumplimiento semántico o físico de la tarea.

La auditoría recorre etiquetas válidas en orden de archivo hasta un límite configurado; las pruebas del generador seleccionan un ejemplo por familia del lote. Son controles parciales de expansión. Los resultados y la cobertura comprobada se presentan en la Sección 5.2.

Los metadatos describen procedencia y estructura, pero no registran el resultado individual de CLIPS. La evaluación independiente utiliza referencias revisadas a partir de la intención y las capacidades declaradas, sin condicionar su aprobación a que CLIPS resuelva el caso.

4.4. Diseño de la arquitectura Qwen–CLIPS

4.4.1. Función asignada a Qwen

Se especializaron las variantes base Qwen2.5-3B y Qwen2.5-7B para producir objetivos canónicos serializados en JSON. Ambas comparten representación, corpus y procesamiento simbólico posterior. La interfaz permite examinar la interpretación antes de su expansión en acciones.

4.4.2. Especialización para interpretación semántica

Ambas variantes base se especializan con el mismo procedimiento supervisado. La comparación no incluye variantes Instruct y, por tanto, no evalúa qué punto de partida sería superior para esta tarea.

La especialización se realizó mediante aprendizaje supervisado con los pares de instrucciones y objetivos construidos para el dominio. Cada entrada se presentó al modelo

como contexto y la secuencia canónica como respuesta esperada. Los metadatos de generación se conservaron para el análisis del conjunto y no se incluyeron en la respuesta que el modelo aprende a producir.

Para concentrar el ajuste en la generación de los objetivos, se calculó la pérdida sobre los tokens de la respuesta, excluyendo las posiciones del prompt y del relleno. Se utilizó QLoRA (7) con el modelo base cuantizado a cuatro bits y adaptadores entrenables, lo que permitió especializarlo dentro de los recursos de cómputo disponibles.

Para mantener condiciones comparables, ambos ajustes compartieron el número de épocas, la tasa de aprendizaje, la configuración de los adaptadores y un lote efectivo de ocho muestras. El lote por dispositivo y la acumulación de gradientes se ajustaron para cada variante, conservando ese tamaño efectivo. Sus valores se presentan en la Tabla 5.1; la organización de los datos se describe en la Sección 4.3.6.

La selección se realizó por separado para cada modelo mediante el menor valor de pérdida de validación (`eval_loss`), con restauración automática del checkpoint seleccionado al terminar. Compartir el criterio no implica seleccionar el mismo paso de entrenamiento.

Durante el desarrollo se exploró inicialmente Qwen2.5-0.5B. Las pruebas realizadas mostraron limitaciones para las órdenes complejas consideradas, por lo que se continuó con Qwen2.5-7B. Posteriormente se incorporó Qwen2.5-3B para examinar las diferencias de calidad y rendimiento respecto de la variante de mayor tamaño, manteniendo el mismo procedimiento de especialización y la misma etapa simbólica. La exploración con Qwen2.5-0.5B se conserva como antecedente de desarrollo y no forma parte de la comparación experimental principal.

Los hiperparámetros, las configuraciones de inferencia y la identificación de los adaptadores y checkpoints seleccionados para cada modelo se presentan en el capítulo 5, junto con el procedimiento de entrenamiento.

4.4.3. Interfaz entre objetivos y planificación simbólica

La interfaz distingue cuatro operaciones, de acuerdo con el flujo conceptual del capítulo anterior:

1. **Normalización de entrada:** tratamiento de la orden antes de consultar a Qwen. Los registros conservan la entrada original y la normalizada para identificar su intervención.
2. **Validación y normalización de salida:** comprobación del formato y corres-

pondencia de nombres con identificadores admitidos, sin tratar como equivalentes valores con significados distintos.

3. **Reparación:** inferencias de tipo o cambios en determinadas secuencias que pueden modificar la predicción. Su efecto debe registrarse por separado.
4. **Conversión a hechos:** representación de los objetivos aceptados en estructuras que CLIPS puede procesar. Su corrección exige conservar orden, entidades, atributos y relaciones; esa conservación debe comprobarse.

El adaptador desarrollado incorpora normalización, inferencias de tipo y reparaciones para determinadas secuencias. Estas operaciones forman parte del sistema evaluado. Un objetivo corregido puede mejorar el resultado final sin constituir un acierto original del intérprete. Por ello, se distinguen G_{pred}, la representación posterior al procesamiento y los hechos F entregados a CLIPS. En el programa experimental, G_{pred} corresponde a los objetivos extraídos de la respuesta del servicio de inferencia utilizado, antes de su adaptación local; la respuesta HTTP conservada no equivale al texto bruto completo generado por el modelo.

La conversión se evalúa por la conservación de entidades, atributos, relaciones y orden, considerando las heurísticas de tipo señaladas en la Sección 4.2.2. Perder un destino, confundir referentes u omitir una restricción solicitada constituye un error de adaptación. La Sección 4.6.3 establece cómo se examinan esas transformaciones; sus mecanismos y formatos se describen en la implementación.

4.4.4. Función asignada a CLIPS

Se implementaron reglas para expandir los objetivos en su orden de entrada, utilizando ubicación, posesión y contexto de interacción. Se incorporaron anuncios, control del foco y retorno al punto de instrucción como convenciones del contrato común. El estado y las precondiciones modelados son parciales; sus mecanismos y límites se presentan en la Sección 5.4.2.

Una explicación hablada de que una subtarea no está soportada no sustituye su cumplimiento en el bloque principal. En los casos de reconocimiento de límites, se juzga la adecuación de la respuesta conforme a la ficha del caso.

4.5. Diseño de la arquitectura GPT directo

4.5.1. Funciones asignadas al modelo

Se implementó GPT directo para generar objetivos y acciones en una misma respuesta, sin expansión con CLIPS. La especificación se elaboró a partir del dominio y de las convenciones operativas de Justina. Los dos productos se conservan para evaluar fidelidad y coherencia, conforme a la arquitectura del Capítulo 3.

4.5.2. Conocimiento y restricciones proporcionadas

Para orientar la generación se elaboró un prompt con los catálogos de entidades, el vocabulario de objetivos, las acciones disponibles y las restricciones de composición. Se incluyeron criterios sobre resolución de referencias, dependencias de manipulación, relaciones espaciales e interacción con personas.

La especificación también establece cómo conservar coherencia con el estado esperado durante la generación. El modelo recibe instrucciones para considerar la ubicación, el objeto sostenido y las entidades identificadas al determinar las acciones posteriores. Se incorporaron las convenciones de identificación, inicio y finalización del plan para facilitar su compatibilidad con el ejecutor.

El conocimiento operativo se presenta así mediante instrucciones en la arquitectura directa y mediante reglas en el componente CLIPS. El registro de comparación identifica la versión del modelo, el prompt y sus configuraciones de generación, evitando modificar estas condiciones a partir de los resultados de la prueba final. El identificador de GPT, la versión del prompt y los parámetros de generación aplicables se documentan en el Capítulo 5; la política de reintentos corresponde al Capítulo 6.

La especificación debe reflejar el contrato del ejecutor, incluidas sus limitaciones confirmadas. Las reglas CLIPS son una fuente para recuperar convenciones, pero sus salidas no se adoptan automáticamente como soluciones correctas ni como secuencias obligatorias para GPT. No se incluyen las referencias ni los dictámenes esperados de los casos finales en las entradas del modelo.

4.5.3. Contrato de salida

Se definió una respuesta con estado, mensaje, objetivos, plan y marcas de inicio y cierre. JSON Schema restringe la estructura; los controles locales comprueban campos

y convenciones del contrato. El esquema y los validadores se describen en la Sección 5.8.

Los estados son `executable`, `clarification` y `rejected`. Los dos últimos requieren un mensaje y no contienen acciones. En el bloque principal, aclarar innecesariamente o rechazar una orden resoluble cuenta como ausencia de plan correcto. Los casos ambiguos o fuera del dominio se analizan por separado, sin equiparar un error de validación de Qwen–CLIPS con una aclaración conversacional.

4.6. Diseño de la comparación

Para realizar la comparación se emplean dos programas independientes del sistema de ejecución física del robot. Uno implementa la cadena Qwen–CLIPS y se utiliza con Qwen2.5-3B y Qwen2.5-7B; el otro consulta al planificador GPT directo y valida su respuesta. De este modo, los dos programas permiten evaluar las tres configuraciones sobre los mismos casos de prueba.

Ambos emplean convenciones comunes para registrar las entradas, los objetivos interpretados, los planes generados y los fallos. Esta organización permite aplicar criterios de evaluación comunes sin depender de los módulos de percepción, navegación y manipulación de Justina.

4.6.1. Igualdad del dominio de evaluación

El procedimiento aplica a las tres configuraciones las mismas órdenes, contextos y referencias, bajo las condiciones de la Sección 4.1. Cada caso conserva su familia, firma de objetivos y procedencia lingüística. La aprobación de referencias sigue la Sección 4.2.6, y la independencia respecto del desarrollo se establece conforme a la Sección 4.3.6.

El registro conserva también la entrada efectiva de cada arquitectura. La normalización previa puede modificar aspectos superficiales o resolver ciertas variaciones antes de la inferencia, por lo que forma parte de la configuración evaluada. La medición de latencia considera el recorrido completo de la solicitud y distingue, cuando es posible, el procesamiento del modelo de la comunicación y la expansión.

La ubicación del cómputo se trata como una condición de despliegue. El uso de Qwen con pesos bajo control del proyecto permite alojar la inferencia en infraestructura propia, incluida una computadora remota. Por ello, su comparación temporal con GPT debe considerar las condiciones de comunicación utilizadas en cada caso.

4.6.2. Igualdad del repertorio de acciones

El contrato común define acciones, argumentos, efectos esperados, capacidades de manipulación y convenciones de inicio y cierre. La evaluación distingue su cumplimiento del seguimiento literal de anuncios y numeración. Se admiten planes alternativos que satisfacen la tarea: reproducir CLIPS no es el criterio de corrección.

Las operaciones deben conservar los atributos y referentes solicitados. Anunciar una tarea no sustituye su realización, ni una consulta aproximada acredita una propiedad distinta. La Tabla 5.4 reúne las limitaciones relevantes para aplicar este criterio.

4.6.3. Separación entre interpretación y planificación

El procedimiento evalúa los objetivos $G_{\text{pred}}^{\text{Qwen}}$ y $G_{\text{pred}}^{\text{GPT}}$ frente a G_{ref}, y los planes frente a los objetivos declarados y a los requisitos de tarea, siguiendo la distinción conceptual de la Sección 3.5.3.

En Qwen–CLIPS se registran las transformaciones entre los objetivos recibidos antes de la adaptación local, su representación adaptada y los hechos. El análisis cuantifica los cambios de entidades, tipos, atributos, relaciones u orden, y examina si esos cambios conservan los requisitos de la instrucción. Cuando exista una referencia aplicable, ambas representaciones se contrastan con ella y se analiza cómo las discrepancias aparecen en el plan. Las diferencias de formato se distinguen de las pérdidas o alteraciones de significado.

El control presentado en la Sección 4.1.3 apoya la localización de errores posteriores a la interpretación. Las transformaciones observadas permiten describir la intervención del adaptador, pero no estimar por sí solas una mejora causal del sistema: esa conclusión requeriría una comparación adicional que aislara su intervención.

Las diferencias entre el programa experimental y la integración con ROS 2 se documentan para delimitar qué resultados de la comparación pueden trasladarse al sistema utilizado en Justina.

En GPT directo se analizan los objetivos y las acciones de una misma respuesta. La clasificación permite describir qué producto incumple los criterios de validez, aunque no identifica con certeza una etapa causal interna del modelo. En ambas arquitecturas se conservan los casos inválidos y los fallos de interfaz como parte del registro experimental.

4.6.4. Registro previo y límites de la comparación

El registro de la comparación identifica las versiones de datos, catálogos, modelos, adaptadores, reglas, prompt, esquema y validadores. Cada caso conserva su contexto, estado inicial, condiciones de inferencia y criterios de revisión. La implementación de estos registros se describe en el Capítulo 5; el tamaño del conjunto final, las repeticiones, los tiempos límite, los reintentos y los métodos de análisis corresponden al Capítulo 6.

El alcance de la comparación queda delimitado por la revisión del corpus descrita en la Sección 4.2.5, la variación controlada y la partición de ajuste de la Sección 4.3.6, y los mecanismos de adaptación y seguimiento parcial del estado. Los resultados caracterizan sistemas completos en el dominio evaluado y no aíslan el efecto del modelo, del ajuste o de las reglas. La prueba final se separa del desarrollo local, sin conocer el posible solapamiento con los datos del proveedor GPT. La evaluación computacional no mide éxito de ejecución física. Estas condiciones guían la interpretación de los resultados y el análisis de los límites de cobertura.

Con esta organización, el diseño relaciona la construcción de datos, la especialización del intérprete y la generación de planes con criterios de evaluación diferenciados. El capítulo siguiente presenta la implementación de los componentes desarrollados y su integración con Justina.

Capítulo 5

Implementación

Este capítulo describe los componentes desarrollados para transformar instrucciones del lenguaje natural en planes de acciones para el robot de servicio Justina. La implementación concreta las decisiones metodológicas del Capítulo 4 mediante la generación de datos supervisados, la especialización de los intérpretes Qwen2.5-3B y Qwen2.5-7B, la conversión de objetivos a hechos simbólicos y la expansión de acciones con CLIPS.

El sistema se organiza en componentes con responsabilidades diferenciadas. La construcción del conjunto de datos y el entrenamiento se realizan fuera del ciclo de operación del robot. Durante la planificación, el intérprete recibe una instrucción y devuelve objetivos que se validan y convierten antes de ingresar al motor simbólico. El plan resultante se publica para su consumo por el sistema de ejecución. La arquitectura GPT directo se trata como una implementación separada, con su propio contrato de respuesta e interfaz de generación.

Como ejemplo conductor se utiliza una orden de búsqueda, adquisición y entrega de un objeto al operador. Sus diferentes representaciones permiten relacionar la entrada lingüística, los objetivos, los hechos CLIPS y las acciones del ejecutor. Se incluyen los registros necesarios para identificar el modelo y las comprobaciones preliminares realizadas. El análisis cuantitativo del desempeño se reserva para los capítulos experimentales. Se distinguen la implementación vigente y las versiones conservadas en las campañas de desarrollo mediante sus configuraciones y registros.

El código y la documentación complementaria se encuentran en el repositorio *Qwen–CLIPS para instrucciones GPSR* (21). El Apéndice A.1 relaciona los componentes descritos con su ubicación en el código y precisa los recursos externos necesarios.

5.1. Plataforma experimental

5.1.1. Robot Justina

La implementación se integra en la arquitectura ViRBot de Justina, descrita en la Sección 3.1.1. El sistema de planificación representa solicitudes de navegación, percepción, manipulación e interacción mediante el repertorio de acciones del robot. Su salida especifica las acciones que deberían solicitarse a esos componentes. Las comprobaciones descritas se limitan a la generación del plan y no incluyen la ejecución física de estas capacidades.

La arquitectura Qwen–CLIPS se utilizó integrada en Justina durante la RoboCup 2026. Esa experiencia corresponde a la operación del sistema en el robot y se documenta separadamente de la comparación experimental de planes, en la que no se ejecutan físicamente las salidas de ambas arquitecturas.

La comunicación con la plataforma se realiza mediante ROS 2. El planificador produce identificadores de acciones y parámetros simbólicos, como una ubicación de navegación, un objeto o una referencia a una persona. La resolución física de esas instrucciones corresponde al ejecutor y a los módulos encargados de cada tarea.

En el ciclo GPSR se utiliza `instruction_point` para representar el punto de interacción con el operador. Este símbolo se distingue de `inspection point`, que puede aparecer como una entidad del catálogo. La diferencia se conserva al normalizar los parámetros del plan.

5.1.2. Plataforma de cómputo

La plataforma de cómputo utilizada para el trabajo con Qwen2.5-7B cuenta con una GPU NVIDIA GeForce RTX 4070 SUPER, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM. La GPU dispone de 12 GB nominales de memoria; la consulta del dispositivo reporta 12 282 MiB. Estos recursos condicionaron la selección del modelo y la configuración de adaptación, descritas en la Sección 5.3.

Durante la operación, la inferencia puede ejecutarse en el equipo del robot o en una computadora remota. Las modalidades de despliegue y sus interfaces se describen en la Sección 5.3.6.

5.1.3. Entorno de software y ROS 2

Los componentes de generación, validación e interpretación se desarrollaron en Python; la biblioteca de integración y el gestor de instrucciones incluyen componentes C++. La integración del robot utiliza ROS 2 Jazzy. El inventario de software conservado se presenta en la Sección 5.9; se distingue de los entornos adicionales que pueden prepararse para servir los modelos. El entrenamiento se implementó mediante `Trainer`, de Transformers, con PyTorch para el cálculo de tensores, PEFT para los adaptadores LoRA, bitsandbytes para la cuantización y Datasets para la preparación de las particiones. La interacción con CLIPS utiliza `clipspy`, importado en Python como `clips`.

ROS 2 proporciona los servicios y tópicos que conectan los componentes de Qwen–CLIPS en Justina. Los archivos de lanzamiento agrupan los nodos y configuran nombres, parámetros y espacios de nombres. Para la comparación de planes se emplean programas experimentales independientes del sistema de ejecución del robot; su funcionamiento se describe en las secciones correspondientes. El programa GPT utiliza el cliente de OpenAI y la biblioteca `jsonschema`. Las versiones registradas se reúnen en la Sección 5.9.

5.2. Implementación del conjunto de datos

5.2.1. Generador de órdenes

El generador local parte de RoboCupAtHome/CommandGenerator (19), adaptado para ampliar las plantillas y las formulaciones de las órdenes, como base del desarrollo local. La implementación descrita corresponde a esa versión local. No se recuperó el commit externo utilizado originalmente, por lo que no se atribuyen sus modificaciones a una revisión exacta del repositorio de origen.

El código se organiza en los paquetes `qwen_gpsr/generation`, `domain`, `training`, `runtime` y `evaluation`, para generación, conocimiento y contratos, entrenamiento, inferencia y evaluación, respectivamente. La generación se organiza alrededor de `generate_dataset.py`, que coordina la selección de muestras, el balance, la deduplicación y la validación previa a la escritura. El componente `gpsr_commands.py` construye las órdenes utilizando las familias y variantes definidas en `command_constants.py`.

La generación semántica se implementó en `command_goals.py` mediante la clase `CommandGoalsMixin`. El método `_generate_goals` selecciona la función registrada para

una familia y le entrega el contexto de construcción. Cada función devuelve una lista ordenada de objetivos. Las continuaciones de una orden pueden invocar otras funciones y recibir el objeto o la persona establecidos por las subtareas anteriores.

Por ejemplo, la función de transporte entre dos ubicaciones toma el objeto, el origen y el destino del contexto y construye los objetivos de navegación, búsqueda, adquisición y colocación. La función de entrega al operador utiliza el objeto conservado en el contexto y genera `deliver(object, to=me)`. Esta propagación permite mantener el referente de un pronombre en las distintas partes de una orden.

La representación de entrada y etiqueta del ejemplo conductor es:

```
{
  "input": "Navigate to the bed then find a chocolate jello and take it and give it to me",
  "goals": [
    "go(bed)",
    "find(chocolate jello, kind=object)",
    "take(chocolate jello)",
    "deliver(chocolate jello, to=me)"
  ]
}
```

El ejemplo omite los metadatos para destacar la correspondencia entre entrada y objetivos. En el archivo JSONL, cada muestra completa se serializa en una sola línea.

5.2.2. Catálogos y CompetitionTemplate

El módulo `knowledge.py` carga los catálogos mediante `parse_data`. La estructura de entrada separa los nombres de personas, las ubicaciones, las habitaciones y los objetos en archivos de CompetitionTemplate. Las rutas relativas utilizadas son:

```
names/names.md
maps/location_names.md
maps/room_names.md
objects/objects.md
```

El conocimiento cargado se entrega al generador para sustituir los elementos variables de las plantillas. La validación de catálogos se encapsula en `catalog_validation.py`; incluye la detección de colisiones y diferencias con el catálogo de nombres del banco de conocimiento de Justina.

La localización del espacio de trabajo del robot puede configurarse con `JUSTINA_WS`. Esto permite vincular las comprobaciones con el repositorio utilizado en el despliegue sin incorporar rutas personales al contrato de generación.

La auditoría de catálogos detectó nombres de objeto repetidos (`bowl`, `cup` y `plate`), la capitalización de `Water` y diferencias entre los nombres de personas de Competition-Template y Justina. Estas advertencias no bloquearon la validación del catálogo, pero deben considerarse en la normalización y clasificación de entidades. La comparación entre fuentes no exige que sus listas sean idénticas.

5.2.3. Validadores

La validación de etiquetas se implementó en `goal_schema.py`. La función `parse_goal` separa el nombre del objetivo, sus argumentos posicionales y los campos nombrados, y almacena el resultado en una estructura `ParsedGoal`. La función `validate_goals` comprueba sintaxis, argumentos obligatorios, campos admitidos, compatibilidad de tipos y calificadores, y dependencias programadas entre objetivos. Las incidencias se representan mediante `ValidationIssue`, con un código, un mensaje y, cuando corresponde, la posición del objetivo.

La auditoría del archivo final utiliza `evaluate_dataset.py` y las funciones de `dataset_evaluation.py`. El procedimiento recorre las filas, valida las etiquetas y acumula estadísticas por familia, firma de objetivos, tipos de entidades y campos. La revisión manual realizada durante el desarrollo abarcó partes del conjunto; la comprobación sistemática posterior se ejecutó mediante estos validadores programados.

La comprobación simbólica se implementó en `ClipsPlanValidator`. El componente carga las reglas, reinicia el entorno para cada secuencia, convierte sus objetivos a hechos e inicia la expansión. Para aceptar una secuencia exige que no queden hechos `gpsr-goal`, que se produzca `GPSR_DONE` y que no aparezca el mensaje de objetivo no soportado. La conversión utilizada en esta auditoría se distingue de las heurísticas del adaptador ROS 2 descritas en la Sección 5.4.1.

La auditoría del 4 de noviembre de 2026 recorrió las 40 000 filas del archivo final. Esta comprobación posterior no reconstruye el registro original de validación previo a los entrenamientos. Todas las etiquetas examinadas fueron aceptadas por las comprobaciones del esquema y no se registraron incidencias de validación. Para CLIPS se estableció un límite de 200 secuencias: se examinaron las primeras 200 etiquetas válidas en el orden del archivo y todas superaron los criterios de expansión. Estas secuencias

pertenecen a 25 de las 29 familias presentes en el conjunto. La selección corresponde a un prefijo del archivo y no a una muestra aleatoria ni a una cobertura completa de familias.

Los resultados acreditan el cumplimiento de las restricciones programadas y la expansión de las secuencias examinadas. No constituyen una revisión independiente de la correspondencia semántica de cada orden con su etiqueta ni una comprobación de ejecución física.

Las pruebas de `test_dataset_quality.py` complementan esta auditoría mediante un lote generado de 500 muestras. Comprueban cardinalidad, deduplicación, distribución entre categorías, metadatos y firmas esperadas. La prueba simbólica selecciona el primer ejemplo de cada familia presente en ese lote. Este procedimiento se distingue de las 200 secuencias del archivo final comprobadas en la auditoría.

5.2.4. Corpus de entrenamiento

El corpus de entrenamiento se conserva en `dataset_gpsr.jsonl`, con 40 000 registros y un objeto JSON por línea. La auditoría identifica 29 familias de órdenes, 18 firmas distintas de objetivos y 87 identificadores de plantilla superficial. Cada registro contiene `input`, `goals` y `meta`; el vocabulario de objetivos se define en la Tabla 3.1.

Los metadatos identifican la familia, la categoría, la variante superficial, la firma de objetivos, los tipos de entidades y los campos presentes. Permiten caracterizar el conjunto y analizar su cobertura, pero se excluyen de la entrada y de la respuesta supervisadas. Tampoco constituyen un registro individual de los resultados de CLIPS.

El entrenamiento utiliza la instrucción como entrada y un objeto JSON con la propiedad `goals` como respuesta esperada. El cargador conserva temporalmente la familia y la categoría como columnas auxiliares y las elimina al construir las entradas tokenizadas.

Ambos ajustes utilizaron el mismo archivo, sin cambios entre corridas. La huella del corpus actual coincide con la conservada en la auditoría de procedencia. Los scripts y los registros documentan una partición que utiliza `train_test_split`, una proporción de validación de 0.1 y semilla 42, con 36 000 muestras de entrenamiento y 4 000 de validación. La división se realiza aleatoriamente por filas, sin agrupar ni estratificar por familia o plantilla. Por ello, la pérdida de validación caracteriza ejemplos de la distribución del generador y no acredita por sí sola generalización a familias excluidas del ajuste.

La identidad del archivo auditado se conserva mediante su huella SHA-256, incluida en la Sección 5.9.

5.3. Entrenamiento e inferencia Qwen

5.3.1. Preparación del modelo

Los módulos `qwen_gpsr.training.train` y `qwen_gpsr.training.train_3b` implementan los ajustes de `Qwen/Qwen2.5-7B` y `Qwen/Qwen2.5-3B`, respectivamente. Se parte de las variantes base, sin ajuste Instruct previo. Ambas siguen el mismo formato supervisado y producen objetivos semánticos, no planes. Cada fila se valida y se transforma en una conversación mediante `apply_chat_template`. El código proporciona un mensaje de usuario con la orden y un mensaje de asistente con los objetivos serializados como JSON.

La plantilla conservada en `tokenizer_config.json` incorpora automáticamente un mensaje de sistema cuando la conversación no contiene uno explícito. Su contenido es *You are a helpful assistant.* Por tanto, aunque el código construye dos mensajes, el texto tokenizado incluye también este prefijo predeterminado. El tokenizador utiliza `<|endoftext|>` como token de fin de secuencia y de relleno.

El cálculo de la pérdida se concentra en la respuesta del asistente. Las posiciones del prefijo reciben la etiqueta de exclusión `-100`, al igual que el relleno del lote. El componente `CompletionOnlyCollator` realiza relleno dinámico a la derecha hasta un múltiplo de ocho posiciones y ajusta máscaras de atención y etiquetas.

La conversación completa se tokeniza con truncamiento y una longitud máxima de 512 tokens, que comprende el prefijo, la orden y la respuesta. El prefijo se tokeniza por separado para determinar las posiciones excluidas de la pérdida. El límite configurado para el entrenamiento es independiente de la longitud máxima declarada por el tokenizador; una secuencia extensa puede perder objetivos situados al final de la respuesta.

5.3.2. Configuración QLoRA

El modelo base se carga con cuantización en formato NF4 (*NormalFloat* de cuatro bits), que reduce la precisión de representación de sus pesos. Se utiliza además doble cuantización para reducir la memoria de los parámetros de cuantización. Sus pesos

permanecen congelados y el ajuste se realiza sobre adaptadores LoRA insertados en las proyecciones de atención y del bloque de transformación del modelo.

Los módulos seleccionados son `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj` y `down_proj`. Se utiliza rango 32, factor de escala 64 y una probabilidad de descarte de 0.05. Esta configuración delimita la capacidad entrenable sin actualizar todos los parámetros del modelo base.

El cálculo utiliza BF16 (*bfloat16*) cuando la GPU declara soporte para ese formato y FP16 en caso contrario. Ambos son formatos de punto flotante de 16 bits para las operaciones numéricas; se distinguen de la representación de los pesos base en cuatro bits. La carga selecciona la implementación de atención `eager`, desactiva la ventana deslizante y distribuye el modelo con `device_map='auto'`. Durante el ajuste se desactiva la caché y se habilita *gradient checkpointing* con `use_reentrant=False`, reduciendo las activaciones conservadas a cambio de recalcularlas durante la retropropagación. El script requiere CUDA y limita a 0.9 la fracción de memoria disponible para el asignador de PyTorch en el dispositivo configurado.

Al guardar el modelo se conserva el adaptador y su configuración, junto con el tokenizador. La inferencia requiere cargar estos artefactos sobre la misma versión del modelo base.

5.3.3. Hiperparámetros de entrenamiento

La Tabla 5.1 reúne la configuración común y las diferencias de lote. Los argumentos conservados y las configuraciones LoRA complementan los valores de los scripts. En una GPU, ambas variantes utilizan un lote efectivo de ocho muestras.

El optimizador es `adamw_torch`, con $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, decaimiento de pesos nulo y norma máxima del gradiente de 1.0. La partición utiliza también semilla 42, en una operación distinta del control de aleatoriedad del entrenamiento. El lote efectivo y las dos épocas explican las 9 000 actualizaciones registradas en ambas corridas.

5.3.4. Selección y conservación del checkpoint

La selección utiliza la menor pérdida sobre las 4 000 muestras de validación. En ambas corridas se configuraron `load_best_model_at_end=True`, `metric_for_best_model=."eval_loss"` y `greater_is_better=False`. Al concluir el entrenamiento, `Trainer` restaura ese checkpoint y el script guarda el adaptador y el tokenizador en el directorio de salida.

Tabla 5.1: Configuración de los dos entrenamientos QLoRA.

Parámetro	**Qwen2.5-3B**	**Qwen2.5-7B**
Entrenamiento / validación	36 000 / 4 000	36 000 / 4 000
Longitud máxima de conversación	512 tokens	512 tokens
Lote de entrenamiento por GPU	4	1
Acumulación de gradientes	2	8
Lote de evaluación por GPU	8	8
Épocas / actualizaciones	2 / 9 000	2 / 9 000
Tasa de aprendizaje inicial	10^{-4}	10^{-4}
Planificador / calentamiento	Coseno / 150	Coseno / 150
Evaluación y guardado	Cada 250 pasos	Cada 250 pasos
Registro de pérdida	Cada 50 pasos	Cada 50 pasos
Checkpoints conservados	2	2
Precisión mixta configurada	BF16	BF16
Semilla	42	42
Rango / escala / descarte LoRA	32 / 64 / 0.05	32 / 64 / 0.05

Tabla 5.2: Artefactos seleccionados y registros de los entrenamientos.

Registro	**Qwen2.5-3B**	**Qwen2.5-7B**
Checkpoint seleccionado	7 000	5 000
Mejor pérdida de validación	0.2271966	0.2321400
Último checkpoint	9 000	9 000
Pérdida de validación final	0.2272140	0.2321626
Tiempo de entrenamiento	10 042.3 s	27 772.8 s

Los adaptadores de inferencia se conservan en `models/nl2cd_qwen3b` y `models/nl2cd_qwen7b`. La igualdad SHA-256 entre los pesos de cada directorio raíz y los de su checkpoint seleccionado confirma qué adaptador quedó exportado. El checkpoint del paso 9 000 conserva el estado final y no sustituye esa selección. La pérdida y el tiempo de entrenamiento describen el ajuste; no determinan por sí solos qué configuración produce mejores planes.

La Figura 5.1 compara ambas corridas: la vista completa muestra la estabilización inicial y el detalle de validación permite distinguir los checkpoints seleccionados. La tabla anterior conserva sus valores exactos.

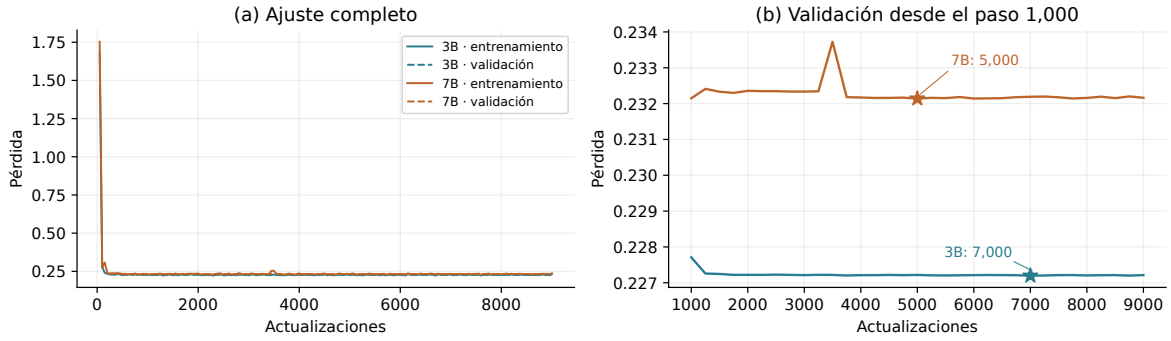


Figura 5.1: Entrenamiento de Qwen2.5-3B y Qwen2.5-7B mediante QLoRA. (a) Pérdidas durante las 9 000 actualizaciones. (b) Detalle de validación desde el paso 1 000; las estrellas señalan los checkpoints seleccionados: 7 000 para 3B y 5 000 para 7B. Las curvas describen el ajuste y no equivalen al cumplimiento de tareas de planificación. Elaboración propia.

Después del ajuste, cada script realiza una comprobación generativa sobre las primeras 50 muestras de la partición de validación, sin el normalizador HTTP y con hasta 256 tokens nuevos. Los registros de ambas corridas indican 50 coincidencias exactas de objetivos, 50 secuencias válidas según el esquema y 50 aceptadas por el validador CLIPS. La comprobación reutiliza parte de la partición de selección; su alcance es preliminar y no sustituye una prueba independiente ni una evaluación del recorrido HTTP–ROS 2. El benchmark fijo se ejecuta y registra por separado para el adaptador que se desea evaluar.

5.3.5. Inferencia

El módulo `qwen_gpsr.runtime.inference` obtiene el identificador del modelo base desde `adapter_config.json`, lo carga en NF4 con doble cuantización e incorpora los

pesos del adaptador mediante `PeftModel.from_pretrained`. Recupera el tokenizador del directorio del adaptador y coloca el modelo en modo de evaluación. Así, seleccionar el adaptador de 3B carga su propia base, en lugar de aplicar sus pesos al modelo de 7B. El dispositivo predeterminado es `cuda:0`; las opciones de ejecución permiten modificar la distribución y los límites de memoria.

Antes de tokenizar la orden se aplica `CommandNormalizer`. El procedimiento convierte el texto a minúsculas, uniforma espacios, guiones y signos, elimina las expresiones de cortesía y muletillas definidas en el código y reduce repeticiones adyacentes. Después sustituye variantes de expresiones, como *pick up* por *take* y *search for* por *find*, y aplica alias de entidades, como *fridge* por *refrigerator*.

El normalizador construye su repertorio con los catálogos de `CompetitionTemplate` y las listas de atributos e información del generador. Tras las sustituciones exactas, busca coincidencias aproximadas mediante `SequenceMatcher`. Compara candidatos con el mismo número de palabras y utiliza un umbral de 0.86 para una palabra y 0.82 para expresiones de varias palabras. Descarta de esta búsqueda los fragmentos de menos de cuatro caracteres y examina primero las secuencias más largas. Las sustituciones se basan en similitud textual; no constituyen una comprobación semántica de la orden.

El modo de diagnóstico del normalizador conserva la entrada original, la normalizada y la lista de transformaciones. En el recorrido de inferencia se utiliza el texto normalizado, que puede devolverse junto con la predicción mediante `return_normalized=True`. Estas transformaciones pertenecen al preprocesamiento y se distinguen de las capacidades aprendidas por Qwen.

La orden procesada se introduce como mensaje de usuario y la plantilla añade el prefijo de sistema predeterminado y el inicio de la respuesta del asistente. La generación utiliza `do_sample=False`, una penalización de repetición de 1.05 y un máximo de 128 tokens nuevos. Las condiciones de parada incluyen el token de fin de secuencia y `<|im_end|>`, cuando está disponible. Las comprobaciones generativas del script de entrenamiento utilizan 256 tokens nuevos; se trata de una configuración distinta de la inferencia del servicio.

Se decodifica únicamente la continuación generada. La extracción localiza el primer fragmento con llaves balanceadas e intenta convertirlo mediante `json.loads`; posteriormente se valida el contrato de objetivos. La generación no impone una gramática JSON. Si la extracción o validación falla, se devuelve un error con hasta 200 caracteres de la respuesta para diagnóstico. La aceptación comprueba estructura y restricciones

programadas, sin acreditar por sí sola fidelidad a la instrucción.

La salida aceptada corresponde a los objetivos extraídos antes de la adaptación simbólica. El servicio no expone el texto completo generado por Qwen en las respuestas válidas; conserva el objeto JSON extraído. El cuerpo HTTP y la generación neuronal completa son productos distintos. Las reparaciones posteriores se analizan por separado de la interpretación.

5.3.6. Despliegue del modelo

El servidor mantiene residentes el modelo, el tokenizador y el normalizador. Puede alojarse en el equipo del robot o en infraestructura remota; el cliente envía órdenes por HTTP y la expansión simbólica permanece en su propio proceso. La ruta del adaptador selecciona la variante de 3B o 7B. El procedimiento de inicio y autenticación se recoge en el Apéndice A.

Tabla 5.3: Interfaz HTTP del servidor Qwen.

Ruta	**Función**
`GET /health`	Comprobar que el servicio responde.
`POST /translate`	Interpretar la orden del campo `command`.
`GET /metadata`	Identificar modelos, archivos, parámetros y entorno efectivos.
`GET /status`	Consultar actividad y terminación de solicitudes identificadas.

Una petición como {`çommand": "go to the kitchen"`} recibe `ok`, `elapsed_ms`, la huella `effective_config_sha256` y un objeto `result`. Este contiene `normalized_input`, `prediction` y mediciones de recursos. El cliente ROS retira la envoltura y devuelve un JSON con `input` y `goals`. La predicción inválida, la petición incorrecta y el fallo interno se comunican como errores.

El servidor acepta conexiones en hilos y serializa las generaciones mediante exclusión mutua. La campaña puede enviar `X-Experiment-Run-ID` y consultar la terminación mediante `/status`; la API ROS no incorpora este seguimiento. Un timeout del cliente no cancela la inferencia remota. El tiempo `elapsed_ms` comprende la traducción observada por el servidor, incluida la espera de su bloqueo; las mediciones internas de recursos excluyen esa espera y la carga del modelo. Ninguna de ellas incluye la adaptación y planificación posteriores del cliente.

5.4. Implementación del planificador CLIPS

5.4.1. Conversión de objetivos a hechos gpsr-goal

El adaptador `goals_to_clips_node.py` recibe la lista de objetivos y construye hechos `gpsr-goal`. El componente `GoalParser` acepta un objeto con la propiedad `goals` o directamente una lista JSON. Analiza cada expresión y separa los argumentos posicionales de los pares `clave=valor`, respetando las comas delimitadoras y los valores entre comillas. Cada hecho conserva la posición, el nombre del objetivo, el objetivo principal y los campos utilizados para su expansión. La estructura compartida es:

```
(deftemplate gpsr-goal
  (slot step (type INTEGER))
  (slot name)
  (slot target (default ""))
  (slot kind (default unknown))
  (slot destination (default ""))
  (slot relation (default ""))
  (slot value (default ""))
  (slot qualifier (default "")))
```

Para el segundo objetivo del ejemplo conductor, la representación es:

```
(gpsr-goal
  (step 2)
  (name find)
  (target "chocolate_jello")
  (kind object)
  (destination "")
  (relation "")
  (value "")
  (qualifier ""))
```

Interpretación del campo `destination`. El campo `destination` representa el destino o destinatario asociado a un objetivo. Su significado depende de la operación indicada en `name`: en una entrega identifica al receptor; en una colocación, el lugar o soporte

donde se deposita el objeto; y en una tarea de guía, la ubicación de destino. Por tanto, su presencia no implica por sí sola una acción de navegación. Las reglas determinan las acciones necesarias a partir del objetivo, sus argumentos y el estado simbólico.

Este campo se distingue de `target`, que conserva el argumento principal del objetivo. Por ejemplo, en `deliver(fork, to=me)`, `target` identifica el objeto y `destination` al destinatario. En cambio, para `go(kitchen)`, la ubicación se almacena directamente en `target` y `destination` permanece vacío.

Los espacios de los argumentos se convierten en guiones bajos para construir los identificadores de ejecución. Los valores de cadena se escapan antes de insertarse en CLIPS, mientras que el nombre y el tipo se representan como símbolos.

La clasificación de entidades utiliza la secuencia completa. Los destinatarios de entrega, los participantes de guía, seguimiento y saludo, los atributos visuales y la capitalización de los nombres aportan indicios para interpretar una búsqueda como búsqueda de persona. El campo `kind` del hecho se calcula mediante estas reglas de contexto; no se copia directamente del argumento homónimo de la predicción. Por ejemplo, `go` y `place` se clasifican como ubicación, `take` y `deliver` como objeto, y las operaciones de información como `info`. Esta clasificación cumple una función operativa en el adaptador y no redefine el vocabulario semántico.

`ClipsGoalFactBuilder` conserva los destinos indicados por `to`, `at`, `on` o `in`, en ese orden de prioridad. También admite el segundo argumento posicional para entrega, guía, seguimiento y colocación. La relación principal selecciona el primer campo disponible entre destino, gesto, postura, vestimenta y propiedad. El campo `qualifier` conserva el primer atributo visual, con prioridad de gesto, postura, vestimenta y propiedad, en la forma `clave:valor`. La representación no codifica una conjunción de varios calificadores visuales.

Además de la normalización léxica, el adaptador aplica una reparación específica cuando recibe una navegación cuyo argumento corresponde a una persona. Ese objetivo se transforma en búsqueda. Si inmediatamente después aparece un seguimiento de la misma persona con destino, se adelanta el destino como navegación, se añade la búsqueda y se conserva el seguimiento sin destino. Esta operación modifica la interpretación de la secuencia y debe registrarse como posprocesamiento, sin atribuir la corrección al modelo.

Para mantener la notación del capítulo anterior, la salida del intérprete conserva el símbolo $G_{\text{pred}}^{\text{Qwen}}$. Se denomina Adaptar a la transformación del adaptador, que comprende

el análisis, las reparaciones, la clasificación y la serialización. El recorrido implementado queda expresado como:

$$P^{\text{Qwen–CLIPS}} = \text{CLIPS}\Big(\text{Adaptar}\Big(G_{\text{pred}}^{\text{Qwen}}\Big)\Big). \tag{5.1}$$

La calidad de la interpretación se calcula sobre la predicción anterior a esta transformación. La evaluación del sistema completo considera también los hechos y el plan que resultan de ella.

La auditoría fuera de línea utiliza `goal_to_clips_fact` para construir el mismo tipo de hecho, pero emplea el análisis del esquema semántico y no reproduce todas las heurísticas y reparaciones del nodo ROS 2. En consecuencia, compartir la plantilla `gpsr-goal` no garantiza equivalencia entre ambos recorridos. La comprobación integrada debe conservar la transformación que efectivamente recibió CLIPS.

5.4.2. Representación del estado simbólico

La planificación comienza con un estado que representa ubicación desconocida y ausencia de un objeto sostenido:

```
(current-state (location "unknown") (holding "nothing"))
```

Este hecho inicializa la representación utilizada para construir el plan; no es una lectura del estado físico de Justina. Las herramientas de navegación y manipulación manejan la información de ubicación y posesión que necesitan al resolver las operaciones solicitadas. CLIPS mantiene, por su parte, un estado previsto durante la expansión, sin sincronizarlo automáticamente con esas observaciones del robot.

Las reglas actualizan este estado al añadir acciones al plan. Los valores representan efectos previstos, no confirmaciones físicas. Por ejemplo, la incorporación de una adquisición modifica el objeto esperado en posesión del robot. La colocación exige que coincida con el objeto solicitado; las reglas de entrega no imponen esa misma igualdad como precondición general. La validación de la secuencia de objetivos y las condiciones de expansión son, por tanto, controles complementarios.

Otros hechos conservan referencias a personas, el último contexto de interacción y análisis visuales. Las acciones de diálogo añaden una anotación interna de interlocutor, utilizada al organizar el foco al final de la expansión. Las referencias `guest_N` y

customer_N permiten reutilizar una entidad durante distintas subtareas. Los objetos se identifican mediante nombres normalizados e índices, como chocolate_jello_1.

La referencia guest_last se utiliza para una persona capturada por find_person, mientras que customer_N identifica una búsqueda por atributo realizada con find_pose. Una persona nombrada puede requerir búsqueda anónima, solicitud del nombre y escucha mediante guest_name_last. La asociación efectiva de estos identificadores con detecciones y respuestas corresponde al ejecutor. Los alias de última persona no garantizan identidades distintas para varios destinatarios nombrados. Asimismo, el estado holding representa un solo objeto; no equivale al conjunto de adquisiciones que puede mantener el validador de objetivos.

5.4.3. Reglas de expansión

Las reglas se agrupan en goals_planning.clp. Una regla de procesamiento consume un objetivo únicamente cuando no existe otro pendiente con un paso menor. Después de expandirlo, retira el hecho correspondiente y añade las acciones de la subtarea.

La navegación a una ubicación distinta de la registrada produce una solicitud go_to y una confirmación hablada. Si ambas ubicaciones coinciden, el objetivo se consume sin repetir el desplazamiento. La regla siguiente muestra esta decisión y la actualización del estado:

```
(defrule process-go
  (declare (salience 200))
  (planning-ready)
  ?g <- (gpsr-goal (step ?step) (name go) (target ?dest))
  (not (gpsr-goal (step ?prev&:(< ?prev ?step))))
  ?st <- (current-state (location ?loc) (holding ?held))
  =>
  (retract ?g)
  (if (neq ?loc ?dest) then
    (append-say (str-cat "I will navigate to " ?dest))
    (append-action go_to ?dest)
    (append-action say (str-cat "I have arrived at " ?dest))
    (retract ?st)
    (assert (current-state (location ?dest) (holding ?held)))))
```

La búsqueda de objetos genera find_object o una operación de análisis cuando se utiliza una propiedad admitida. Las búsquedas de personas recurren a find_person o find_pose y conservan el contexto necesario para la interacción posterior.

La manipulación utiliza take; la entrega incorpora aproximación y give; la colocación puede incluir navegación al destino, find_space y drop. La selección concreta

depende de los argumentos y del estado simbólico. Las acciones de diálogo y de control del foco complementan estas operaciones.

Para la colocación se distinguen puntos de navegación y superficies genéricas. Un destino como `storage_rack` puede añadir navegación, mientras que `table`, `shelf` o `cabinet` se tratan como soportes en el lugar actual. La función siguiente incorpora la búsqueda de espacio y la liberación del objeto; el suelo constituye una excepción que utiliza directamente el argumento `floor`:

```
(deffunction append-drop-sequence (?obj ?destination)
  (bind ?surface (drop-surface ?destination))
  (if (eq ?surface "floor") then
    (append-action drop "floor")
    else
    (append-action find_space ?surface)
    (append-action drop (object-id ?obj))))
```

En este fragmento, `drop` identifica la acción del ejecutor que materializa el objetivo semántico `place`. La búsqueda de espacio selecciona una clase de soporte, como mesa, estante o contenedor, y la liberación utiliza el identificador del objeto.

Tras la colocación, la regla `process-place` asigna a `location` el valor de `destination`, incluso cuando este representa un soporte genérico y no se generó navegación. Por ello, el estado resultante puede mezclar referencias a ubicaciones y a soportes; esta limitación debe considerarse al interpretar las acciones posteriores, sin asumir que hubo un desplazamiento físico.

La guía genera una instrucción hablada, navegación al destino y cierre de la interacción. El seguimiento recupera o busca a la persona, incorpora aproximación cuando corresponde y emite mediante `say` las claves `instruct_follow_only` o `instruct_follow_to_<destino>`. Esta regla no emite una acción explícita de seguimiento ni actualiza la ubicación al destino. Su expansión representa el protocolo de interacción; la realización física del seguimiento depende del tratamiento de esas claves en el ejecutor y debe verificarse por separado.

Las consultas por tamaño utilizan análisis de objetos grandes o pequeños. Las aproximaciones a otras propiedades y sus consecuencias se resumen en la Tabla 5.4.

El conteo genera `analyze_objects`, incorpora retorno al punto de instrucción cuando corresponde y emite la respuesta `count_category`. Para guardar un nombre se solicita y escucha `guest_name_last`; para atributos de persona se utilizan operaciones de

análisis o búsqueda por postura. La información adquirida se comunica mediante las claves de respuesta de `tell`. La respuesta a preguntas combina `listen` y `say` con la clave `answer_question`, y el saludo utiliza la persona referenciada y la clave `greet`.

Tabla 5.4: Límites de cobertura y consecuencias para evaluar el cumplimiento.

Requisito	**Soporte implementado**	**Consecuencia**
Varios atributos visuales	El adaptador conserva un calificador.	Comprobar la pérdida de restricciones conjuntas.
Peso y grosor	Algunas solicitudes se aproximan por tamaño.	La consulta no acredita la propiedad solicitada.
Conteo por vestimenta	La expansión puede emitir un conteo general.	Sin filtro, el plan no satisface la restricción.
Varios destinatarios	Las referencias de última persona tienen alcance limitado.	Comprobar la identidad de cada destinatario.
Posesión y ubicación	Un objeto sostenido; algunos soportes se registran como ubicación.	Revisar precondiciones y contexto de acciones posteriores.
Seguimiento	Se emiten claves habladas del protocolo.	El soporte depende de su interpretación por el ejecutor.

5.4.4. Generación de acciones

Las reglas construyen hechos intermedios `action-step`. Tras organizar las interacciones, los pasos finales se convierten en `ros-message`, como se detalla en la Sección 5.4.5. Los anuncios generales utilizan numeración desde 1 y las acciones operativas desde 1001. Al ordenar la salida, se presenta primero el anuncio de las tareas y después las operaciones del plan.

La expansión completa de una orden de navegación, búsqueda, adquisición y entrega se presenta en la Sección 5.6, junto con las representaciones intermedias y el plan resultante.

5.4.5. Finalización y organización del diálogo

La construcción se cierra en cuatro etapas. Cuando no quedan objetivos, `add-final-return` añade el retorno a `instruction_point` si corresponde y una única operación final `say task completed`. Después, `organize-interaction-focus` invoca `normalize-interactions`, que ordena los pasos y organiza el foco de toda la secuencia.

Los `say` y `listen` consecutivos dirigidos al mismo interlocutor comparten un ciclo de `focus enable/disable`, incluso si proceden de objetivos distintos. El cambio de interlocutor, una acción no dialogada o un anuncio general cierra ese ciclo. Los anuncios de navegación, manipulación y finalización no activan foco. Las anotaciones de interlocutor son internas y no se publican como argumentos adicionales del ejecutor. La pasada final concentra esta decisión; no se utiliza el mecanismo anterior de `focus-active`.

Los pasos organizados se guardan como `finalized-step`, con anuncios desde 1 y operaciones renumeradas desde 1001. `serialize-action-step` los convierte a `ros-message`; `goals-finish` emite `GPSR_DONE` cuando tampoco quedan pasos por serializar. Las cuatro reglas tienen prioridades 20, 7, 5 y 1.

`PlanExtractor` ordena los mensajes y coloca las notas de inicio y cierre. El nodo publica el texto si encuentra mensajes y una salida no vacía; no exige por sí mismo ausencia de objetivos pendientes ni comprueba todos los criterios del validador experimental. Una regla de objetivo desconocido puede consumirlo, añadir una explicación y terminar con `task completed`. Por ello, las marcas de cierre y el consumo de metas no acreditan cumplimiento de la orden.

5.5. Integración Qwen–CLIPS con Justina

5.5.1. Componentes ROS 2

La Figura 5.2 sitúa los procesos y sus conexiones. El cliente coordinador solicita la interpretación al nodo semántico y entrega los objetivos devueltos al conversor. El servidor Qwen puede residir en otro equipo; el adaptador y CLIPS funcionan en el entorno ROS de Justina. La publicación del plan permite observarlo y entregar sus instrucciones al gestor.

El nodo CLIPS delega el motor, la carga de reglas, la extracción y el registro en `ClipsEngine`, `RuleLoader`, `PlanExtractor` y `PlanLogger`, respectivamente. Los callbacks encolan las solicitudes y responden antes de completarlas; un trabajador único procesa secuencialmente las operaciones simbólicas.

El nodo declara el parámetro `plan_name`, cuyo valor predeterminado es `default`, y lo entrega al cargador de reglas. El archivo `planning.launch.xml`, del paquete `planning_tools`, agrupa los lanzamientos de CLIPS y del parser semántico bajo el espacio de nombres `/planning`. El lanzamiento superior selecciona `plan_name=goals` por defecto

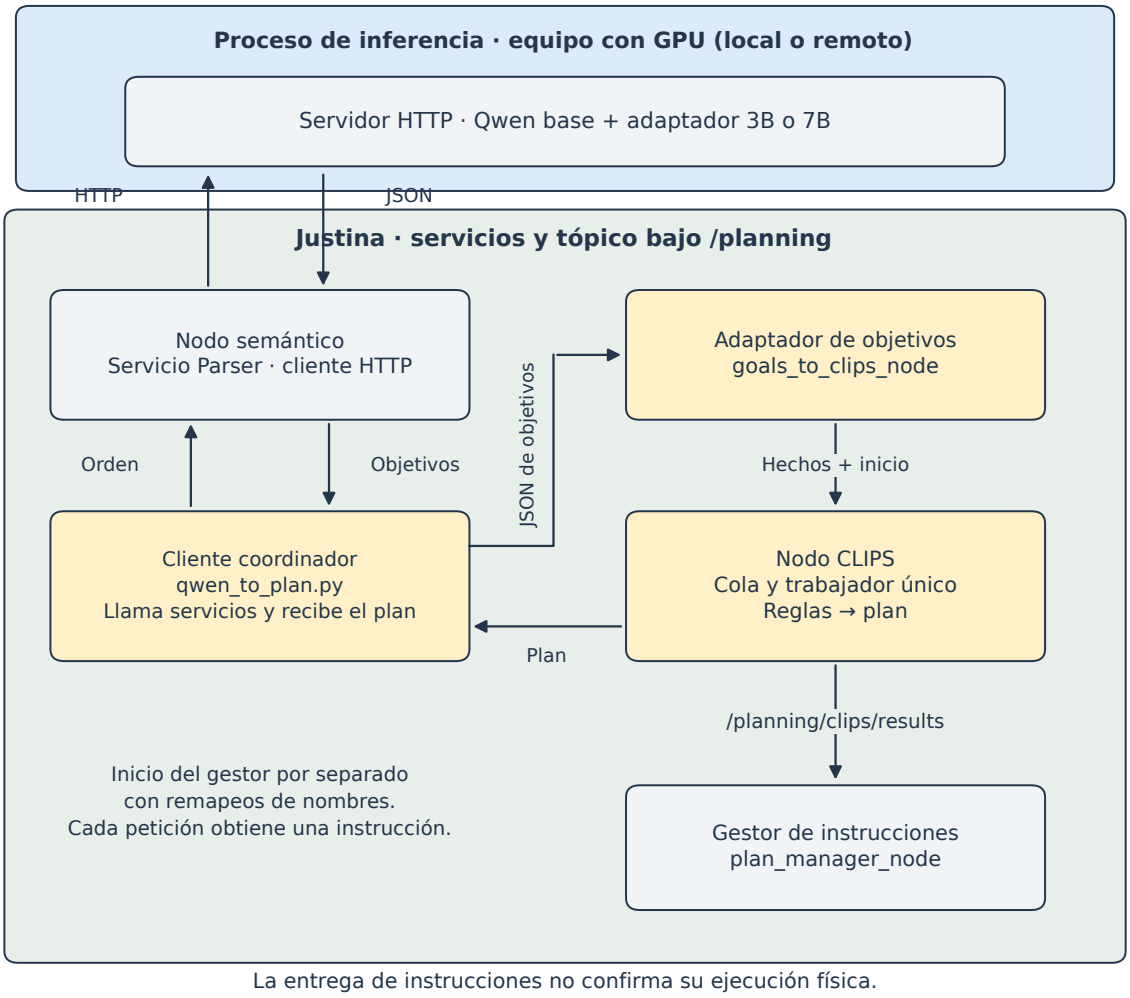


Figura 5.2: Despliegue e interfaces del recorrido Qwen–CLIPS en Justina. El cliente coordina interpretación y conversión; CLIPS publica el plan para sus consumidores. El gestor se inicia por separado y requiere los remapeos descritos en el texto. Elaboración propia.

y propaga ese argumento al lanzamiento de CLIPS.

El lanzamiento mantiene `qwen_response_format=goals`, `use_goals_to_clips=true` y `use_entity_extractor=true`. También propaga al parser un tiempo de espera Qwen de 5.0 segundos. Este valor sustituye los 15 segundos predeterminados del lanzamiento semántico incluido.

Los argumentos `qwen_url` y `qwen_api_key` llegan vacíos en la invocación utilizada. El nodo resuelve cada valor consultando primero el parámetro ROS; si está vacío, utiliza `QWEN_SERVER_URL` o `QWEN_API_KEY`, respectivamente. Si la variable de URL no existe, utiliza `http://127.0.0.1:8008`; si no existe la variable de clave, emplea una cadena vacía. En el despliegue se utilizan las variables de entorno configuradas en la sesión de terminal del usuario.

El nodo semántico proporciona el servicio mediante el tipo `Parser` y delega la solicitud HTTP en `QwenHTTPClient`. Utiliza un grupo de callbacks reentrante y un ejecutor de dos hilos. También ofrece un servicio de recepción de órdenes desde el componente de escucha; las pruebas por consola utilizan directamente la entrada textual.

El adaptador mantiene sus clientes de servicios en un nodo auxiliar. El ejecutor `SingleThreadedExecutor` funciona en un hilo dedicado y procesa las respuestas mientras el callback de conversión espera la aceptación de sus solicitudes dependientes.

5.5.2. Comunicación y entrega del plan

El coordinador recibe los objetivos del servicio semántico, solicita su conversión e inicia CLIPS. Los servicios confirman la aceptación de las solicitudes; el plan se obtiene mediante la publicación posterior. El nodo simbólico procesa las tareas secuencialmente y reinicia el entorno al insertar cada lote de objetivos.

El cliente se suscribe antes de enviar la orden y utiliza una solicitud a la vez, pues el tópico no vincula el plan con un identificador de petición. El gestor entrega instrucciones sucesivas sin confirmar su ejecución física. El Apéndice A documenta interfaces, tiempos de espera, comandos y remapeos.

5.5.3. Extracción de entidades durante la interacción

`entity_extractor_node` interpreta respuestas habladas cuando una acción `listen` necesita un nombre, una confirmación u otro dato. No es una etapa obligatoria entre Qwen y CLIPS y no consulta un modelo de lenguaje. El ejecutor llama a `Planning::`

`extractEntity` y al servicio `/planning/semantic_parser/extract_entity`.

La interfaz `ExtractEntity` recibe `sentence` y `category`; devuelve `value` y `values`. Los catálogos YAML del paquete `knowledge_bank` se cargan al iniciar el nodo. La búsqueda ordinaria utiliza expresiones regulares, palabras completas y prioridad de patrones largos, sin inferir entidades ausentes. Las ubicaciones y las confirmaciones tienen tratamientos específicos; una confirmación contradictoria no se toma automáticamente como afirmativa. El resultado puede utilizarse para actualizar atributos de una persona en el modo completo del ejecutor, sin convertir esa capacidad de interfaz en evidencia de ejecución física en la comparación.

5.5.4. Alcance de las comprobaciones de integración

Se conservaron 48 órdenes de pruebas funcionales y, por separado, un piloto de 60 órdenes para desarrollar la evaluación. No se atribuyen resultados históricos a un adaptador, prompt o conjunto de reglas posterior. Las pruebas locales de extracción y errores simulados, y los ejemplos expandidos con CLIPS, comprueban componentes concretos. La revisión documental de noviembre de 2026 no incluyó una ejecución completa de ROS con servidor Qwen y robot.

Para contrastar el recorrido experimental con Justina se ejecutó `experiments.verify_justina`. La comprobación comparó la estructura de las clases del adaptador, la huella de las reglas, los hechos adaptados y el texto del plan extraído mediante las clases de Justina. Se verificaron 57 secuencias, sin discrepancias. Esta evidencia respalda la equivalencia comprobada del procesamiento simbólico; no generaliza el resultado a cualquier entrada ni certifica operación física.

5.6. Ejemplo de funcionamiento Qwen–CLIPS

Esta sección reúne los componentes anteriores mediante una orden de búsqueda, adquisición y entrega. Las figuras muestran las transformaciones de la instrucción y la expansión por reglas, mientras que la lista final permite consultar las acciones y los mensajes exactos del plan. Se considera una predicción que interpreta correctamente la orden, con la manipulación habilitada. El ejemplo es ilustrativo y no constituye una medición de precisión de Qwen ni una prueba de ejecución física.

5.6.1. De la instrucción a los hechos simbólicos

La Figura 5.3 parte de una instrucción que solicita desplazarse a `bed`, buscar un objeto, adquirirlo y entregarlo al operador. La recepción puede utilizar la inferencia local o el servicio remoto descritos anteriormente. El recorrido contempla normalización, tokenización, generación, extracción del JSON y validación de los objetivos. La figura presenta la predicción considerada para el ejemplo, $G_{\text{pred}}^{\text{Qwen}}$.

De la instrucción a los hechos

Instrucción recibida
Navigate to the bed then find a chocolate jello
and take it and give it to me.

Objetivos: predicción ilustrativa de Qwen
go(bed)
find(chocolate jello, kind=object)
take(chocolate jello)
deliver(chocolate jello, to=me)

Adaptador de objetivos
Conservar el orden, normalizar nombres y asignar tipos

step	name	target	kind	destination
1	go	bed	location	""
2	find	chocolate_jello	object	""
3	take	chocolate_jello	object	""
4	deliver	chocolate_jello	object	me

En deliver: relation="to" y value="me". Los demás campos no mostrados están vacíos.

Figura 5.3: Transformaciones de la instrucción del ejemplo: orden, objetivos predichos y campos de los hechos `gpsr-goal`. La predicción de Qwen es ilustrativa. Cada fila de la tabla representa un hecho construido por el adaptador. Elaboración propia.

La referencia *it* se vincula con `chocolate jello`, y *me* se conserva como destinatario de la entrega. El adaptador normaliza los espacios como guiones bajos, asigna tipos operativos y conserva el orden mediante `step`. En este caso no se activa la repara-

ción de navegaciones dirigidas a personas. El destino y la relación de entrega permiten seleccionar posteriormente la regla dirigida al operador.

El adaptador conserva el orden mediante el campo `step`; CLIPS utiliza ese campo para seleccionar la siguiente subtarea.

5.6.2. Aplicación de reglas y evolución del estado

La regla `goals-initialize` establece una ubicación desconocida, ausencia de un objeto sostenido, los contadores y la nota `GPSR_START`. Desde este punto, cada regla de procesamiento exige que no exista otro objetivo pendiente con un paso menor. La Figura 5.4 se lee de arriba hacia abajo; cada fila relaciona una regla, sus acciones operativas principales y el estado resultante.

La prioridad no permite adelantar una regla cuyas condiciones aún no se cumplen. Por ejemplo, la entrega tiene prioridad 190 y la adquisición habilitada, 181; sin embargo, la entrega debe esperar a que desaparezca el objetivo de adquisición con paso 3. El orden de los objetivos y la prioridad de las reglas cumplen funciones distintas.

La búsqueda mantiene la ubicación y el objeto sostenido; la adquisición modifica la posesión prevista. El estado utiliza `chocolate_jello`, mientras que las acciones utilizan el identificador operativo `chocolate_jello_1`. En la entrega, `me` se concreta mediante el retorno a `instruction_point` y la aproximación a `host_1`. Estos detalles operativos se incorporan mediante las reglas y no estaban expresados como objetivos independientes en la predicción.

5.6.3. Finalización y construcción de la salida

Tras la entrega, la ubicación esperada es `instruction_point`; el cierre añade `task completed` sin otro retorno. La organización del foco no añade operaciones porque el ejemplo no contiene diálogo dirigido. La serialización convierte los pasos finales en mensajes, ordena anuncios y operaciones y emite `GPSR_DONE`, según la Sección 5.4.5.

5.6.4. Plan resultante

La expansión de estos objetivos, suministrados directamente al adaptador y a las reglas vigentes, produjo los siguientes pasos. La comprobación no consultó Qwen ni utilizó servicios ROS:

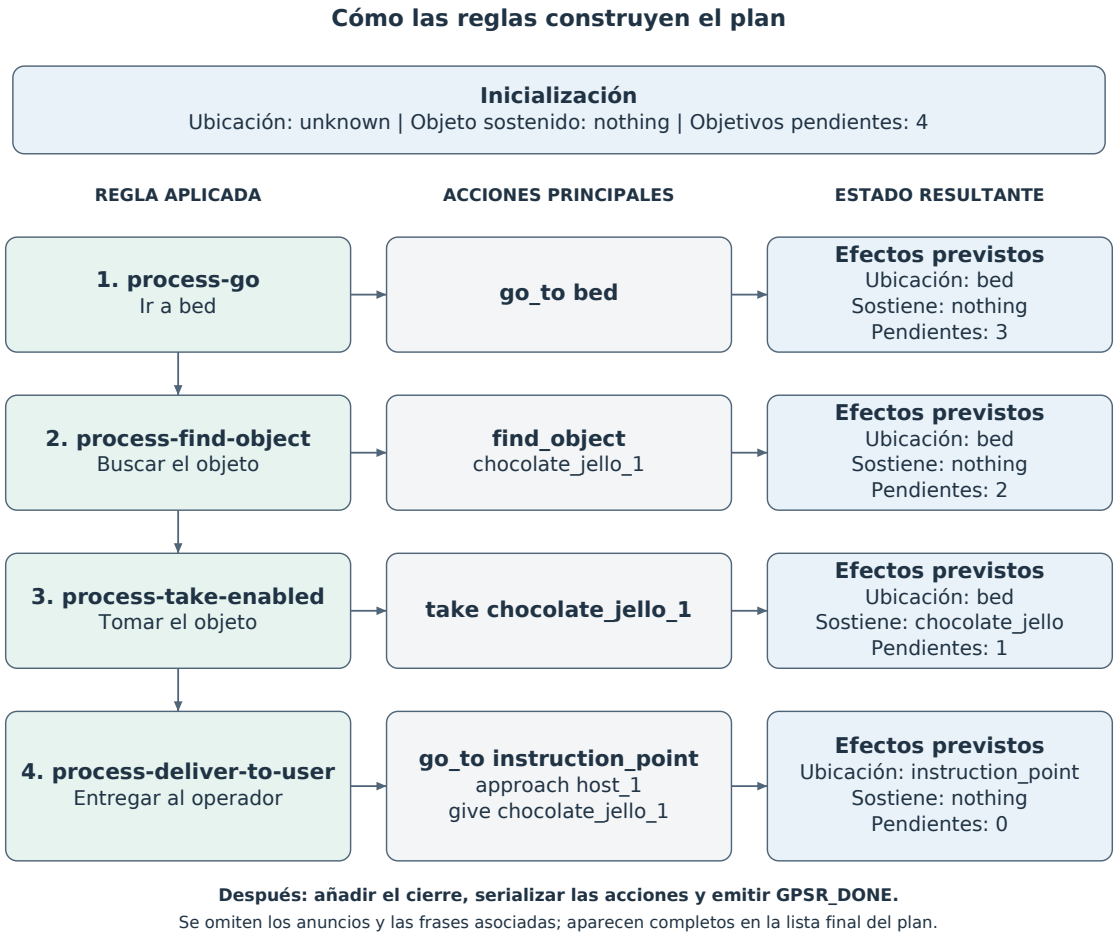


Figura 5.4: Expansión de los cuatro objetivos y evolución del estado simbólico. Se muestran las acciones operativas principales; los anuncios y las frases asociadas se conservan en el plan completo. Los cambios de estado representan efectos previstos durante la planificación. Elaboración propia.

```
==Justina GPSR_START==
Paso 1: Justina say I will navigate to bed
Paso 2: Justina say I will search for chocolate_jello
Paso 3: Justina say I will pick up chocolate_jello
Paso 4: Justina say I will return to the instruction point
Paso 5: Justina say I will deliver chocolate_jello to you
Paso 1001: Justina go_to bed
Paso 1002: Justina say I have arrived at bed
Paso 1003: Justina find_object chocolate_jello_1
Paso 1004: Justina take chocolate_jello_1
Paso 1005: Justina say I have picked up chocolate_jello
Paso 1006: Justina go_to instruction_point
Paso 1007: Justina approach host_1
Paso 1008: Justina give chocolate_jello_1
Paso 1009: Justina say I have delivered chocolate_jello
Paso 1010: Justina say task completed
==Justina GPSR_DONE==
```

Los cuatro objetivos generan quince pasos: cinco anuncios iniciales y diez instrucciones posteriores. Estas últimas comprenden seis acciones de navegación, percepción y manipulación, y cuatro instrucciones habladas. Las marcas de inicio y cierre no se cuentan como pasos.

El plan queda disponible para su publicación y consumo por el ejecutor. Las frases de llegada, adquisición y entrega forman parte de las instrucciones generadas; no representan observaciones recibidas durante la planificación. La verificación de que el robot realizó físicamente cada operación corresponde a la etapa de ejecución.

5.7. Instrumento de campañas experimentales

El paquete `experiments` permite preparar, ejecutar, reanudar y evaluar campañas sin ROS ni ejecución física. Distingue las condiciones `qwen3b_clips`, `qwen7b_clips` y `gpt_direct`; `reference_clips` constituye un control auxiliar, no una cuarta configuración competidora.

5.7.1. Preparación y conservación de la configuración

La operación `init` conserva entradas, referencias y recursos junto con un manifiesto. Este identifica condiciones, repeticiones, semilla de orden, límites de tiempo, modelos, parámetros y huellas. Las campañas copian el prompt, el esquema y las reglas de la campaña. Para Qwen se captura `/metadata`; antes de ejecutar se contrasta la configuración efectiva del servidor con la esperada. Cambiar una etiqueta de condición no

cambia el modelo cargado: debe seleccionarse el adaptador de 3B o 7B en el servidor correspondiente.

Las operaciones `run`, `check`, `evaluate`, `summary` y `view` separan ejecución, integridad, evaluación y consulta. El visor permite inspeccionar respuestas históricas sin consultar modelos; esa lectura no certifica reproducibilidad. Una campaña anterior no se reanuda con fuentes modificadas como si constituyeran la misma configuración.

5.7.2. Recorrido Qwen–CLIPS y control auxiliar

`experiments.backends` solicita la interpretación por HTTP, conserva el cuerpo recibido y extrae los objetivos antes de adaptarlos. `experiments.symbolic` utiliza las clases versionadas del adaptador y las reglas de `experiments/vendor` para construir hechos y expandir el plan. Su equivalencia con las fuentes de Justina se comprueba por separado, como se describió en la integración. Este recorrido difiere del validador del corpus, que no reproduce todas las heurísticas del adaptador de producción.

El entorno CLIPS se reinicia para cada ejecución con ubicación desconocida y ausencia de objeto sostenido, según la representación descrita en la Sección 5.4.2. El instrumento de campañas utiliza ese estado inicial fijo. Los supuestos adicionales del escenario delimitan el dictamen y no se convierten automáticamente en hechos nuevos del motor.

El control auxiliar introduce objetivos de referencia aprobados y aplicables por el mismo adaptador y reglas. Si no existe una referencia utilizable, se registra como control no realizado. Su plan se evalúa contra la tarea: una referencia aprobada no garantiza que la expansión sea correcta. Los criterios de cobertura y los denominadores siguen el Capítulo 4.

5.7.3. Productos y tratamiento de incidencias

Cada ejecución identifica campaña, configuración, caso, repetición e intento. La Tabla 5.5 reúne los productos que permiten localizar discrepancias entre etapas. El cuerpo HTTP de Qwen contiene los objetivos extraídos y no la generación neuronal completa. GPT conserva el texto de salida y la respuesta del proveedor disponible.

Se separan el estado técnico de ejecución, el tipo de respuesta y el dictamen semántico. Un proceso completado puede devolver un plan incorrecto; una aclaración bien formada no acredita que aclarar fuera lo adecuado. El instrumento registra fallos y

Tabla 5.5: Productos registrados y su función en la revisión de campañas.

Etapa	Producto	Finalidad
Configuración	Manifiesto, identificadores, copias y huellas.	Identificar las condiciones de cada ejecución.
Interpretación	Orden original, cuerpo de respuesta y objetivos recibidos; entrada normalizada en Qwen.	Examinar la interpretación antes de la adaptación.
Adaptación Qwen	Objetos adaptados, transformaciones y hechos CLIPS.	Distinguir normalización y reparaciones de la predicción original.
Planificación	Plan; en CLIPS, notas de cierre, número de reglas disparadas y metas restantes.	Revisar expansión y terminación.
Validación	Comprobaciones del contrato e incidencias.	Separar aceptación técnica y dictamen semántico.
Ejecución experimental	Tiempos, recursos observables, estado técnico y errores.	Conservar fallos y delimitar las mediciones.

timeouts y conserva los intentos anteriores al reanudar. Los reintentos manuales constituyen intentos nuevos y no sobrescriben el primero. La disponibilidad del servidor tras un timeout y los límites de recuperación se tratan en el recorrido experimental, sin atribuir ese mecanismo al cliente ROS.

5.8. Implementación del planificador GPT

El planificador directo se implementó mediante un cliente individual y un backend de campañas que comparten prompt y esquema. Recibe una instrucción en lenguaje natural y solicita en una misma respuesta los objetivos semánticos y el plan de acciones. El programa no utiliza CLIPS para construir el plan y no publica acciones al robot.

El componente se organiza en `planner_test_node.py`, un prompt estático, un esquema JSON y ejemplos del contrato. Su salida consiste en una representación estructurada del plan y un registro de la solicitud, que permiten analizar el comportamiento del modelo sin ejecutar físicamente las acciones.

5.8.1. Interfaz con la API

El programa utiliza el cliente de OpenAI y el método `responses.create`. La solicitud contiene un mensaje de sistema con las instrucciones del planificador y un mensaje

de usuario con la orden. Cada invocación realiza una solicitud independiente y no conserva una conversación de varios turnos.

Las campañas de desarrollo conservaron el identificador solicitado `gpt-5.6-terra`. Este identificador documenta las campañas registradas y no acredita por sí solo un snapshot inmutable del proveedor. El programa recibe su identificador mediante `--model` o la variable `OPENAI_MODEL`, sin fijar un modelo predeterminado en el código. El registro distingue el identificador solicitado del modelo informado en la respuesta, cuando el servicio proporciona este último.

El cliente `planner_test_node.py` permite comprobaciones individuales. Las solicitudes repetidas, la reanudación y el registro común de condiciones se implementan en el instrumento de campañas de la Sección 5.7.

El cliente configura un tiempo de espera de 60 segundos y desactiva los reintentos automáticos. El límite predeterminado de salida es 8 192 tokens y puede modificarse mediante `--max-output-tokens`. La petición utiliza `store=False`; el programa conserva su propio registro local. No especifica temperatura, `top_p` ni un parámetro de esfuerzo de razonamiento en la solicitud.

5.8.2. Instrucciones del sistema

El prompt estático vigente, identificado como versión 1.6, se elaboró en inglés. Define los catálogos de entidades, las formas canónicas de los doce objetivos semánticos, la resolución de referencias y las convenciones de normalización. También establece las dependencias entre búsqueda, adquisición, transporte e interacción.

Las instrucciones describen la expansión de cada objetivo en acciones y el mantenimiento de un estado previsto: ubicación, objeto sostenido, foco de interacción y referencias a personas. Especifican la separación entre anuncios y operaciones, la numeración de pasos y las condiciones de cierre. Los cambios de estado pertenecen a la construcción del plan y no representan observaciones del robot.

La versión vigente incluye la selección de superficies por coincidencias ordenadas y la organización global del foco entre operaciones de diálogo, con un único `task completed` al final. El historial de versiones y las huellas permiten distinguir estos cambios de las campañas anteriores; las pruebas locales del protocolo no demuestran que el modelo remoto cumpla todas las instrucciones.

El prompt se conserva separado de la orden y su huella SHA-256 se registra en cada solicitud para identificar el contenido utilizado.

5.8.3. JSON Schema y Structured Outputs

La solicitud incorpora el esquema mediante `text.format`, con tipo `json_schema`, nombre `gpsr_direct_plan` y `strict=True`. Los seis campos del objeto raíz son obligatorios y no se admiten propiedades adicionales.

Tabla 5.6: Contrato estructurado del planificador GPT directo.

Campo	Contenido
`status`	`executable`, `clarification` o `rejected`.
`message`	Pregunta o explicación; vacío para planes ejecutables.
`goals`	Lista de expresiones de objetivos semánticos.
`plan`	Lista de pasos con `step`, `robot`, `action` y `params`.
`start_note`	`GPSR_START` o cadena vacía.
`end_note`	`GPSR_DONE` o cadena vacía.

Cada paso exige un número entero, el identificador `Justina`, una acción del repertorio de catorce acciones y una lista de cadenas como parámetros. El repertorio comprende navegación, comunicación, búsqueda, análisis, aproximación, control del foco y manipulación. El esquema controla estos tipos y valores admitidos; la coherencia entre `status` y los demás campos, la numeración y el número de parámetros se comprueban adicionalmente en el programa.

5.8.4. Generación de objetivos y planes

Para una instrucción ejecutable, la respuesta contiene $G_{\text{pred}}^{\text{GPT}}$ y P^{GPT}. Los objetivos permiten evaluar la interpretación frente a G_{ref}, mientras que los pasos permiten analizar la secuencia de acciones y su correspondencia con las convenciones del ejecutor. Ambos elementos proceden de la misma respuesta del modelo.

El programa convierte `response.output_text` mediante `json.loads`, conserva el resultado y lo somete a las comprobaciones locales. La respuesta se imprime como JSON y se añade al registro de ejecuciones. En este programa de pruebas no se realiza una conversión posterior a mensajes ROS ni una entrega al ejecutor.

Los ejemplos incluidos ilustran una instrucción de navegación, búsqueda, adquisición y colocación, una solicitud de aclaración y un rechazo de una tarea fuera del repertorio. Se utilizan para verificar el contrato local y se distinguen de las respuestas observadas en solicitudes reales al modelo.

Las respuestas reales de desarrollo se conservan en sus campañas, junto con las versiones del prompt y del esquema utilizados. Se consultan como evidencia histórica; no se presentan como salidas de la versión vigente sin comprobar sus huellas.

5.8.5. Tratamiento de aclaraciones y rechazos

El estado `clarification` representa una pregunta para resolver información faltante o ambigua. El estado `rejected` representa una tarea fuera del dominio o de las capacidades admitidas. En ambos casos se exige un mensaje no vacío y objetivos, plan y marcas de inicio y cierre vacíos. El programa conserva e imprime esa respuesta; no implementa automáticamente un diálogo posterior.

Estos estados del contrato se distinguen de una respuesta incompleta, un rechazo comunicado mediante el campo `refusal` de la API, un error de comunicación o una salida que no supera la validación. Las incidencias se registran como fallos de la solicitud o del procesamiento, conservando la respuesta recibida cuando está disponible.

5.8.6. Validación determinista

El cliente individual y el backend de campañas realizan comprobaciones locales, con diferencias en su implementación y en la forma de registrar el resultado. En el cliente individual, `validate_result` aplica el esquema mediante `Draft202012Validator`. Para una respuesta ejecutable exige objetivos y plan no vacíos, mensaje vacío y las marcas `GPSR_START` y `GPSR_DONE`. Las expresiones de objetivos se revisan con el validador compartido.

Ese cliente comprueba la numeración, exige anuncios `say` desde 1 y operaciones desde 1001, y verifica un parámetro no vacío por paso y `say task completed` como última operación. Sin embargo, esta última comprobación no detecta por sí sola un cierre duplicado anterior. Para aclaraciones y rechazos exige mensaje y campos operativos vacíos. Su indicador `valid` informa de la aceptación por esos controles.

En las campañas, `validate_gpt`, del módulo `experiments.validation`, registra por separado la conformidad con el esquema de respuesta, la validez de los objetivos y la del contrato del plan mediante `response_schema_valid`, `goals_schema_valid` y `plan_contract_valid`. La comprobación del plan, implementada en `plan_issues`, exige un único cierre `task completed` al final de las operaciones y rechaza cierres prematuros o duplicados, sin repararlos.

Estos indicadores describen las comprobaciones implementadas. Una aclaración bien formada puede ser inadecuada para la tarea, y un plan aceptado puede incumplir una restricción de la orden. Por tanto, no sustituyen la revisión semántica ni demuestran satisfacción de todas las precondiciones. Las verificaciones pedidas al modelo en el prompt se distinguen de estos controles externos.

El cliente incorpora un modo de validación local de los tres ejemplos del contrato. Las pruebas unitarias examinan respuestas inconsistentes, numeración, parámetros, cierre, objetivos y registro de errores. Las comprobaciones de interfaz utilizan respuestas simuladas, sin solicitudes al servicio.

5.9. Reproducibilidad y alcance de los registros

5.9.1. Disponibilidad del código

La revisión pública consultada para documentar el código es `3e7fd13b5537` (21). Este enlace fija una versión de consulta, aunque la rama principal cambie. Las versiones utilizadas en cada experimento se identifican mediante sus manifiestos, copias y huellas; la revisión citada no se atribuye automáticamente a las campañas históricas.

5.9.2. Identidad de modelos, datos y recursos

Se conservaron identificadores de modelos, adaptadores, corpus, prompt y esquema mediante versiones, copias y huellas. En Qwen, las revisiones observadas durante inferencia no acreditan por sí solas una revisión fijada durante el entrenamiento. La huella del corpus identifica el archivo conservado y auditado. En GPT se distinguen modelo solicitado e informado por el proveedor. El Apéndice A.4 reúne los identificadores completos y los límites de su procedencia.

5.9.3. Entornos y configuración efectiva

El entorno documentado utiliza Ubuntu 24.04.5 LTS, Python 3.12.3 y ROS 2 Jazzy. El inventario de bibliotecas se conserva en la Tabla A.2, en el apéndice, y se distingue de los metadatos capturados por ejecución.

El servidor actual calcula un inventario al cargar el modelo. Su `/metadata` identifica la base, revisión observada, pesos del adaptador, tokenizador, normalizador, catálogos,

fuentes y parámetros. `effective_config_sha256` vincula ese inventario con las respuestas. El cliente experimental verifica la identidad antes de consultar y contrasta la huella recibida. El manifiesto registra la configuración; no reconfigura por sí mismo un servidor ya iniciado.

La inferencia Qwen compartida utiliza generación sin muestreo, hasta 128 tokens nuevos y penalización de repetición de 1.05. El cliente GPT individual configura 60 segundos, hasta 8 192 tokens y ausencia de reintentos automáticos; no envía temperatura, `top_p` ni esfuerzo de razonamiento. Las campañas conservan sus propios parámetros y tiempos, que no deben deducirse únicamente de esos valores predeterminados. Los valores informados por el proveedor se distinguen de los enviados explícitamente.

5.9.4. Tiempos, recursos y conservación de evidencia

El instrumento separa petición al modelo, extracción, adaptación, planificación y validación. El monitor del servidor observa CPU, RAM y GPU en la inferencia, excluyendo carga y espera del bloqueo; el del cliente describe su propio proceso y procesamiento local. La memoria GPU asignada por PyTorch se distingue de la reservada y de otras mediciones del dispositivo. Los recursos internos del proveedor GPT no se infieren a partir del consumo del cliente.

El uso de tokens, incluido el detalle de caché y razonamiento cuando se proporciona, se conserva con la respuesta GPT. El gasto observado o estimado se documenta por campaña y no se deduce solo de la latencia. La definición de ventanas temporales, denominadores y política de reintentos corresponde al protocolo experimental del capítulo siguiente.

En ROS, `PlanLogger` añade planes a `plan_execution.log` y vacía el archivo al iniciar el nodo. Ese registro de sesión no contiene por sí solo la orden, el identificador de solicitud ni los tiempos de cada etapa. Es distinto del diario persistente de campañas. El registro experimental conserva los fallos y las transformaciones anteriores a la evaluación para permitir revisar dónde se produjo una discrepancia.

Durante algunas comprobaciones CLIPS emitió `ENVRNMNT8`, relativo a memoria no liberada por completo al cerrar entornos. La incidencia se conserva sin afirmar ausencia de crecimiento de memoria en campañas prolongadas. La revisión local de equivalencia y el ejemplo simbólico descritos en este capítulo no consultaron modelos remotos.

Las campañas de desarrollo documentaron 60 órdenes, cinco repeticiones y tres condiciones, con 900 primeros intentos. Se conservan sus versiones y dictámenes como evi-

dencia de comprobación del instrumento. La evaluación independiente y sus resultados se tratan en los capítulos experimentales.

Capítulo 6

Diseño experimental y criterios de evaluación

Este capítulo establece cómo se compararán las variantes Qwen–CLIPS de 3B y 7B con GPT directo para contrastar las hipótesis del capítulo 1. Se especifican la organización de los casos, las condiciones de ejecución y los criterios para evaluar objetivos y planes. El diseño se apoya en las representaciones del Capítulo 3, las decisiones metodológicas del Capítulo 4 y el instrumento descrito en el Capítulo 5.

Se distingue el piloto ya realizado de la prueba final independiente. La organización propuesta para esta última comprende 300 órdenes y cinco repeticiones por configuración. Las cantidades se presentan como propuesta de diseño: las fichas finales se encuentran en construcción y su inclusión requiere revisar la tarea, las referencias y la cobertura. El capítulo define el procedimiento que guiará esa revisión y la campaña; los resultados de la evaluación se presentarán en el capítulo siguiente.

6.1. Objetivo y diseño de la comparación

La comparación principal examinará si cada configuración produce planes que satisfagan las subtareas, restricciones y dependencias de las órdenes del bloque resoluble. Cada variante Qwen–CLIPS se contrastará por separado con GPT directo. La comparación entre 3B y 7B permitirá examinar el cambio de intérprete dentro de un procesamiento simbólico común, sin anticipar que la variante de mayor tamaño obtendrá mejores resultados.

El diseño será pareado: las tres configuraciones recibirán las mismas órdenes, con un

contrato y criterios de tarea comunes. La unidad de observación será una ejecución experimental de una orden por una configuración en una repetición. La orden identifica el elemento comparado; el escenario agrupa formulaciones equivalentes y sus repeticiones, según las unidades de la Sección 4.2.4.

La fidelidad semántica, la robustez lingüística, la consistencia entre ejecuciones y el rendimiento computacional complementarán el cumplimiento. La hipótesis de robustez se examinará mediante reformulaciones revisadas para conservar la intención y elaboradas de manera independiente de las plantillas de entrenamiento. Repetir una salida estable no bastará para considerarla correcta.

La evaluación por comportamientos específicos se relaciona con el enfoque de CheckList (22); la distinción entre una respuesta plausible y un plan verificable se vincula con los problemas estudiados por PlanBench (23). En esta investigación se utilizará una rúbrica propia, basada en el contrato de Justina y en los requisitos de cada ficha. No se adoptan las tareas ni las puntuaciones de esos trabajos como si fueran directamente equivalentes a las presentes.

La comparación caracteriza sistemas completos. Entre Qwen–CLIPS y GPT directo cambian el modelo, su especialización y el mecanismo de generación del plan; las diferencias no aislarán el efecto causal de las reglas o del ajuste fino. Los planes se evaluarán sin ejecutarlos físicamente. La experiencia de integración de Qwen–CLIPS durante RoboCup 2026 conserva el alcance separado establecido en los capítulos anteriores.

6.2. Conjuntos de desarrollo y evaluación final

6.2.1. Antecedentes de desarrollo

El corpus de 40 000 muestras y su partición de entrenamiento y validación cumplen la función descrita en los capítulos 4 y 5. Las comprobaciones sobre esa validación no sustituyen una evaluación independiente. Asimismo, las 48 órdenes funcionales utilizadas para probar la integración se distinguen del piloto de comparación.

El piloto consolidado del 24 de octubre de 2026 contiene 60 órdenes, 28 intenciones, cinco repeticiones y tres configuraciones: 900 primeros intentos. Sus 48 órdenes agrupadas en 16 tripletas permiten estudiar variación lingüística; otras 12 órdenes cumplen funciones diagnósticas. Los diez casos iniciales forman parte del desarrollo de ese conjunto y no se cuentan como órdenes adicionales. La Tabla 6.1 resume la función de estos

antecedentes.

Tabla 6.1: Conjuntos utilizados antes de la evaluación final.

Conjunto	**Tamaño**	**Función**
Corpus supervisado	40 000 muestras	Entrenar y seleccionar los adaptadores Qwen mediante su partición de ajuste.
Prueba funcional	48 órdenes	Comprobar el recorrido de integración; se conserva su versión histórica.
Piloto comparativo	60 órdenes, 28 intenciones	Desarrollar el instrumento y los criterios; 900 primeros intentos entre las tres configuraciones.

Las auditorías de desarrollo identificaron coincidencias entre parte de las órdenes y el corpus, después de limpieza textual o de aplicar el normalizador. Cada auditoría corresponde a versiones concretas de los archivos; sus recuentos no se combinan como si fueran una única medición. Estos antecedentes se utilizan para desarrollar el procedimiento y no se presentan como evidencia de generalización sobre una prueba reservada.

Las referencias de desarrollo conservan estados de aprobación. Ese estado se distingue de la aplicabilidad de los objetivos al control CLIPS y de la aprobación de un dictamen sobre una respuesta concreta. El consolidado incluye revisiones propuestas con apoyo de IA y decisiones explícitas del investigador; no acredita una revisión humana completa de todas las salidas. Sus resultados se conservarán con esa procedencia.

Las campañas históricas mantienen sus respuestas, configuraciones y copias de recursos. En la campaña GPT de repeticiones con pausa se solicitó el identificador `gpt-5.6-terra`, con prompt 1.6, `max_output_tokens=8192` y `store=False`. Esos valores documentan desarrollo y no fijan la configuración final. Los cambios posteriores del prompt, adaptador o reglas requieren una campaña nueva para evaluar su efecto.

6.2.2. Bloques del conjunto final propuesto

La Tabla 6.2 organiza la propuesta en un bloque principal y tres bloques de diagnóstico. El primero permitirá contrastar la hipótesis de cumplimiento; los demás caracterizarán composición y comportamiento ante límites. Sus resultados se presentarán por separado.

Con cinco repeticiones por orden y configuración, la propuesta comprende 4 500 ejecuciones de los tres sistemas, de las cuales 2 700 corresponden al bloque principal. Los controles auxiliares y los reintentos se cuentan aparte. La cantidad definitiva y las cuotas

Tabla 6.2: Distribución propuesta para el conjunto final.

Bloque	**Cantidad propuesta**	**Propósito**
Principal	60 escenarios, tres formulaciones: 180 órdenes	Cumplimiento dentro de la cobertura declarada y robustez lingüística.
Composición	40 órdenes	Combinaciones de operaciones conocidas fuera de las rutas del generador local.
Límites de capacidad	40 órdenes	Restricciones representables o comprensibles que exceden alguna capacidad disponible.
Ambigüedad o contradicción	40 órdenes	Adecuación de aclaraciones, rechazos y reconocimiento de información insuficiente.
Total	300 órdenes	Evaluación principal y diagnósticos diferenciados.

se fijarán al cerrar el conjunto, antes de obtener respuestas finales. La propuesta no supone una cobertura uniforme de las 29 familias ni un tamaño calculado por potencia estadística.

En el bloque principal, cada escenario tendrá una formulación cercana al generador, una paráfrasis léxica y una paráfrasis sintáctica. Las tres conservarán entidades, subtareas y restricciones. Añadir acciones o cambiar condiciones produce otra tarea y no una reformulación equivalente. Las órdenes se mantendrán en inglés; una traducción española podrá acompañar la ficha para revisión, sin enviarse al modelo.

La inclusión en el bloque principal dependerá de que la tarea esté suficientemente especificada y sea evaluable bajo el contrato declarado, sin exigir que ambas arquitecturas la resuelvan. Una incapacidad de una configuración no justificará retirar el caso después de observar su salida. Cambiar únicamente el nombre o el objeto no acreditará una familia nueva.

6.2.3. Fichas, referencias y auditoría de cobertura

Cada ficha conservará identificadores de orden, escenario, intención y familia; bloque y procedencia lingüística; contexto; subtareas, restricciones y precedencias; respuestas admisibles; objetivos de referencia cuando correspondan; y criterios de aceptación. La revisión incluirá fecha, responsable, versión y resolución de discrepancias.

Se documentarán cinco aspectos de cobertura: admisión por la versión identificada

del generador original, admisión por la adaptación local, relación con el corpus y el desarrollo, representabilidad mediante objetivos canónicos y soporte en adaptación, reglas y contrato del ejecutor. La pertenencia a una plantilla no garantizará soporte de todos sus atributos. Cuando no haya evidencia para uno de esos aspectos, se conservará ese alcance sin convertirlo en una clasificación comprobada.

La auditoría distinguirá coincidencias literales, normalizadas y estructurales. Se excluirán los duplicados textuales detectados con el corpus y las órdenes del piloto, incluidas las reformulaciones de sus mismas instancias de tarea. Las transformaciones utilizadas para detectar duplicados se conservarán con la auditoría. Si se comparten familias entrenadas, las conclusiones se referirán a nuevas instancias y formulaciones de esas familias. Se desconoce el posible solapamiento con el entrenamiento GPT.

Las referencias se revisarán antes de consultar las respuestas finales. Una etiqueta automática podrá utilizarse como propuesta, pero no se considerará aprobada únicamente por haber sido generada o por expandirse en CLIPS. Cuando sean válidas varias representaciones o planes, se establecerán sus condiciones de equivalencia. Los modelos no recibirán objetivos de referencia ni dictámenes esperados.

La Tabla 6.3 muestra cómo vincular una orden con requisitos comprobables. Es una ficha ilustrativa; no se incorpora por ello al conjunto final ni constituye un resultado observado.

Tabla 6.3: Ejemplo ilustrativo de criterios de una ficha de evaluación.

Campo	**Contenido ilustrativo**
Orden	*Go to the kitchen, find an apple, take it and bring it to me.*
Condiciones	Estado inicial del planificador fijado; manzana disponible en la cocina; operador en el punto de instrucción; manipulación habilitada.
Subtareas	Navegar a la cocina, buscar la manzana, adquirirla y entregarla al operador.
Dependencias	Buscar antes de adquirir; adquirir antes de entregar; conservar el mismo objeto.
Referencia propuesta	`go(kitchen)`, `find(apple, kind=object)`, `take(apple)`, `deliver(apple, to=me)`.
Aceptación	El plan realiza todas las subtareas y cumple las convenciones del contrato, admitiendo secuencias alternativas válidas.
Fallo identificable	Entregar otro objeto, omitir la adquisición o dirigir la entrega a un destinatario distinto.

6.3. Configuraciones y condiciones experimentales

Las condiciones principales serán Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo. Se utilizará el instrumento de campañas de la Sección 5.7, con configuración y copias de recursos propias para la evaluación final. El cliente GPT individual y la prueba ROS por consola conservan su función de diagnóstico; no sustituyen el registro común de la campaña.

Antes de ejecutar se fijarán el conjunto y sus referencias, modelos base y adaptadores, tokenizadores, parámetros de generación, prompt y esquema, normalizador, catálogos, reglas, validadores y contrato. Se conservarán las huellas, las versiones de software y la plataforma de cómputo junto con las condiciones efectivas de cada ejecución. El identificador de GPT solicitado se distinguirá del informado por el proveedor y de un snapshot inmutable cuando no pueda acreditarse esa identidad.

Qwen utilizará el mismo preprocesamiento y recorrido simbólico para 3B y 7B. La selección de cada adaptador se comprobará en los metadatos del servidor. GPT recibirá la orden y su especificación fijada; sus objetivos no se enviarán a CLIPS para corregir el plan. La entrada original y la efectiva se conservarán, pues el normalizador Qwen forma parte del sistema comparado y puede resolver variaciones antes de la inferencia.

El estado inicial del planificador será ubicación desconocida y ausencia de objeto sostenido, conforme a la Sección 5.4.2. Los supuestos sobre ubicación de entidades, identidad de personas y disponibilidad de objetos se fijarán en las fichas. No se supondrá que todo contexto declarado se transforma automáticamente en hechos CLIPS. El conocimiento necesario para resolver la orden deberá estar disponible de forma coherente en cada configuración. La información de estado manejada por las herramientas físicas de Justina se distingue de esta comparación de planes.

6.3.1. Control auxiliar con objetivos de referencia

El control procesará referencias aprobadas y aplicables mediante el mismo adaptador, reglas y estado inicial de Qwen–CLIPS. Se registrará su plan por separado y se juzgará contra los requisitos de la ficha. No constituye una cuarta configuración principal ni una solución automáticamente correcta.

Se informarán referencias elegibles, controles realizados, terminados y correctos. La ausencia de referencia utilizable se clasificará como control no realizado, distinguiéndola de un fallo de adaptación o expansión. Las conclusiones del control se limitarán a su

cobertura efectiva. Para localizar una discrepancia se examinarán objetivos recibidos, transformaciones, hechos y acciones; la diferencia de resultado no basta por sí sola para aislar una causa.

6.4. Procedimiento de ejecución y revisión

6.4.1. Preparación y secuencia de la campaña

La preparación verificará el registro, las mediciones y la recuperación con datos de desarrollo. Los validadores se comprobarán con ejemplos correctos y con errores controlados, como omitir una adquisición, cambiar un destinatario o duplicar el cierre. Estas pruebas no utilizarán las órdenes reservadas para ajustar el sistema después de observar sus salidas.

Una vez fijadas las fichas y configuraciones, se seguirá este procedimiento:

1. Guardar el manifiesto de datos, criterios y recursos, y comprobar que las configuraciones activas coincidan con él.
2. Registrar la carga y el calentamiento por separado de las consultas medidas.
3. Procesar las órdenes en rondas con un orden aleatorizado reproducible y conservar la semilla y el calendario efectivos.
4. Registrar el primer intento de cada orden, configuración y repetición, incluidas respuestas inválidas, errores y tiempos agotados.
5. Ejecutar el control auxiliar donde corresponda, conservando su identificación independiente.
6. Aplicar las comprobaciones automáticas y la revisión semántica, guardar los dictámenes y resolver discrepancias sin borrar su historial.
7. Calcular métricas y comparaciones sobre los registros conservados, sin seleccionar los mejores intentos.

Las consultas serán secuenciales, independientes y sin conversación acumulada entre órdenes. Si el equipo requiere ejecutar los modelos Qwen por bloques para liberar GPU, se registrarán los cambios de modelo y los periodos de medición. El orden entre sistemas se alternará cuando sea viable; cualquier limitación se conservará para interpretar la latencia. Las rondas vinculan repeticiones comparables, sin implicar que los modelos compartan una semilla o un mecanismo de aleatoriedad idéntico.

6.4.2. Fallos, tiempos límite y recuperación

Los límites de petición, planificación y tiempo total se fijarán antes de la campaña y se distinguirán de los valores predeterminados descritos en implementación. Una ejecución iniciada que falle o agote su tiempo permanecerá en el registro. Un reintento constituirá un intento nuevo, con su propio tiempo y resultado; no reemplazará el primer intento en el análisis principal.

Después de un timeout Qwen se comprobará que el servidor esté libre mediante el mecanismo de actividad documentado. Una respuesta tardía no se atribuirá a otra orden. Si no puede establecerse la disponibilidad, se suspenderán las consultas hasta recuperar un estado verificable. Las pausas entre peticiones y la espera para liberar el servidor se registrarán aparte de la latencia individual y formarán parte del tiempo total de campaña. También se conservará cualquier reintento automático del cliente o SDK, si existe.

Una ejecución no iniciada se identificará como pendiente, sin convertirla en un fallo observado. Se informará cuántos registros estaban previstos, cuántos se iniciaron y cuáles quedaron sin ejecutar. La presentación de una campaña incompleta deberá conservar esa cobertura.

6.4.3. Revisión semántica y procedencia de los dictámenes

La revisión seguirá este orden: integridad y estado técnico; esquema y contrato; fidelidad de objetivos; cumplimiento del plan; y adecuación del tipo de respuesta. Cada dictamen conservará la evidencia del requisito comprobado, la versión de los criterios, fecha y responsable o herramienta.

La revisión principal requerirá aprobación humana de los dictámenes. Podrán agruparse salidas idénticas si coinciden también orden, contexto y criterios. La similitud de redacción no bastará para transferir una evaluación. La IA podrá organizar evidencia y proponer dictámenes, conservando su procedencia sin identificarlos como revisión humana.

Se contempla una segunda revisión independiente de una muestra estratificada del 20 % de las órdenes, incluyendo sus configuraciones y repeticiones. Es una medida propuesta, condicionada a disponer de otro revisor y a fijar el muestreo antes de la revisión. Los casos dudosos se examinarán adicionalmente y se distinguirán de esa muestra. Si se realiza, el acuerdo se calculará antes de conciliar discrepancias, informando su unidad

de análisis, porcentaje de acuerdo y distribución de categorías.

6.5. Criterios y métricas de evaluación

6.5.1. Dictámenes y cumplimiento principal

Se conservarán por separado el estado técnico, la validez de formato, la fidelidad de los objetivos, la coherencia del plan y el cumplimiento. La Tabla 6.4 establece la interpretación de los dictámenes. Un defecto observable determinará incumplimiento aunque otra parte del plan no pueda verificarse.

Tabla 6.4: Dictámenes de tarea y su tratamiento.

Dictamen	**Criterio**
Cumple	Satisface todas las subtareas, restricciones y dependencias con acciones compatibles bajo el contexto declarado.
Incumple	Existe una omisión, contradicción o argumento incompatible, o falta una respuesta válida cuando se exige plan. Se identifica el origen técnico o semántico.
No verificable	La evidencia o el contrato no permiten resolver una condición necesaria. No acredita éxito y se conserva en el denominador principal.
No aplicable al plan	La ficha exige otra clase de respuesta. Se evalúa su adecuación; esta categoría no se asigna porque un sistema haya fallado.

Se denota mediante $Y_{i,r}^{a}$ el resultado binario del primer intento de la orden i, repetición r y configuración a en el bloque principal:

$$Y_{i,r}^{a} = \begin{cases} 1, & \text{si se acredita cumplimiento simbólico completo;} \\ 0, & \text{si no se acredita ese cumplimiento.} \end{cases} \tag{6.1}$$

El valor cero incluye incumplimientos, errores, tiempos agotados y casos no verificables, manteniendo sus categorías separadas para el diagnóstico. Una aclaración o un rechazo innecesarios en este bloque tampoco acreditan cumplimiento. Un caso no verificable no se describirá por ello como un fallo semántico demostrado, sino como éxito no acreditado.

Sea $\mathcal{I}_a$ el conjunto de primeros intentos iniciados del bloque principal para la configuración a, y M_a su cardinalidad. La medida principal será:

$$\widehat{p}_a = \frac{1}{M_a} \sum_{(i,r)\in\mathcal{I}_a} Y^a_{i,r}. \tag{6.2}$$

La tasa se calculará cuando $M_a > 0$; sin ejecuciones iniciadas se informará como no estimable. Se publicarán numerador, denominador y cobertura de ejecución. Con las 180 órdenes y cinco repeticiones propuestas, una campaña completa aportaría 900 observaciones principales por configuración. Los casos sin iniciar se contabilizarán aparte. La falta de registros en alguna configuración se hará explícita al construir los contrastes pareados; no se sustituirá por una tasa calculada solo sobre planes válidos.

La cobertura parcial de subtareas se utilizará como diagnóstico. También se podrá informar cumplimiento sostenido: proporción de órdenes que cumplen en todas las repeticiones realizadas según el número fijado. Se restringirá a órdenes con todas esas repeticiones y se informará cuántas cumplen esa condición de cobertura.

6.5.2. Interpretación y coherencia interna del plan

Los objetivos se compararán con referencias aprobadas y aplicables. Se distinguirá coincidencia exacta de equivalencia semántica: la primera exige la lista canónica completa con orden y argumentos; la segunda admite únicamente alternativas justificadas que preserven la tarea. Una salida ausente o no interpretable no constituirá un acierto y se conservará en el total de ejecuciones que requieren esa interpretación.

La revisión identificará omisiones, adiciones, sustituciones de entidades, errores de tipo y pérdida de atributos o relaciones. Las listas de distinta longitud se alinearán por subtareas para localizar discrepancias, sin ocultar omisiones ni reordenamientos que violen dependencias. El universo de referencias aplicables y los denominadores de cualquier medida por objetivo o campo se informarán de forma explícita.

La coherencia interna se examinará frente a los objetivos declarados y al contrato. El cumplimiento se juzgará directamente contra la orden y su contexto. Por ello, un plan puede cumplir la tarea pese a contener objetivos declarados incorrectos, o ser coherente con una interpretación que omite parte de la orden. Se informarán ambos resultados y, como medida complementaria, la proporción que satisface simultáneamente interpretación y plan.

La aceptación del esquema y de las comprobaciones deterministas se registrará aparte, con el alcance descrito en la Sección 5.8. El estado técnico completado no se identificará automáticamente con un plan correcto.

6.5.3. Condiciones de escenario y límites de capacidad

Las equivalencias de identidad, ubicación y argumentos se establecerán antes de observar respuestas. Si una ficha declara que una única persona presente corresponde al destinatario, esa condición podrá utilizarse para juzgar su identificación. No se extenderá a escenarios con varias personas ni se utilizará para eliminar arbitrariamente atributos solicitados.

Las claves habladas que activan un protocolo, como las de seguimiento, se distinguirán de anuncios descriptivos. Su aceptación requerirá que el contrato documente la operación que activan; decir que se realizará una tarea no bastará para acreditar su realización simbólica. Se comprobarán las dependencias de adquisición y uso de información, sin asumir que mencionar una referencia de memoria implica haber obtenido su valor.

El conteo por vestimenta y la selección por peso ilustran límites del soporte descrito en implementación. Un conteo general no satisface un filtro solicitado, y una selección por tamaño no acredita una selección por peso. Las excepciones aceptadas en el piloto se conservarán como antecedentes históricos, sin trasladarlas automáticamente al conjunto final. Si una capacidad cambia antes de fijar la campaña, se registrará la versión y el nuevo alcance del contrato.

En ambigüedad y fuera de cobertura se evaluará la adecuación de las respuestas admitidas por la ficha. Cuando puedan aceptarse tanto un plan completo como el reconocimiento de un límite, se informarán resolución y reconocimiento por separado. Un plan parcial que omite una restricción no satisface ninguna de esas dos condiciones. Los fallos técnicos de Qwen–CLIPS no se identificarán como aclaraciones o rechazos adecuados.

6.5.4. Robustez y consistencia entre ejecuciones

La robustez se analizará comparando las tres formulaciones de cada escenario del bloque principal. Se informará cumplimiento por formulación y cuántos grupos conservan cumplimiento en las tres órdenes de una misma ronda. Con la propuesta actual serían 60 escenarios por cinco rondas; esas 300 combinaciones no son unidades independientes. El cumplimiento en las quince ejecuciones de un escenario permitirá examinar robustez sostenida.

La consistencia se medirá por orden al comparar sus repeticiones. Se registrarán

cambios de objetivos, plan, tipo de respuesta y dictamen. Se informará si la salida estable corresponde a un plan correcto, a una respuesta inadecuada o a un fallo repetido. El análisis de estabilidad utilizará órdenes con el número completo de repeticiones e indicará su cobertura.

Para los planes se distinguirá comparación literal y comparación de una proyección operativa fijada previamente. La utilizada en el piloto elimina numeración y anuncios `say` iniciados por *I will* o *I have*, conservando otras acciones, argumentos y claves de diálogo. Si se utiliza en la prueba final, su implementación se conservará con la configuración. Esa reducción no demuestra equivalencia funcional ni permite eliminar todas las acciones de comunicación, pues algunas activan protocolos. Las respuestas sin plan se compararán por clase y adecuación, además de las diferencias textuales que resulte pertinente describir.

6.5.5. Clasificación y diagnóstico de errores

Se distinguirán errores de interpretación, adaptación y expansión simbólica, además de fallos de transporte o procesamiento. En Qwen–CLIPS se examinarán los objetivos recibidos antes de la adaptación, los objetos adaptados, los hechos y el plan. Se contará con qué frecuencia cambian entidades, tipos, atributos, relaciones u orden y si se conserva el requisito de tarea. Una reparación correcta no se atribuirá como acierto original de Qwen.

El control auxiliar ayudará a localizar problemas posteriores a la interpretación, sin garantizar por sí solo la atribución causal. En GPT, objetivos y plan proceden de una misma respuesta: las discrepancias permiten identificar el producto incorrecto, sin reconstruir una etapa causal interna del modelo. Los ejemplos de error conservarán orden, configuración y evidencia suficiente para revisar el dictamen.

6.5.6. Latencia, recursos y costo observable

La latencia de extremo a extremo se delimitará desde el envío del intento hasta la obtención y procesamiento del resultado por el programa de campaña, incluyendo adaptación y planificación en Qwen–CLIPS. Se distinguirán, cuando sean observables, petición, extracción, adaptación, expansión y validación. La carga del modelo y el calentamiento se presentarán aparte. La ubicación del servidor y las condiciones de comunicación formarán parte del registro.

Se informarán número de observaciones, mediana, media, cuartiles y percentiles altos. Las respuestas completadas se distinguirán de errores y tiempos agotados. El tiempo límite de un timeout no se interpretará como tiempo real de finalización; los fallos rápidos tampoco demostrarán mayor eficiencia para resolver la tarea. Las pausas entre consultas se incluirán en el tiempo de campaña, separadas de la latencia individual.

Para Qwen se distinguirán RAM del proceso, memoria GPU asignada y reservada por PyTorch, y las demás mediciones disponibles del dispositivo. Se indicarán proceso observado, herramienta, unidad, ventana y frecuencia de medición. Las métricas del servidor no se sumarán indiscriminadamente a las del cliente ni se interpretará memoria como consumo energético. Para GPT, los recursos internos del proveedor se identificarán como no disponibles; el consumo del cliente se presentará por separado.

Se conservarán tokens de entrada y salida, detalles de caché y razonamiento cuando existan, identificador de respuesta y modelo informado. Los detalles no se sumarán de nuevo a los totales de tokens. El costo consultado o facturado se distinguirá de una estimación por tarifas y se asociará a la campaña y sus intentos. El acceso local no se describirá como costo total nulo: equipo y operación son componentes distintos del gasto por solicitud.

6.6. Análisis estadístico y presentación de resultados

Las comparaciones principales serán las diferencias de cumplimiento de Qwen2.5-3B–CLIPS frente a GPT y de Qwen2.5-7B–CLIPS frente a GPT. El contraste entre las dos variantes Qwen se informará por separado. Para cada par se conservará la correspondencia entre órdenes, escenarios y repeticiones. El signo y la magnitud de la diferencia se acompañarán de su incertidumbre; una diferencia positiva aislada no bastará para sostener una superioridad concluyente.

Se propone estimar intervalos de confianza del 95 % mediante remuestreo agrupado por escenario. En cada muestra se conservarán juntas sus formulaciones, repeticiones y configuraciones, y se recalculará la diferencia pareada. Si varias instancias comparten una intención estrecha, se examinará la sensibilidad al agrupamiento por intención o familia. El número de remuestreos, la semilla y la regla para interpretar los contrastes se fijarán antes del análisis. Las 4 500 ejecuciones propuestas no se tratarán como observaciones independientes ni como tamaño efectivo de la muestra estadística.

El informe mostrará el cumplimiento por configuración, familia y formulación, con denominadores y cobertura. Los bloques de composición, límites y ambigüedad tendrán resultados propios. No se construirá una única tasa global que mezcle cumplimiento de planes con adecuación de rechazos. Los análisis secundarios se identificarán como tales y no sustituirán la hipótesis principal después de observar los resultados.

Se presentarán también consistencia, distribución de latencias y recursos, frecuencia de errores y ejemplos trazables. Una matriz de confusión solo se utilizará si las fichas definen clases de respuesta comparables; cuando admitan varias clases correctas, se declarará su tratamiento. El resultado principal no se reducirá a reproducción literal de una expansión CLIPS.

6.7. Controles y alcance de las conclusiones

El conjunto final se reservará respecto del desarrollo local. Si se utiliza posteriormente para corregir el sistema, una nueva evaluación se identificará como otra versión. Las exclusiones seguirán criterios previos o incidencias justificadas, conservando su efecto en los denominadores; no se eliminarán casos porque revelen errores de una configuración.

El registro final vinculará protocolo, fichas, referencias, configuraciones, respuestas y dictámenes. Las correcciones de evaluación conservarán sus versiones y motivos. El cierre de datos y configuraciones se realizará antes de iniciar la campaña final, de modo que lo reportado corresponda al procedimiento realmente aplicado.

La revisión humana, el alcance del contrato y las condiciones del proveedor pueden limitar la evaluación. Se documentarán los controles efectivamente realizados y cualquier desviación. El conjunto reservado no demuestra por sí solo generalización a familias inéditas ni elimina el posible solapamiento con datos del proveedor. Los resultados caracterizarán el comportamiento simbólico de las configuraciones en el dominio examinado; la percepción, navegación, manipulación e interacción físicas requieren una evaluación diferente. Con estos límites, el capítulo siguiente presentará los resultados y las discrepancias observadas bajo el protocolo aplicado.

Capítulo 7

Resultados

7.1. Resultados de la especialización de Qwen

7.1.1. Comportamiento durante entrenamiento

7.1.2. Evaluación del modelo seleccionado

7.2. Resultados de interpretación semántica

7.2.1. Qwen

7.2.2. GPT

7.2.3. Comparación

7.3. Resultados de planificación

7.3.1. Qwen–CLIPS

7.3.2. GPT directo

7.3.3. Condición de control CLIPS

7.3.4. Comparación

7.4. Análisis de errores

7.4.1. Errores de interpretación

7.4.2. Errores de planificación

7.4.3. Casos representativos

Capítulo 8

Discusión

8.1. Interpretación de los resultados respecto a la hipótesis

8.2. Interpretación semántica

8.2.1. Especialización de Qwen

8.2.2. Capacidad de GPT

8.2.3. Efecto de la variabilidad lingüística

8.3. Planificación de acciones

8.3.1. Expansión determinista mediante CLIPS

8.3.2. Planificación generativa mediante GPT

8.3.3. Separación entre errores de interpretación y planificación

8.4. Comparación Qwen–CLIPS frente a GPT directo

8.4.1. Exactitud

8.4.2. Consistencia

8.4.3. Latencia

Capítulo 9

Conclusiones y trabajo futuro

9.1. Conclusiones principales

9.2. Cumplimiento de los objetivos

9.3. Evaluación de la hipótesis

9.4. Contribuciones de la investigación

9.4.1. Representación canónica de órdenes

9.4.2. Arquitectura Qwen–CLIPS

9.4.3. Comparación con planificación directa mediante GPT

9.4.4. Metodología para separar errores de interpretación y planificación

9.5. Trabajo futuro

Apéndice A

Procedimientos de operación e integración

Este apéndice reúne los detalles operativos del sistema descrito en el Capítulo 5. Los procedimientos corresponden a las interfaces y configuraciones documentadas; la confirmación de un servicio no acredita la ejecución física de las acciones.

A.1. Consulta del código y documentación

El repositorio `https://github.com/Biorobotica-UNAM/qwen` contiene el código de generación, entrenamiento, inferencia y comparación experimental (21). Su README explica instalación, configuración y comandos. La Tabla A.1 ofrece enlaces a la revisión pública consultada, identificada por el commit `3e7fd13b5537`.

Los adaptadores y checkpoints de `models/` son artefactos locales excluidos del control de versiones; su disponibilidad no se deduce del acceso al repositorio. Los modelos base se obtienen de sus fuentes originales. La integración ROS 2 reside en el espacio de trabajo de Justina, mientras que `experiments/vendor/` conserva el adaptador y las reglas utilizados por el recorrido experimental. Las dependencias, los datos y los pesos necesarios deben identificarse junto con la configuración de cada ejecución.

A.2. Comunicación ROS 2 y operación del sistema

El coordinador envía la orden en `Parser.Request.command` y recupera un string JSON en `Parser.Response.response`. Una respuesta aceptada contiene `input` y `goals`;

Tabla A.1: Ubicación de los componentes en el repositorio de código.

Componente	**Ubicación**
Generación de datos	`qwen_gpsr/generation`
Contratos, catálogos y normalización	`qwen_gpsr/domain`
Entrenamiento QLoRA	`qwen_gpsr/training`
Inferencia y servidor HTTP	`qwen_gpsr/runtime`
Planificador GPT directo	`chatgpt_planner`
Campañas y procesamiento simbólico	`experiments`
Reglas y adaptador versionados	`experiments/vendor`
Catálogos GPSR	`data/CompetitionTemplate`
Informes y registros	`reports`

`input` utiliza la entrada normalizada si está disponible y, en caso contrario, la orden enviada. Ante una excepción, el nodo devuelve `status=aborted` y la causa.

El cliente comprueba que haya una lista `goals` en el nivel superior antes de llamar al conversor. Este solicita la inserción por lote y después el inicio de planificación. Con el lanzamiento integrado, las interfaces corresponden a:

```
/planning/semantic_parser/semantic_parser
/planning/goals_to_clips/convert
/planning/clips/assert_facts_batch
/planning/clips/start_planning
/planning/clips/results
```

La conversión utiliza `clips_interfaces/srv/String`: recibe el JSON en `request.data` y devuelve un mensaje de estado en `response.data`. La inserción por lote utiliza el tipo `StringArray`, del mismo paquete. El inicio de planificación utiliza `std_srvs/srv/Trigger`.

Al recibir un lote, el nodo encola una tarea que borra su copia del plan actual, reinicia el entorno mediante `reset`, inserta los hechos recibidos y ejecuta el motor. La solicitud de inicio encola otra tarea que borra la copia del plan, inserta el disparador `(start (name action-planning))` y vuelve a ejecutar el motor. El reinicio del entorno se realiza en la tarea de inserción por lote; la tarea de inicio no reinicia el entorno y conserva los hechos cargados para la planificación.

El servicio por lote devuelve `successful=True` después de encolar la tarea. El servicio de inicio devuelve `success=True` y el mensaje `Planning started.` en ese mismo

punto del procedimiento. Estas respuestas confirman la aceptación de las solicitudes, no la terminación de la inserción ni de la planificación. Si una tarea falla posteriormente, el trabajador escribe el error en el registro ROS; el callback que ya respondió no comunica ese fallo mediante su respuesta original.

El adaptador espera hasta tres segundos por la disponibilidad de cada servicio dependiente y hasta cinco segundos por su respuesta. Rechaza una lista vacía y devuelve `OK` cuando ambas solicitudes reciben respuestas favorables. Estos límites corresponden a la comunicación de cada operación y no establecen un tiempo máximo global de generación del plan.

Después de ejecutar el motor, el nodo busca hechos `ros-message`. Si existen, solicita su extracción y publica la salida no vacía mediante `std_msgs/msg/String`, con una instrucción por línea. La obtención del plan se verifica sobre esa publicación y su contenido, por separado del estado devuelto por los servicios. El tópico publicado tiene el nombre relativo `clips/results`.

A.2.1. Recepción del plan y entrega de instrucciones

El cliente `qwen_to_plan.py` se suscribe al tópico de planes antes de enviar la orden. Imprime la respuesta semántica, el estado de conversión y el plan recibido. Puede ejecutarse desde la raíz del workspace Justina, con el entorno ROS cargado:

```
ros2 launch planning_tools planning.launch.xml
python3 src/Planning/planning_tools/scripts/qwen_to_plan.py \
  "go to the kitchen and find an apple"
```

El lanzamiento y el cliente se mantienen en terminales independientes. El servidor Qwen debe estar iniciado y accesible mediante `QWEN_SERVER_URL`. El launch no inicia el servidor, el listener, el gestor de instrucciones ni el ejecutor físico. Las reglas se localizan respecto al directorio de trabajo, por lo que el arranque desde la raíz del workspace forma parte del procedimiento.

El cliente espera, por defecto, diez segundos por servicio o respuesta y cinco por el plan. El HTTP del launch integrado utiliza cinco segundos, frente a los quince del launch semántico aislado. Aumentar la espera del cliente no modifica el timeout HTTP. La suscripción conserva el último texto recibido, sin identificador que lo vincule con la orden; por ello, este flujo se utiliza secuencialmente, con una orden a la vez.

La biblioteca C++ `Planning` ofrece los clientes `parseSemanticCommand`, `listen GpsrCommand` y `goalsToClips`. Los dos primeros devuelven objetivos y no inician CLIPS por sí solos; el coordinador solicita la conversión después. El servicio de escucha obtiene texto de `/hri/listener/listen` y lo envía a Qwen, con tiempos de escucha y traducción sucesivos. Los errores del nodo distinguen `gpsr_unavailable`, para fallos de transporte y timeout, de `semantic_error`, para otras excepciones. Ningún código acredita por sí solo que la orden sea inválida.

El componente `plan_manager_node` recibe el plan y entrega una instrucción en cada petición. Conserva nombres absolutos históricos; para conectarlo con las rutas de `Planning` debe iniciarse aparte:

```
ros2 run plan_manager_node plan_manager_node --ros-args \
  -r /clips/results:=/planning/clips/results \
  -r /request_instruction:=/planning/request_instruction \
  -r /instruction_msg:=/planning/instruction_msg
```

Asignar un namespace no modifica esos nombres absolutos. El gestor elimina la numeración textual, avanza al entregar cada instrucción y responde `NoPlanAssigned` cuando ya no tiene un plan activo. No espera confirmación de que la acción entregada se haya completado. El cliente de generación termina en el plan y no llama a los controladores.

A.3. Inicio del servidor Qwen

La inferencia puede ejecutarse en el equipo del robot o en una computadora remota. La entrada `server.py` delega en `qwen_gpsr.runtime.server`; mantiene residentes el modelo, el tokenizador y el normalizador. El cliente ROS envía la orden por HTTP, mientras que la conversión a hechos y CLIPS permanecen en el entorno de Justina. Las campañas experimentales utilizan su propio proceso CLIPS.

La ruta del adaptador se selecciona explícitamente al iniciar el servicio. Por ejemplo, desde la raíz del repositorio:

```
python3 server.py --adapter-path models/nl2cd_qwen3b \
  --host 127.0.0.1 --port 8008
```

Para 7B se utiliza `models/nl2cd_qwen7b`, que también es la ruta predeterminada cuando se omite `--adapter-path`. La dirección de escucha y la autenticación se configuran según el emplazamiento del cliente; `127.0.0.1` solo admite conexiones locales. La clave, si se utiliza, se proporciona fuera de los registros mediante `QWEN_API_KEY` y el encabezado Bearer. El servidor carga el adaptador indicado, sin comparar checkpoints al arrancar.

A.4. Identidad de modelos, datos y recursos

Los modelos base son `Qwen/Qwen2.5-3B` y `Qwen/Qwen2.5-7B`. Los metadatos conservados de inferencia identifican las siguientes revisiones, respectivamente:

```
3B: 3aab1f1954e9cc14eb9509a215f9e5ca08227a9b
7B: d149729398750b98c0af14eb82c78cfe92750796
```

Estas revisiones identifican las bases observadas en las campañas registradas. El campo `revision` de los adaptadores es nulo y los scripts no fijan ese argumento; por tanto, la evidencia no acredita una selección explícita de esas revisiones durante el entrenamiento. Los estados de `Trainer`, argumentos y pesos seleccionados se conservan como productos distintos, según la Sección 5.3.4.

El corpus compartido está en `data/datasets/dataset_gpsr.jsonl`. La huella SHA-256 del archivo auditado es:

```
0695f400370b93abbe97fc403a2acaa9f2d6bc8241667788bd466c6ad7c1c406
```

Se utilizó el mismo corpus en ambos entrenamientos; la huella actual identifica el archivo conservado, pero no corresponde a una medición realizada durante cada corrida. La auditoría posterior de las etiquetas y del prefijo comprobado con CLIPS tampoco sustituye un reporte original de generación que no se conservó.

En GPT se identifican por separado el modelo solicitado, el informado en la respuesta, la fecha, el prompt y el esquema. La versión nominal se complementa con huellas y copias de recursos; el prompt vigente no se atribuye a todas las campañas históricas. El manifiesto de cada campaña identifica el modelo y los parámetros efectivos utilizados.

A.5. Inventario de software

El inventario conservado del entorno empleado en el trabajo con 7B incluye Ubuntu 24.04.5 LTS, Python 3.12.3 y ROS 2 Jazzy. La Tabla A.2 recoge las bibliotecas regis-

tradas. El inventario se distingue de una captura por ejecución y no se extrapola a cualquier servidor remoto ni al ajuste de 3B sin su registro asociado.

Tabla A.2: Bibliotecas del inventario de software conservado.

Componente	**Versión registrada**
Controlador NVIDIA	580.178.04
PyTorch / CUDA de su compilación	2.9.0+cu130 / 13.0
Transformers	4.51.3
PEFT	0.18.1
bitsandbytes	0.49.2
Accelerate	1.13.0
Datasets	4.8.4
Tokenizers	0.21.4
clipspy	1.0.5
Cliente OpenAI	2.8.1
jsonschema	4.25.1
NumPy	1.26.4

Referencias

[1] J. Savage, D. A. Rosenblueth, M. Matamoros, M. Negrete, L. Contreras, J. Cruz, R. Martell, H. Estrada, and H. Okada, "Semantic reasoning in service robots using expert systems," *Robotics and Autonomous Systems*, vol. 114, pp. 77–92, 2019.

[2] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.

[4] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.

[5] S. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, "ChatGPT for Robotics: Design Principles and Model Abilities," *arXiv preprint arXiv:2306.17582*, 2023.

[6] J. Hart, A. Moriarty, K. Pasternak, J. Kummert, M. Leonetti, L. Contreras, L. Ruegemer, A. Mitzutani, T. Ribeiro, A. Golding, T. Kang, and F. Pimentel, "RoboCup@Home 2026: Rules and Regulations." `https://github.com/RoboCupAtHome/RuleBook/releases/tag/2026.2`, 2026. RoboCup@Home Technical Committee.

[7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.

[8] Secret Society Software, LLC, *CLIPS Reference Manual, Volume I: Basic Programming Guide*, version 6.4.2 ed., Jan. 2025. Documentación de referencia; no identifica la versión de ejecución del experimento.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Informa-*

tion Processing Systems, vol. 30, pp. 5998–6008, 2017.

[10] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[13] JSON Schema, "JSON Schema Specification." `https://json-schema.org/specification`. Consultado el 28 de octubre de 2026.

[14] A. d'Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, "Neural-symbolic learning and reasoning: Contributions and challenges," in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.

[15] T. R. Besold, A. d'Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, "Neural-symbolic learning and reasoning: A survey and interpretation," *arXiv preprint arXiv:1711.03902*, 2017.

[16] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[17] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," *arXiv preprint arXiv:2209.07753*, 2022.

[18] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[19] RoboCup@Home, "CommandGenerator: Command Generator for RoboCup@Home." `https://github.com/RoboCupAtHome/CommandGenerator`, 2026. Repositorio oficial de RoboCup@Home.

[20] RoboCup@Home, "CompetitionTemplate." `https://github.com/RoboCupAtHome/CompetitionTemplate`, 2026. Repositorio oficial de RoboCup@Home.

[21] Laboratorio de Bio-Robótica, UNAM, "Qwen–CLIPS para instrucciones GPSR: código y documentación." `https://github.com/Biorobotica-UNAM/qwen`. Revisión consultada: `3e7fd13b5537`. Consultado el 5 de noviembre de 2026.

[22] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912, Association for Computational Linguistics, 2020.

[23] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, "PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.